

claude-windows / 02b9e9c9-3806-45ca-9b24-2ecc35a81efd

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\subagents\workflows\wf_27df53c8-8e0\agent-a57770743bb570f09.jsonl`  
- SHA-256: `fa388ef742df21e18d373f3c3021dcfd2187f6eda9878c5b12eb9a5b06abb5be`  
- Source modified: `2026-06-10T16:23:06+00:00`  
- Imported at: `2026-07-05T16:48:21+00:00`  
- project: `wf_27df53c8-8e0`  
- session_id: `02b9e9c9-3806-45ca-9b24-2ecc35a81efd`
```

Transcript

1. user (2026-06-10T16:13:27.796Z)

You are auditing the DESIGN DOCS, ROADMAP, and doc-vs-code consistency of muneem (C:/Users/avidu/Projects/muneem). The user (Avi) drives design via docs and parks open decisions explicitly; docs are load-bearing.

SENSITIVE-DATA RULES (mandatory, highest priority):

- testdata/aubank-statements/, testdata/axis-statements/, testdata/hdfc-statements/, testdata/snapshots/, and testdata/credentials/ in the muneem repo contain the user's REAL financial documents and a real OAuth client file. Do NOT open/read the PDFs. You MAY check file existence and git-tracking status (git ls-files, git log --all -- <path>).
- If you must inspect testdata/snapshots/*.json or testdata/credentials/*.json to assess sensitivity, read at most the JSON key structure and NEVER quote values (no account numbers, balances, person names, client secrets, transaction amounts) in your output.
- Never include sensitive values in your findings. Refer to them abstractly ("a real account number", "a client_secret value").
- Do not use WebSearch/WebFetch and do not send any repo content to any external service.

Read COMPLETELY: README.md, DESIGN.md, docs/ROADMAP.md, docs/CURRENT_STATE.md, docs/parser-architecture.md, docs/schema.md, docs/ingesting-statements.md, docs/onboarding-a-bank.md, docs/encryption-at-rest.md, docs/feedbackJune2026.md, docs/prior-art-email-mining.md, docs/research-open-banking-india.md, docs/TESTING.md, docs/DEPENDENCIES.md, testdata/promo-emails/README.md, CLAUDE.md.

Then spot-check the claims against the code (src/muneem) ? e.g.: does the CLI actually expose the commands CURRENT_STATE lists (read cli.py)? does schema.md match db.py's actual DDL? are Outlook/IMAP marked experimental in code as promised? is the provenance table (PR #46) reflected in schema.md? does parser-architecture.md match engine.py/profiles.py reality?

Assess:

1. Internal contradictions between docs; stale sections (which doc lags reality the most).
2. Roadmap critique: the phase plan in feedbackJune2026 §4 (A=recovery/foundation done ? B=provenance done ? C=instrument family ? D=regression CI ? E=L3 vision ? F=fetch ? G=merchant canon ? H=ops). Is the sequencing sound? What is the riskiest/most-mis-scoped phase? What's MISSING entirely from the roadmap (e.g. a query/reporting surface the user would actually use daily, data-entry for cash, equity broker statement reality-check, multi-year archive strategy, what 'done' looks like)?
3. The relationship with the sister repo gmail-attachments-to-gdrive (C:/Users/avidu/Projects/gmail-attachments-to-gdrive): its VISION.md claims it is THE ONLY code that touches Gmail and that muneem 'never holds the Gmail grant' and receives only minimized non-PII signal ? yet muneem has its own full Gmail OAuth ingestion (src/muneem/ingestion/gmail.py, tools/gmail_auth.py) and a planned 'muneem fetch' (Phase F). Read both sides and characterize this contradiction precisely: which roadmap is stale, what are the options to reconcile (muneem keeps its own ingest vs consumes the handoff), and what each option costs. Note that bank-statement PDFs are PII ? can they even flow through the sister repo's 'non-PII minimized signal' contract?
4. Open-decision discipline: are the still-open decisions (log format, self-hosted runner,

macOS) actually still open in code, or silently locked?

5. Doc debt that would bite a restart-from-zero session (the docs are the project's memory).

Return the structured report. The 'map' field must compactly state: the DESIGN.md vision + five-stage pipeline, the open decisions list and their current status, what ROADMAP.md tracks vs feedbackJune2026 phases, and the prior-art lessons. Structured output only.

2. user (2026-06-10T16:13:34.222Z)

```
1      # muneem
2
3      **Your automated muneem ? a personal-finance bookkeeper that pulls financial documents
from your inbox, unlocks them, and builds a local consolidated ledger.**
4
5      In traditional Indian households a *muneem* (?????) was the trusted bookkeeper who
tracked every statement, bill, and transaction on the family's behalf. This project is the same
idea, automated and personal.
6
7      **Scope is the whole financial picture, not just banks.** Bank and credit-card
statements are the first vertical slice; **equities, mutual funds, ETFs, and bonds are core
targets**, parsed into the same normalized local ledger (with transaction/instrument
classification to follow). The parser core is deliberately *family-agnostic* ? adding an
instrument family is new data + a reconciliation rule, not a new pipeline.
8
9      ## What it does (intended)
10
11     Reads Gmail and Outlook for financial documents (bank statements, credit-card bills,
brokerage statements, mutual-fund reports, utility bills, grocery and shopping receipts),
decrypts the password-protected ones, parses them into a normalized local database, and surfaces
insights:
12
13     - Grocery/shopping price-tracking across merchants over time
14     - Investment portfolio timeline and cross-broker view
15     - Bank/card performance comparison
16     - Catalog of purchased items with their bills (warranty, returns, age)
17     - Forgotten deals and coupons sitting unread in your inbox
18
19     ## Status
20
21     **Early implementation.** The design and plan are settled ? see
[DESIGN.md](./DESIGN.md) (architecture + threat model), [docs/ROADMAP.md](./docs/ROADMAP.md)
(milestones), and [docs/TESTING.md](./docs/TESTING.md) (test strategy). The Python project
skeleton, CLI, and offline CI are in place; build proceeds by the roadmap's milestones. Prior
art: [docs/prior-art-email-mining.md](./docs/prior-art-email-mining.md).
22
23     **Ingesting bank statements** (HDFC / Axis / AU Bank, with automatic password unlock):
[docs/ingesting-statements.md](./docs/ingesting-statements.md).
24
25     ## Getting started
26
27     Runs on Linux, macOS, and Windows. Requires **Python ? 3.11** and git.
28
29     ### Linux / macOS
30
31     ```bash
32     python3 -m venv .venv
33     source .venv/bin/activate
34     pip install -e ".[dev]"
35
36     muneem --version
37     muneem info
38     muneem ingest testdata/promo-emails/dataquest.pdf    # -> JS231sz
39     muneem offers list
40     ```
41
```

```

42 Or use the `Makefile`: `make install`, `make test`, `make lint`, `make run`.
43
44 ### Windows (PowerShell)
45
46 ```powershell
47 python -m venv .venv
48 .venv\Scripts\Activate.ps1
49 pip install -e ".[dev]"
50 muneem info
51 ```
52
53 ### Optional: OCR for codes inside banner images
54
55 The `--ocr` flag uses Tesseract (Apache-2.0, fully local ? no paid API):
56
57 | OS | Install |
58 |---|---|
59 | Debian / Ubuntu | `sudo apt-get install tesseract-ocr` |
60 | Fedora | `sudo dnf install tesseract` |
61 | macOS | `brew install tesseract` |
62 | Windows | `winget install UB-Mannheim.TesseractOCR` |
63
64 muneem finds it on `PATH`, at the usual install locations, or via the
`MUNEEM_TESSERACT_CMD` env var. Then `muneem ingest <pdf> --ocr` (run OCR tests with `make test-ocr`).
65
66 ### Notes
67
68 - Headless Linux: secret storage (used once Gmail/unlock land) needs a keyring
backend ? a desktop keyring (GNOME Keyring / KWallet), or `pip install keyrings.alt` for a file
backend. Not needed yet.
69
70 - CI runs the offline suite on Linux across Python 3.11?3.13 on every push.
71
72 ## Privacy
73
74 Single-user, fully local by design. All raw statement data stays on the machine. See
`DESIGN.md` § Security for the threat model and standing rules.

```

3. user (2026-06-10T16:13:34.750Z)

```

1 # muneem ? Design Document
2
3 > Status: draft, pre-implementation. This document captures the vision, the pipeline
shape, the security posture, and the open decisions to resolve before code lands. It is
intentionally opinionated where the author has a strong view, and explicitly undecided where the
trade-offs need a human call.
4
5 ---
6
7 ## 1. What muneem is
8
9 muneem is a personal-finance bookkeeper that ingests financial documents from the
user's email accounts, unlocks the password-protected ones, parses them into a normalized
schema, stores them in a local database, and surfaces analytics across the user's full financial
life.
10
11 The name is the Hindi word for the traditional household/business bookkeeper ? a trusted
figure who maintained the family ledger by hand. This project is the same role, automated.
12
13 Single user. Local-first. Privacy-critical.
14
15 ## 2. Why
16
17 The user's financial life today is fragmented across many silos:

```

18

19 - 325 bank accounts (statements arrive monthly as password-protected PDFs)

20 - Multiple credit cards (monthly bills, also password-protected PDFs)

21 - Brokerage and mutual-fund accounts (quarterly statements, portfolio snapshots)

22 - Grocery and shopping receipts (Swiggy, Zomato, Amazon, Flipkart, BigBasket, Blinkit ?
 mostly plain HTML/PDF in email body)

23 - Utility bills, insurance premiums, EMI schedules

24 - Promotional offers and coupons (often forgotten)

25

26 Each silo has its own portal, login, and format. There is no consolidated view. ****The
 inbox is the only place where they already converge.****

27

28 Concrete use-cases the user wants once this exists:

29

30 1. ****Grocery & shopping price intelligence**** ? track what was bought, where, and at what
 price, over time. Detect when a merchant is no longer competitive on staples.

31 2. ****Investment portfolio timeline**** ? a single time-series of holdings across brokers,
 so allocation drift and rebalancing decisions are obvious.

32 3. ****Bank/card comparison**** ? fees paid, interest earned, reward redemption efficiency,
 surfaced as objective numbers.

33 4. ****Item catalog**** ? every significant purchase linked to its receipt/invoice, so
 warranty windows, return windows, and replacement cycles are queryable.

34 5. ****Forgotten-deals reminder**** ? surface the unused coupons and offers already sitting
 in the inbox (this is the original goal of the predecessor project; see § 11).

35

36 **## 3. Origin**

37

38 muneem is the successor to `~/Projects/Email-Mining`, a small prototype the user built
 earlier that extracted promo codes from manually-saved email PDFs using PyMuPDF + Gemini for
 image OCR. Same insight ("the inbox contains structured value I'm forgetting"), generalized
 scope.

39

40 See [`docs/prior-art-email-mining.md`](./docs/prior-art-email-mining.md) for what it
 did, what to inherit, and what not to.

41

42 **## 4. The pipeline (five stages)**

43

44 Each stage is independent enough to develop and test in isolation. They communicate
 through well-defined data boundaries (raw files on disk ? unlocked files on disk ? structured
 records in DB ? derived views).

45

46 **### Stage 1 ? Email ingestion**

47

48 ****Goal:**** pull messages and attachments from Gmail and Outlook into a local staging
 area.

49

50 - ****Gmail:**** [Gmail API](https://developers.google.com/gmail/api) over OAuth2. Read-only
 scope (`gmail.readonly`). Supports server-side search queries (`has:attachment filename:pdf
 from:hdfc newer_than:90d`) which is far cheaper than pulling everything and filtering locally.
 Has good rate limits for personal use.

51 - ****Outlook:**** [Microsoft Graph API](https://learn.microsoft.com/en-
 us/graph/api/resources/mail-api-overview) over OAuth2. Read-only scope (`Mail.Read`). Similar
 search capability.

52 - ****Why not IMAP:**** worse search, no labels, harder OAuth, and we lose Gmail's powerful
 query syntax. Use the native APIs.

53

54 Mechanically straightforward. The main design questions are: ****how is "financial
 document" detected**** (sender allowlist? subject patterns? attachment heuristics? hybrid?), and
****how is incremental sync state tracked**** (Gmail `historyId` / Graph `deltaLink`).

55

56 **### Stage 2 ? Attachment unlocking**

57

58 ****Goal:**** decrypt password-protected PDFs.

59

60 The decryption itself is trivial ? every mainstream PDF library handles the standard

```

security handler. In Go, `github.com/pdfcpu/pdfcpu`; in Python, `pikepdf` or `pypdf`.
61
62     The real design question is **where the passwords come from.** Indian bank and CC
statement passwords are almost always derived from user attributes:
63
64     - "First 4 letters of name (uppercase) + DDMM of DOB" ? common at HDFC, Axis
65     - "PAN number" ? common at some brokers
66     - "Last 4 digits of card number" ? common for CC statements
67     - "Customer ID" ? some banks
68     - "Mobile number" ? utilities
69
70     Three plausible approaches, each with trade-offs:
71
72     | Approach | Pro | Con |
73     |---|---|---|
74     | **Per-sender rule template** the user configures once (e.g., `from:alerts@hdfcbank.net`
? first4(name)+ddmm(dob)) | Deterministic, no guessing, auditable | Requires upfront setup;
rules must be maintained as banks change formats |
75     | **Candidate dictionary** (try a small list of derived strings per PDF) | Zero config;
"just works" | Brute-forceable feel; fails noisily on unknown senders |
76     | **Interactive prompt** (ask the user the first time, cache for that sender) | Safe,
simple | Friction during initial backfill of years of statements |
77
78     A hybrid is likely best: candidate dictionary first, fall back to rule template, fall
back to interactive prompt. **Decision parked.** See § 8.
79
80     ### Stage 3 ? Document parsing
81
82     **This is the hard 80% of the project.** Every issuer's PDF layout is different and
changes over time.
83
84     The space of approaches, in increasing robustness and cost:
85
86     1. **Per-issuer regex/template parsers.** Precise; brittle. One parser per format,
breaks when the issuer redesigns. Realistic for the top ~5 senders that dominate volume.
87     2. **Text-layer extraction + structural heuristics.** Pull text with `pdfminer.six` /
`pdfplumber` (both MIT); rely on coordinates and font runs to recover tables. Works well for
"born digital" PDFs (most bank statements); fails on scanned/image PDFs. (We avoid
PyMuPDF/`fitz` ? it is AGPL; see § 5 rule 8.)
88     3. **OCR (Tesseract) for image-based PDFs and embedded raster content.** Slower; need
layout reconstruction.
89     4. **LLM-assisted structured extraction.** Hand page text (+ optionally page rasters) to
a model with a target JSON schema. Generalizes across layouts; expensive per page; **must be
local for sensitive document categories**. Options: Ollama-hosted models (Llama 3, Qwen,
Mistral), or a small purpose-built model.
90
91     In practice this will be **a hybrid**: deterministic template parsers for the top ~5
high-volume senders (where they pay for themselves), LLM-assisted extraction as the long-tail
fallback. Promo emails and other non-sensitive categories can use cloud LLMs;
bank/card/investment statements **must not** without explicit per-category approval.
92
93     Embedded image deduplication (Email-Mining used MD5 hash + SSIM) is a real win here:
bank statements repeat the same logo/letterhead on every page, and we don't want to OCR that 30
times per document.
94
95     ### Stage 4 ? Normalization & storage
96
97     **Goal:** map heterogeneous parser outputs into one schema, persist locally.
98
99     Storage: **SQLite**. Single file, no server, easy to back up, easy to encrypt at rest
(SQLCipher), good enough performance for years of personal data. The schema needs to span:
100
101     - `documents` ? every ingested file with sender, date, type, hash, path
102     - `accounts` ? bank accounts, cards, broker accounts, identified by issuer + last-4 or
similar

```

```

103     - `transactions` ? line items from statements, normalized (date, amount, currency,
counterparty, category, source_document)
104     - `holdings` ? point-in-time snapshots of investment positions per account
105     - `merchants` ? canonicalized merchant identities (so "SWIGGY*BANGALORE", "Swiggy
Instamart", and "SWIGGY BNGLR" collapse to one)
106     - `items` ? line-items from itemized receipts (grocery, shopping)
107     - `offers` ? promotional offers/coupons with expiry, source, redemption status
108
109     The merchant canonicalization step is non-trivial ? it deserves its own thought (fuzzy
match + manual override table is the usual answer).
110
111     ### Stage 5 ? Analytics & insights
112
113     **Goal:** turn the normalized DB into the use-cases in § 2.
114
115     Once the data is in the schema, the analytics are mostly SQL + light application logic.
Likely surface: a local CLI initially, possibly a small local web UI later. Notebook-style
exploration is fine for prototyping queries before they're promoted to first-class views.
116
117     ---
118
119     ## 5. Security posture (first class)
120
121     The threat model deserves to be stated plainly: **muneem aggregates the user's entire
financial life ? every account, every transaction, every holding ? into one local store. That
store is a honeypot.** A leak would be catastrophic.
122
123     Standing rules:
124
125     1. **All raw statement data stays on the local machine.** No cloud sync of the staging
dir or the DB.
126     2. **No cloud LLM APIs for sensitive document categories** (bank, CC, broker, MF,
insurance) without explicit per-category user approval discussed in the current session. The
default extraction path for these is local (Tesseract + local LLM via Ollama, or deterministic
parsers).
127     3. **Non-sensitive categories** (promo emails, generic newsletters, public receipts) may
use cloud LLMs after a privacy review of the specific content type.
128     4. **DB encrypted at rest** ? *implemented* (decision D.2, 2026-05) with **SQLCipher**:
transparent AES-256 page encryption via the permissive `sqlcipher3` binding, so SQL queries work
unchanged. The 256-bit key is random, lives only in the OS keychain (never in the repo, config,
args, or logs), passed in raw-key form. OS full-disk encryption (BitLocker/LUKS) is a
recommended complement, not a substitute. See [docs/encryption-at-rest.md](./docs/encryption-
at-rest.md).
129     5. **OAuth tokens** stored in the OS keychain (Windows Credential Manager / macOS
Keychain), not on disk.
130     6. **Logs must not contain extracted statement text.** Sanitize aggressively; redact
account numbers, balances, line-items.
131     7. **Nothing sensitive ever in git.** `.gitignore` blocks real PDFs, the DB file,
`.env`, and password files. Do not weaken it.
132     8. **Commercial-license safety.** Only dependencies under permissive, commercial-safe
licenses (MIT, BSD, Apache-2.0, ISC, HPND). No copyleft (GPL/LGPL/**AGPL**) or source-available
licenses. **PyMuPDF/`fitz` is AGPL-3.0 and is banned** ? use pdfminer.six (MIT) / pypdfium2
(Apache/BSD) instead. Enforced by `tests/test_dependency_licenses.py`; tracked in
[docs/DEPENDENCIES.md](./docs/DEPENDENCIES.md).
133     9. **Minimize paid dependencies.** Prefer local / self-hosted / open-source (and, where
needed, own-trained) models and tools. Paid APIs ? cloud LLMs, paid OCR/vision ? are a
**fallback only**, gated behind explicit per-use approval. This extends rule 2 from privacy to
cost.
134
135     ## 6. Non-goals (for v1)
136
137     - **Multi-user / SaaS / hosted product.** Single user, single machine.
138     - **Cloud sync of raw data.** Period.
139     - **Cracking unknown passwords.** muneem decrypts PDFs the user is authorized to read,
using passwords the user knows or can derive.

```

```

140     - **Browser scraping bank portals.** Email APIs only. Portal scraping is a fragile, ToS-
fraught rabbit hole; we deliberately avoid it.
141     - **Real-time alerting / push.** Batch-pull on a schedule is sufficient.
142     - **Tax preparation, investment advice, or anything regulated.** muneem surfaces data;
the user decides.
143
144     ## 7. Recommended first vertical slice
145
146     Before generalizing, prove the end-to-end pipeline on the smallest meaningful scope:
147
148     > **Gmail ? one specific bank's monthly statements ? unlocked ? parsed into
`transactions` ? stored in SQLite ? a CLI command that lists last month's transactions.**
149
150     This exercises every stage exactly once, on a real input the user cares about, with no
detours. It will surface the actual hard parts (which we can only guess at now) and force the
open decisions in § 8 to get made on real evidence.
151
152     Concretely: pick **one** Indian bank statement format the user has many examples of
(HDFC, given the user's geography and statement history), build the parser for that one format
only, and put the rest of the pipeline around it. Then evaluate before generalizing.
153
154     ## 8. Open architectural decisions
155
156     These were parked for explicit user discussion (**don't pick silently**). Items **8.1,
8.2, and 8.4 are now resolved** ? see the inline resolution notes below and
[docs/ROADMAP.md](./docs/ROADMAP.md) § 1. Items **8.3 and 8.5 remain open**.
157
158     ### 8.1 Language
159
160     | Option | Pro | Con |
161     |---|---|---|
162     | **Go only** | Single binary, easy distribution, great concurrency for the ingestion
side, the user already uses Go for the badminton-highlight-indexer project | Weak PDF parsing /
OCR / ML ecosystem; rolling our own is painful |
163     | **Python only** | Best-in-class PDF, OCR, and ML libraries (pdfplumber, PyMuPDF,
Tesseract bindings, Ollama clients, every parser under the sun); fastest to prototype | Heavier
deployment story; less ergonomic for the long-running ingestion daemon |
164     | **Go ingestion + Python parsing** | Plays to each language's strengths | Two-language
project complexity; inter-process boundary to maintain |
165
166     Recommended default: **Python-only for v1**, accept the deployment-story cost in
exchange for moving fast on the hard parts. Revisit if the ingestion side outgrows Python.
167
168     > **Resolved (2026-05-30): Python-only for v1.** Adopted the recommended default. See
[docs/ROADMAP.md](./docs/ROADMAP.md) § 1.
169
170     ### 8.2 Password sourcing strategy
171
172     See § 4 Stage 2. The hybrid (dictionary ? template ? prompt) is the natural answer, but
the design of the rule-template config format and the candidate dictionary needs a user call.
173
174     > **Resolved (2026-05-30): Hybrid ? candidate dictionary ? per-sender rule template ?
interactive prompt**, caching the winning method per sender; password material never in the repo
or logs. The rule-template config format and dictionary design are deferred to implementation
(ROADMAP Milestone C.1). See [docs/ROADMAP.md](./docs/ROADMAP.md) § 1.
175
176     ### 8.3 LLM locality, per document category
177
178     The default is "local for sensitive, cloud allowed for non-sensitive after review." But
which models? Ollama with Llama 3 / Qwen / Mistral on the user's hardware? A small fine-tuned
model? Cloud (Anthropic / OpenAI / Gemini) with strict redaction for the non-sensitive
categories?
179
180     This decision is closely coupled to the user's hardware capacity (GPU? RAM?) and
willingness to wait on inference latency.

```

```

181
182     ### 8.4 First-slice scope confirmation
183
184     § 7 proposes "Gmail + one bank ? CLI". Confirm with user before building.
185
186     > **Resolved (2026-05-30): Build both tracks in parallel** ? the promo-corpus
parse?store?query spine *and* the Gmail ingestion plumbing, as two decoupled tracks that
converge at the first bank slice; first bank = **HDFC**. This refines § 7 (which proposed a
single end-to-end bank slice first). See [ `docs/ROADMAP.md` ](./docs/ROADMAP.md) §§ 1 and 4.
187
188     ### 8.5 Merchant canonicalization approach
189
190     Fuzzy string matching has well-known failure modes on merchant names. Do we use a
curated rules file (user maintains)? A library? A learned model? Defer until real data exposes
the actual mess.
191
192     ### 8.6 Ingestion source ? Open Banking (Account Aggregator) vs PDF parsing ? **RESOLVED
(2026-06): PDF-first, AA deferred**
193
194     Researched in depth ([ `docs/research-open-banking-india.md` ](./docs/research-open-
banking-india.md)). India's "open banking" is the RBI **Account Aggregator** framework. A
single-user, local-first, no-paid-API tool **cannot** consume AA today ? **FIU status requires a
regulated entity** (no individual path), and the only reachable integration rides under a
licensed FIU via a **paid TSP that decrypts the entire feed in its cloud** (on-device/self-
custody-key decryption is *not* an available AA mode). AA is *safer in transit* than emailed
PDFs (E2E-encrypted, data-blind aggregator), but for muneem's threat model (one local honeypot,
zero third parties) the realistic integration is a **net privacy regression**. AA also can't
cover muneem's full scope (no credit cards / bonds / broker detail / non-demat MF) and still
needs the enrichment/reconcile layer. **Decision:** PDF-first core; AA only ever a **deferred,
opt-in complement** (import from a regulated FIU's export, oracle-validated, provenance-tagged),
with explicit re-adopt triggers (chiefly: a TSP offering true on-device decryption). A new
local-only candidate surfaced ? **SMS/email transaction-alert parsing** ? as a freshness
complement to PDF.
195
196     ### 8.7 Storage shape ? **RESOLVED (2026-06): flat per-issuer source of truth + derived
serving layer**
197
198     Each parser writes its own **flat, isolated per-issuer table** (the source of truth ?
regression-isolated, trivially backfilled/rolled back). Cross-source querying is a **derived
serving layer** (a SQL view/materialised table now; a separate serving DB only when latency/cost
demand). **Provenance + confidence** are first-class but live in a **separate `provenance` table
keyed by `document_id`** (not duplicated per row): source_type (pdf/aa/sms), parser,
parser_version, mapping_layer, confidence. The reconciliation oracle treats every source as a
hint to validate. See [ `docs/schema.md` ](./docs/schema.md) and
[ `docs/feedbackJune2026.md` ](./docs/feedbackJune2026.md) P3.
199
200     ## 9. Proposed (not built) file layout
201
202     This is a *suggestion* for once the language decision is made. Don't materialize it
prematurely.
203
204     ``
205     muneem/
206     ??? README.md
207     ??? CLAUDE.md
208     ??? DESIGN.md
209     ??? docs/
210     ?   ??? prior-art-email-mining.md
211     ?   ??? (future: parser-notes-per-issuer.md, schema.md, threat-model.md)
212     ??? testdata/
213     ?   ??? promo-emails/           # non-sensitive corpus (in repo)
214     ?   ??? statements/            # GITIGNORED ? real statements, never committed
215     ??? (source tree TBD based on § 8.1)
216     ??? (DB file, GITIGNORED)
217     ``

```


218
 219 **## 10. Glossary**
 220
 221 - ****Muneem**** (?????) ? traditional Indian household/business bookkeeper.
 222 - ****Issuer**** ? the bank, card network, broker, or merchant that produced a document.
 223 - ****Statement**** ? a periodic financial summary from an issuer (bank statement, CC bill, broker statement, MF report).
 224 - ****Holding**** ? a position in an investment account at a point in time.
 225 - ****Line item**** ? a single transaction or itemized purchase row within a statement or receipt.
 226 - ****Born-digital PDF**** ? a PDF generated from a software source with a real text layer (most bank statements). Contrast with scanned/image PDFs.
 227
 228 **## 11. Vision continuity from Email-Mining**
 229
 230 The predecessor project (`~/Projects/Email-Mining`) extracted promo codes and expiry dates from manually-saved promo email PDFs, using PyMuPDF for text+image extraction and Gemini for image OCR and summarization. Its scope was just "find the coupon I forgot about."
 231
 232 muneem ****subsumes**** that scope. The deals/coupons feature survives as ****one of several extraction targets**** ? one of the analytics surfaces, fed by the same five-stage pipeline that handles bank statements and everything else. The promo-email PDFs from that project are preserved in [`testdata/promo-emails/`] (./testdata/promo-emails/) as the first regression corpus.
 233
 234 What was learned there and worth keeping: the PDF text+image extraction ***shape*** is right (but PyMuPDF/fitz itself is AGPL-3.0 ? muneem uses pdfminer.six (MIT) instead; see § 5 rule 8); image dedup by hash + SSIM is a real win for repeated chrome; the shape "page text + per-image OCR ? unified text ? structured extraction" generalizes well. What we change: programmatic Gmail/Outlook fetching instead of manual save-as-PDF; local extraction by default for sensitive categories; persistent normalized DB instead of one-shot stdout.
 235
 236 Full details in [`docs/prior-art-email-mining.md`] (./docs/prior-art-email-mining.md).
 237

4. user (2026-06-10T16:13:35.601Z)

1 # muneem ? Project Plan & Roadmap
 2
 3 > Status: ****active plan; mid-implementation.**** Milestone 0 + Track A are done and the parser concept-core has landed (see the Progress snapshot below; [`CURRENT\_STATE.md`] (./CURRENT_STATE.md) is the live status). This is the "how and in what order" companion to [`DESIGN.md`] (./DESIGN.md) (the "what and why"); DESIGN.md owns the architecture and threat model, this doc owns sequencing. Where they disagree, DESIGN.md wins.
 4 >
 5 > Created 2026-05-30.
 6 >
 7 > ****Scope:**** muneem is a **_whole-finance_** bookkeeper. Bank/CC statements are the first slice;
 8 > ****equity, mutual-fund, ETF, and bond statements are core scope**** (not merely the § 6 "generalize"
 9 > backlog), enabled by the family-agnostic concept core. Sequencing still leads with the bank slice,
 10 > but the data model and parser core are built to host an instrument family + instrument-event
 11 > classification (buy / sell / dividend / fees).
 12 >
 13 > ****Progress snapshot (2026-06):**** Milestone 0 (scaffolding / offline CI / config + keyring) ? and
 14 > ****Track A**** (promo spine: `ingest` ? SQLite ? `offers list`, green regression) ? are done.
 15 > ****Track B:**** Gmail OAuth2 + search + attachment download ?; the `muneem fetch` CLI (B.4) is not
 16 > built yet. ****Parsers:**** the concept-core rework (parser-architecture.md ****steps 123****) landed ?

```

17     > HDFC 6/6 real, AU Bank 4/4, Axis 6/7, ICICI removed. **Password unlock is now wired
into `ingest`**
18     > (C.1 ? hybrid candidates ? prompt, keychain-cached; see ingesting-statements.md).
**Not yet:** the
19     > unified `transactions`/`holdings` schema (C.3 / step 5). The
20     > checkbox lists below predate the concept-core pivot ? treat
[ `CURRENT_STATE.md` ](./CURRENT_STATE.md)
21     > as the live status.
22
23     ---
24
25     ## ?? Known issues
26
27     > **Axis savings ? rewrite landed (2026-06), partially complete.** The old fixed-column
28     > `extract_tables()` mapping (wrong columns, layouts that drift between vintages) was
replaced by the
29     > concept-core path: coordinate clustering + per-table mapping ? engine ? reconciliation
oracle
30     > (per-row walk, else total reconcile), oracle-gated. See [ `parser-
architecture.md` ](./parser-architecture.md)
31     > step 3. **Real-data precision: 6/7** ? June 2025 was recovered via **balance-delta
reconstruction**
32     > (derive debit/credit from the running-balance sign, then cross-check `amount ==
|?balance|`). **May
33     > 2025 still fails**: the clusterer drops one transaction line ? incomplete ledger,
which the oracle
34     > correctly rejects (it stores **nothing** ? never mislabeled rows).
35     >
36     > **Remaining:** recover the dropped line (smarter line recovery in the clusterer) for
May 2025 and
37     > future vintages.
38     > The credit-card parser (`AxisCreditCardParser`) is still unverified on real data.
39
40     ---
41
42     ## ? Open critical work items (tracked)
43
44     Live cross-cutting work, each detailed in its linked home:
45
46     1. **Regression CI for statement parsing** ? two-layer (synthetic cloud CI + self-hosted
real-statement
47         runner); decided, **build deferred**. ? § "Planned ? Regression CI for statement
parsing" (below).
48     2. **Axis May 2025 recovery** ? recover the one transaction line the clusterer drops
(incomplete
49         ledger; oracle-gated, stores nothing). June 2025 recovered via balance-delta. ? "??
Known issues".
50     3. **L3 local-vision fallback** ? DeepSeek-OCR-3B (MIT), reconcile-gated; dev RTX 2070S
? 4-bit + SDPA.
51         Research done, **build deferred**. ? `parser-architecture.md` §4/§7.
52     4. **Golden eval sets** ? local-golden + synthetic-golden; cloud-golden eval-only &
approval-gated.
53         ? `parser-architecture.md` §10a. **Seed?golden onboarding harness landed:** `muneem
eval <folder>`
54         (reconcile-rate report) + [ `onboarding-a-bank.md` ](./onboarding-a-bank.md).
55     5. **Unified `transactions`/`holdings` schema + instrument family** (equity/MF/ETF/bond
+ buy/sell/
56         dividend/fee events) ? the whole-finance milestone. ? §10 step 5 / `scope-all-
personal-finance` memory.
57
58     ---
59
60     ## 1. Decisions locked (2026-05-30)
61
62     These resolve open decisions from DESIGN.md § 8. They were made by the user explicitly;

```

this section is the start of the decision log.

```
63
64 | DESIGN.md ref | Decision | Choice |
65 |---|---|---|
66 | § 8.1 Language | Implementation language for v1 | **Python-only.** Accept the heavier
deployment story in exchange for the best PDF/OCR/LLM ecosystem and fastest path on the hard
parsing work. Revisit only if the ingestion daemon outgrows Python. |
67 | § 8.4 First slice | What to build first | **Both tracks in parallel** ? the promo-
corpus parse?store?query spine *and* the Gmail ingestion plumbing, developed as two decoupled
tracks that converge at the first bank slice. |
68 | § 8.2 Passwords | Password sourcing strategy | **Hybrid: candidate dictionary ? per-
sender rule template ? interactive prompt.** Cache the winning method per sender. Secrets never
touch the repo or logs. |
69 | (practical) | First real bank parser target | **HDFC** ? born-digital statements,
name+DDMM-style PDF password (per CLAUDE.md), and the issuer with the longest consistent
statement history to test against. |
70
71 **Still parked** (do not pick silently ? see § 9 below): § 8.3 LLM model for *sensitive*
categories, and § 8.5 merchant canonicalization. _(Encryption-at-rest ? D.2 ? is now **decided &
implemented**: SQLCipher via `sqlcipher3` + keychain key. See [docs/encryption-at-
rest.md](./encryption-at-rest.md).)_
72
73 ---
74
75 ## 2. Guiding principles (inherited from DESIGN.md § 5 + CLAUDE.md)
76
77 1. **Local by default.** All raw statement data and the DB stay on the machine. OCR/LLM
work runs locally for any sensitive category. No cloud sync of staging or DB, ever.
78 2. **No cloud LLM on sensitive contents** (bank, CC, broker, MF, insurance) without
explicit per-category approval in-session. The **promo corpus is the one non-sensitive
playground** where cloud LLMs are policy-permitted ? so it's where we validate any LLM-
extraction pattern *before* it could ever see a statement.
79 3. **Test on safe, committed data wherever possible.** `testdata/promo-emails/` is the
regression corpus and runs in CI. Statement parsing gets CI coverage via **synthetic/redacted
fixtures** in the issuer's layout ? never real statements.
80 4. **Secrets discipline.** OAuth tokens ? OS keychain (Windows Credential Manager).
Nothing sensitive in git; don't weaken `.gitignore`. Logs are redacted (no account numbers,
balances, line items, or statement text).
81 5. **Decisions parked for humans stay parked.** Don't write code that silently locks in
§ 9's open items.
82
83 ---
84
85 ## 3. Proposed Python stack
86
87 User-locked is Python-only. The libraries below are *recommendations* (well-known OSS,
per CLAUDE.md's "batteries-included stdlib + well-known OSS over novel dependencies") ? open to
override, not parked decisions. Anything that is a cloud **service** is flagged and needs a nod
before adoption.
88
89 | Concern | Proposed | Notes |
90 |---|---|---|
91 | Packaging / deps / venv | **uv** + `pyproject.toml` | Fast, modern. Poetry is an
equally fine fallback. |
92 | Lint + format | **ruff** | One tool, fast. |
93 | Tests | **pytest** | Regression suite runs offline in CI on the promo corpus +
synthetic fixtures. |
94 | PDF text + image extraction | **pdfminer.six** + **pypdfium2** | PyMuPDF/`fitz` is
**banned (AGPL)** ? see DESIGN §5 rule 8 / DEPENDENCIES.md. |
95 | Table recovery (born-digital statements) | **pdfplumber** | Coordinate/word-level
extraction (HDFC clustering; Axis/AU via the shared savings path). |
96 | PDF decryption | **pypdf** (BSD-3) | Adopted; pikepdf (MPL-2.0) was the flagged
fallback, not used. |
97 | Image dedup | **Pillow + scikit-image SSIM** | Carried from Email-Mining (MD5 hash +
SSIM ? 0.95). |
```

```

98      | OCR (image PDFs, later) | **Tesseract** via `pytesseract` | Local. Windows needs the
Tesseract binary installed (setup task). |
99      | Gmail | **google-api-python-client + google-auth-oauthlib** | Read-only scope
`gmail.readonly`. |
100     | Secret storage | **keyring** | Uses Windows Credential Manager on win32. |
101     | DB | **SQLCipher** via `sqlcipher3` (stdlib `sqlite3` API) + lightweight versioned
migrations | **Encryption-at-rest implemented** (transparent AES-256; 256-bit key in the OS
keychain). `:memory:`/tests stay on plaintext stdlib `sqlite3`. See [docs/encryption-at-
rest.md](./encryption-at-rest.md). |
102     | CLI | **Typer** | Clean ergonomics; `argparse` (stdlib) is the zero-dependency
fallback. |
103     | LLM (promo only, non-sensitive) | deterministic-first; cloud LLM **only if needed and
approved** | See § 5 Track A. Sensitive-doc LLM is parked (§ 9 / DESIGN § 8.3). |
104
105     ---
106
107     ## 4. Build order at a glance
108
109     ...
110     Milestone 0 ? Decisions & scaffolding (no sensitive data)
111         ?
112         ?????????????????????????????????????????????????????????????
113         ?                                     ?
114         TRACK A: Promo spine                TRACK B: Gmail ingestion          (run in parallel,
115         parse ? store ? query                fetch ? stage ? catalog          fully decoupled)
116         (committed corpus, CI)              (real email, local only)
117         ?                                     ?
118         ?????????????????????????????????????
119         ?
120         Milestone C ? HDFC vertical slice (CONVERGENCE)
121         unlock (hybrid passwords) ? HDFC parser ? transactions ? CLI
122         ? completes DESIGN.md § 7
123         ?
124         ?
125         Milestone D ? Harden & evaluate (the pre-generalize checkpoint)
126         ?
127         ?
128         Generalize (more issuers, local-LLM fallback, holdings, items, analytics, UI)
129     ...
130
131     Tracks A and B are genuinely independent: A needs no OAuth and no secrets; B needs no
parsing. As a single developer you can interleave them in any order ? neither blocks the other.
**Both must land before Milestone C**, where a fetched-and-unlocked HDFC PDF flows through the
parser for the first time.
132
133     Effort tags below are rough (S / M / L), for relative sequencing only ? not calendar
estimates.
134
135     ---
136
137     ## 5. Milestones & tasks
138
139     ### ? Milestone 0 ? Decisions & scaffolding · DONE
140
141     - [x] **0.1 (S)** Project skeleton: `pyproject.toml` (uv), package dir (name TBD, e.g.
`muneem/`), `ruff` + `pytest` configured. *Do not over-materialize the source tree (DESIGN § 9)
? create only what 0.x needs.*
142     - [x] **0.2 (S)** Offline CI: GitHub Actions (or local pre-commit) that installs deps
and runs `pytest` against committed data only. A trivial `test_smoke` proves the loop is green.
143     - [x] **0.3 (S)** Config + secrets foundation: a config loader (env + a gitignored
`config.toml`), and a `keyring`-backed secret accessor stub. Confirm `.gitignore` covers
`config.toml` and any new secret paths.
144     - [x] **0.4 (S)** `gitignore` audit: verify the proposed local-only dirs
(`testdata/statements/`, staging dir, `*.db`) are blocked. Add a
`testdata/statements/.gitkeep`-style note or README so the expected-but-absent dir is

```

```

documented.
145     - **Exit:** `uv run pytest` is green in CI on a clean checkout; repo layout agreed.
146
147     ### ? Track A ? Promo spine: parse ? store ? query · DONE (committed corpus, green in
CI)
148
149     > Resurrects the Email-Mining deals feature as muneem's first working output. Zero
secrets, zero OAuth.
150
151     - [x] **A.1 (M)** Port Email-Mining's extraction into a real module: **pdfminer.six**
text + **pypdfium2** image extraction (PyMuPDF is banned/AGPL), size filter, dedup. Proper
arguments/config; returns a value instead of one-shot stdout. *(Done ? `muneem.extract`.)*
152     - [x] **A.2 (S)** Minimal schema slice + migration v1: `documents` (id, source,
external_id, sender, subject, received_date, doc_type, sha256, path, status, created_at) and
`offers` (id, document_id, merchant, promo_code, description, expiry_date, redeemed,
source_text). Full schema is future work (`docs/schema.md`).
153     - [x] **A.3 (M)** Promo extraction, **deterministic-first**: regex/heuristics for promo
codes + expiry dates over the unified text. Where deterministic extraction misses, an **LLM
fallback** is permitted *because promo is non-sensitive* ? but that pulls in a cloud service, so
it's gated on a quick okay (reuse Email-Mining's Gemini path, go local via Ollama, or use
Anthropic ? your call when we get there).
154     - [x] **A.4 (M)** `muneem ingest <pdf>` writes a `documents` + `offers` row; `muneem
offers list` reads them back. **Ground-truth + regression test:** establish the known promo
codes for each of the 10 corpus PDFs, then a `pytest` that runs the pipeline over
`testdata/promo-emails/` and asserts recovery. (Building the ground-truth answer key is itself
part of this task.)
155     - **Exit:** `ingest ? SQLite ? offers list` works end-to-end; green regression test on
the committed corpus. Stages 3?4?5 proven with zero risk.
156
157     ### Track B ? Gmail ingestion · PARTIAL (OAuth + search + download ?; `muneem fetch` B.4
pending)
158
159     > First contact with real (sensitive) email. Everything it writes is gitignored.
160
161     - [ ] **B.1 (M)** Gmail OAuth2: register a Google Cloud project, `gmail.readonly` scope,
store the token in `keyring`. Document the one-time consent flow.
162     - [ ] **B.2 (M)** Financial-document detection v1: a **sender allowlist** + server-side
query (`has:attachment filename:pdf from:? newer_than:?.`). Start narrow and precise; broaden
later. Pull matching messages + PDF attachments into a gitignored staging dir.
163     - [ ] **B.3 (S)** Incremental sync: persist Gmail `historyId`; on re-run, fetch only
what's new. Idempotent (re-running never duplicates `documents`, keyed by message id +
attachment hash).
164     - [ ] **B.4 (S)** Catalog fetched attachments as `documents` rows (status
`locked`/`unparsed`), with sha256 + staging path. `muneem fetch --since 90d` is the entry point.
165     - **Exit:** `muneem fetch` populates the staging dir + `documents` rows from a narrow
real-Gmail query. (Manual/local verification ? not CI; it touches a real account.)
166
167     ### Milestone C ? HDFC vertical slice · IN PROGRESS (HDFC/Axis/AU parsers ?, unlock-
wiring C.1 ?; unified schema C.3 pending)
168
169     > A fetched HDFC statement gets unlocked and parsed into real transactions. This is the
moment every stage runs once on real data the user cares about.
170
171     - [x] **C.1 (M)** Unlocking (Stage 2), the **hybrid** strategy ? **wired into `muneem
ingest`** (`statement_unlock.py`): keychain-cached + `password_rules.yaml` candidates ?
interactive prompt, with the winning password cached in the keychain. **No password material in
repo or logs.** Decrypt via pypdf/pdfminer/pdfplumber (pikepdf was the flagged fallback, not
used). See [ingesting-statements.md](./ingesting-statements.md).
172     - [x] **C.2 (L)** HDFC parser (Stage 3, the hard 80%): born-digital text-layer +
pdfplumber table recovery ? normalized transactions (txn_date, value_date, amount, direction,
narration, running balance). **Synthetic HDFC-layout fixtures** committed for CI; real
statements stay gitignored locally.
173     - [ ] **C.3 (M)** Schema migration v2: `accounts` (issuer, account_type, last4, holder)
+ `transactions` (account_id, document_id, txn fields, category, merchant_id-nullable). Populate
from the parser. `muneem txns list --last-month` prints real parsed transactions.

```

```

174 - **Exit:** **DESIGN.md § 7 vertical slice complete** ? Gmail ? HDFC statement ?
unlocked ? parsed ? SQLite ? CLI lists last month's transactions, on real data, locally.
175
176 ### Milestone D ? Harden & evaluate · the pre-generalize checkpoint
177
178 - [ ] **D.1 (M)** Redacted structured logging (assert no statement text / account
numbers leak); idempotent, resumable re-runs across all stages.
179 - [x] **D.2 (S)** Encryption-at-rest decision (DESIGN § 5.4): **DONE (2026-05)** ?
adopted SQLCipher now (transparent AES-256 via `sqlcipher3`), 256-bit key in the OS keychain;
queryable, permissive, Windows+Linux wheels. Research-backed (see § 9 forced-decision note).
Details in [ `docs/encryption-at-rest.md` ](./encryption-at-rest.md).
180 - [ ] **D.3 (M)** **Evaluation writeup:** how well did the deterministic HDFC parser
hold across statement vintages? Where does it break? Which categories will need the local-LLM
fallback? This is the evidence DESIGN § 7 asks for *before* generalizing. Capture in
`docs/parser-notes-hdfc.md`.
181 - **Exit:** the slice is robust and re-runnable; a written go/no-go on generalization
with real evidence.
182
183 ---
184
185 ## 6. Generalize (post-checkpoint backlog ? scope after Milestone D)
186
187 Ordered roughly by leverage, not committed:
188
189 - Second/third bank parsers + the first **credit-card bill** parser (reuses unlock +
parser scaffolding).
190 - **Local-LLM long-tail fallback** (Ollama) for issuers that don't justify a bespoke
parser ? **local, because these are sensitive** (resolves DESIGN § 8.3).
191 - Broker / MF statements ? `holdings` (point-in-time positions; the investment-timeline
use-case).
192 - Itemized receipts (Swiggy/Amazon/BigBasket/etc.) ? `items` (the price-intelligence +
catalog use-cases).
193 - **Merchant canonicalization** ? `merchants` (DESIGN § 8.5 ? defer until real data
exposes the actual mess, then decide rules-file vs. library vs. learned).
194 - Analytics surfaces for the DESIGN § 2 use-cases (CLI views first; optional small
**local** web UI later).
195
196 ---
197
198 ## 7. Cross-cutting workstreams (run throughout, not a phase)
199
200 - **Security & secrets hygiene** ? keychain for tokens + cached passwords; redacted
logs; gitignore audits each milestone; encryption-at-rest tracked to D.2.
201 - **Schema migrations from day one** ? versioned migration scripts; the schema *will*
churn as new issuers land. Keep `docs/schema.md` current.
202 - **Test-corpus curation** ? ground-truth answer key for the promo corpus (A.4);
synthetic/redacted statement fixtures for CI (C.2). Never commit real statements.
203 - **Decision log** ? append resolutions to § 1 here and mark the corresponding DESIGN.md
§ 8 item resolved.
204
205 ---
206
207 ## Planned ? Regression CI for statement parsing (decided 2026-06, build deferred)
208
209 > **Status:** approach **decided** with the user; **implementation deferred** (we paused
before
210 > coding). This note grounds the decision so it can be resumed without re-litigating it.
211
212 **Goal.** Automatically catch parser regressions against the **real HDFC / AU (/ Axis)
statements**
213 when working across branches ? as GitHub Actions ? without putting that financial data
at risk.
214
215 **Decision: a two-layer "smart mixture" (the user's call), *not* committing real
statements:

```

```

216
217     1. **Synthetic full-pipeline fixtures in normal cloud CI** ? every branch, no secrets,
no real data,
218         always-on (covers collaborators / when the self-hosted runner is offline).
219     2. **A self-hosted runner that exercises the real statements** ? the statements stay on
the user's
220         machine; GitHub only orchestrates.
221
222     **Why we are NOT committing the real (even encrypted) statements** ? the rejected
"simplest" option:
223     git history is permanent and lives on GitHub's servers, so a committed encrypted PDF is
recoverable
224     (with its password) **forever** ? surviving the repo being made public, forked, or a
225     token/collaborator compromise. That is exactly the threat the local-first design and the
repo's own
226     guards (`.gitignore`, `.github/hooks/precommit_check.py`, `tests/test_repo_hygiene.py`)
exist to prevent.
227     Real statement data stays local; GitHub never stores it. (DESIGN.md §5.)
228
229     ### Layer 1 ? synthetic cloud CI (design)
230     - A **dependency-free PDF generator** (stdlib; positioned text via PDF `Tm`/`Tj`,
Helvetica base-14)
231     generates born-digital statement PDFs at test time ? no new dependency, stays
commercial-safe.
232     - Two fixtures run the **full pipeline** (`extract` ? `get_parser` ? `parse_and_insert`
? assert
233         `reconciled`):
234     - **HDFC layout** (coordinate clustering): leading date `x0?52`, narration `x0?111`,
three
235         right-aligned amount columns whose right edges land in the clusterer's bands
(withdrawal `<400`,
236         deposit `<490`, balance `?490`), plus a `SUMMARY` block for the bookend.
237     - **Savings layout** (the shared `savings.py` engine used by Axis/AU): a clean
238         date | narration | debit | credit | balance table ? clustering/engine ? per-row
reconcile.
239     - This catches **extraction / clustering / engine** regressions in the cloud with zero
real data ?
240         the gap the existing synthetic *word-list / table* tests (which bypass pdfplumber)
don't cover.
241     - **Broaden `ci.yml` triggers** from `push:[main]` + PR to **all-branch pushes**, so
feature branches
242         gate even without a PR.
243
244     ### Layer 2 ? self-hosted runner for real statements (design)
245     - New workflow `.github/workflows/real-statements.yml`, `runs-on: self-hosted`. Triggers
on branch
246         push / PR / manual dispatch.
247     - A fresh CI checkout does **not** contain the gitignored statements, so a step **stages
them from a
248         local path** the user sets on the runner host (e.g. env `MUNEEM_LOCAL_TESTDATA` ?
copied into
249         `testdata/`), then runs `pytest -m local_statements`.
250     - **Passwords come from the local `password_rules.yaml`** in the muneem config dir on
the runner
251         machine (the same file `muneem ingest` uses) ? **nothing sensitive on GitHub, no
Secrets needed.**
252         (GitHub Secrets remain an easier-but-less-private opt-in.)
253     - The `local_statements` tests already assert what we want: HDFC reconcile **+
bookend**; Axis/AU
254         oracle-gated safety + reconcile (`test_hdfc.py`, `test_local_axis.py`,
`test_local_au.py`).
255
256     ### Small supporting change
257     - Unify password sourcing: have the **HDFC** real tests (`test_hdfc.py`,
`test_unlock.py`) also read

```

```

258     `get_password_for_bank("hdfc")`, so one `password_rules.yaml` drives all issuers
(today they read
259     only `MUNEEM_TEST_HDFC_PW`).
260
261     ### One-time setup the user must do (to actually run Layer 2)
262     1. Register a self-hosted Actions runner on the machine (Settings ? Actions ? Runners;
Windows host).
263     2. Put the real statements in a stable local dir and point `MUNEEM_LOCAL_TESTDATA` at
it.
264     3. Have `password_rules.yaml` in the config dir (see [ingesting-
statements.md](./ingesting-statements.md)).
265     4. Keep the runner online when CI should run.
266
267     ### Caveats to weigh at build time
268     - A self-hosted runner executes workflow code on the user's machine ? fine for this
**trusted private
269     repo**, but be careful running workflows from untrusted branches/forks on it.
270     - Layer 2 only runs when the machine is online; Layer 1 is the always-on baseline.
271     - The synthetic HDFC fixture is pinned to the clusterer's hard-coded x-bands ? keep it
in sync if
272     those change.
273
274     **Resume by** implementing Layers 1 & 2 above; nothing here is built yet.
275
276     ---
277
278     ## 8. Risks & mitigations
279
280     | Risk | Mitigation |
281     |---|---|
282     | Sensitive data leaks into git, logs, or a cloud LLM | Local-by-default; promo-only LLM
playground; redacted logging (D.1); gitignore audits; the § 2 principles are non-negotiable. |
283     | HDFC parser is brittle across statement vintages | Synthetic fixtures + the D.3
evaluation gate *before* generalizing; deterministic-first with a planned local-LLM fallback for
the long tail. |
284     | Password sourcing fails noisily on backfill | Hybrid strategy degrades to interactive
prompt; cache per sender so it's a one-time cost. |
285     | Gmail OAuth / quota friction | Read-only scope, server-side queries (cheap),
incremental `historyId` sync; narrow allowlist first. |
286     | Scope creep into the full § 2 vision before the slice works | Hard checkpoint at
Milestone D; generalization is explicitly post-checkpoint. |
287     | Two parallel tracks diverge / integration surprises at C | Both write through the
*same* `documents` table and schema-migration discipline from A.2/B.4, so convergence is a join,
not a rewrite. |
288
289     ---
290
291     ## 9. Still-open decisions (do NOT pick silently)
292
293     Carried from DESIGN.md § 8, not yet resolved ? surface and ask when a milestone forces
them:
294
295     - **§ 8.3 ? LLM model for sensitive categories.** Coupled to your hardware (GPU? RAM?)
and latency tolerance. Forced at the start of the generalize phase (local-LLM fallback). Ollama
+ Llama/Qwen/Mistral is the likely shape, but unconfirmed.
296     - **§ 8.5 ? Merchant canonicalization.** Deferred until real receipt data exposes the
mess. Forced when `items`/`merchants` land.
297     - ~~**Encryption-at-rest timing** (D.2) ? SQLCipher now vs. later.~~ **Resolved
2026-05:** SQLCipher adopted now (`sqlcipher3` + keychain key), forced by storing real numbers;
research-backed. See [docs/encryption-at-rest.md](./encryption-at-rest.md).
298     - ~~**Ingestion source ? Open Banking (Account Aggregator) vs PDF parsing.**~~
**Resolved 2026-06:** **PDF-first; AA deferred** as an opt-in complement only (regulatory FIU
gate + paid cloud-decrypting TSP make it non-viable and a net privacy regression for a single-
user local tool; re-adopt triggers tracked). Research-backed ? see [docs/research-open-banking-
india.md](./research-open-banking-india.md) and DESIGN § 8.6. Surfaced a new local-only

```


candidate: ****SMS/email transaction-alert parsing**** (freshness complement to PDF, zero third party).

299 - **~~**Storage shape.**~** ****Resolved 2026-06:**** ****flat per-issuer source-of-truth tables + a derived serving layer; provenance/confidence in a separate `provenance` table**** (DESIGN § 8.7, feedback June 2026 P3). First forward migration adds `provenance`.

300 - ****Promo-track LLM**** (A.3) ? only reached if deterministic extraction underperforms; cloud is policy-permitted for promo but still needs a quick okay on which vendor.

301

302 ---

303

304 **## 10. Definition of done for v1**

305

306 The DESIGN.md § 7 slice, hardened: ****`muneem fetch`** pulls HDFC statements from Gmail ? unlocks them with the hybrid password strategy ? the HDFC parser writes normalized `transactions` to SQLite ? ****`muneem txns list`** shows last month's activity ? all locally, with redacted logs, idempotent re-runs, green CI on synthetic fixtures, and a written evaluation of where to go next.** The promo "forgotten deals" feature ships alongside as the Track A deliverable.

307

5. user (2026-06-10T16:13:35.654Z)

1 # Current State

2

3 > ****Last Updated:**** 2026-06 (concept-core engine + issuer refocus: HDFC / Axis / AU Bank)

4 > Summarizes current capabilities, completed features, and near-term next steps for `muneem`.

5 > See [`ROADMAP.md`](./ROADMAP.md) for milestone tracking and [`parser-architecture.md`](./parser-architecture.md) for the parser design.

6

7 > ****Scope reminder ? muneem is a _whole-finance_ bookkeeper, not a bank tool.**** Bank & credit-card

8 > statements are the first vertical slice; ****equities, mutual funds, ETFs, and bonds are core**

9 > ****targets****, parsed into the same local ledger, with transaction/instrument-event classification to

10 > follow. The parser core is deliberately ****family-agnostic**** (see §2), so adding an instrument

11 > family is ***new data + a reconciliation rule***, not a new pipeline.

12

13 **## 1. Core Ingestion Pipeline**

14 The ingestion pipeline supports a generic `SearchCriteria` interface and multiple providers (Track B plumbing):

15

16 * ****Gmail**** (``muneem.ingestion.gmail``): Full OAuth2 read-only + search + attachment download implemented.

17 * ****IMAP**** (``muneem.ingestion.imap``): Basic auth + search implemented; ``get_headers`` / attachment methods are `NotImplemented` (experimental).

18 * ****Outlook**** (``muneem.ingestion.outlook``): Device-flow auth + basic search implemented; attachment/header/download `NotImplemented` (experimental).

19

20 Companion one-time auth tools live in ``tools/``. No high-level ``muneem fetch`` CLI command yet (see Roadmap B.4).

21

22 **## 2. Statement Parsers (concept-core architecture)**

23 Deterministic parsers live under ``muneem.parsers``, registered via decorator and selected by ``get_parser`` on text heuristics + sender. Submodules are ****auto-loaded on package import**** (``_autoload_parsers``), so the registry is populated in the production path (not just under tests).

24

25 The parser core is ****concept-first and verification-first**** (see [``parser-architecture.md``](./parser-architecture.md)): an extractor maps an issuer's columns onto a bank-agnostic ****concept schema**** (``concepts.py``); a generic ****engine**** (``engine.py``) does the column?concept mapping (L0 header lexicon ? profile column-order ? L2 content search); and a

shared ****verifier**** (``validation.py``) decides trust by **arithmetic** ? per-row balance walk + total reconcile + SUMMARY bookend + a recorded confidence verdict. The engine is ****family-agnostic****: parameterized by a ``FamilySpec`` + an injected reconciliation oracle, so a future ****instrument family**** (equity/MF/ETF/bond holdings + buy/sell/dividend/fee events) reuses the same control flow with its own concepts + oracle. Issuer knowledge lives in ``profiles.py`` as ****data, not code****. Bank ****savings**** parsers share one engine-driven path ? ``savings.py`` (per-table + coordinate-clustered candidates ? engine ? oracle), so adding a savings issuer is mostly a profile; HDFC keeps bespoke coordinate clustering (the documented ``extract_tables`` exception).

26
27 ****Active issuers**** ? we have real statements + passwords, so these get validated on real data:

28
29 * ****HDFC Bank**** (``hdfc.py``): Savings / transaction-account statements via ****coordinate clustering**** (deliberately **not** ``extract_tables()`` ? HDFC's table is semi-ruled). ****Self-validating**** running-balance reconciliation + a cross-check against the bank's own SUMMARY (Dr/Cr counts, opening/closing balance); records ``documents.status`` = ``reconciled`/`unreconciled``. Refactored onto the concept core as the ****reference implementation****. ? ****Verified on the user's 6 real statements**** ? all reconcile **and** bookend, without inspecting any value.

30 * ****Axis Bank**** (``axis.py``): Savings + Credit Card. Savings runs the shared engine-driven path (``savings.py``) ? per-table + coordinate-clustered candidates ? engine mapping ? reconciliation oracle ? plus a ****balance-delta reconstruction****: when columns don't split into a clean debit/credit (a merged amount column, an interleaved value-date, a sparse deposit column), it derives direction from the running-balance sign and the oracle cross-checks ``amount == |?balance|``. Oracle-gated, so it never persists unreconciled rows. ? ****6/7 real statements reconcile + persist****; the 1 miss (May 2025) has a transaction line the clusterer drops ? incomplete ledger ? the oracle quarantines it (stores ****nothing****). Credit-card has no running balance (count only).

31 * ****AU Bank**** (``au.py``): engine-driven savings via the shared ``savings.py`` path ? same recipe, just ``AUBANK_PROFILE``. ? ****4/4 real statements reconcile + persist**** (the cleanest issuer so far).

32
33 > **_Dropped: **ICICI**** ? no statements on hand to validate against, so it was removed to focus effort on the issuers we can actually verify (HDFC/Axis/AU). It can return as a profile when data exists._

34
35 All balance-bearing parsers share the verifier and split parsing from DB-insert, so row logic is unit-testable without a PDF/DB and a parse is validated before it's trusted. ****Concept-core steps 1?3 have landed**** (the verifier + concept schema; the generic mapping engine + profiles; the shared savings path with Axis + AU Bank on it). Parsers use ``pdfplumber`` (with optional password). ****Password resolution is now wired into `muneem ingest --doc-type bank\_statement`**** ? the hybrid unlock (keychain-cached + ``password_rules.yaml`` candidates ? interactive prompt; ``unlock.py`` + ``statement_unlock.py``) runs before extraction and forwards the password to the parser, in-memory only. See [ingesting-statements.md](./ingesting-statements.md).

36
37 See ``password_rules.py`` (prefers user config dir) and ``cli.py:ingest``.

38
39 **## 3. Database & CLI**

40 * Schema v4 with ``documents``, ``offers``, and per-bank txn tables (``axis_``, ``hdfc_transactions``) ? documented in [schema.md](./schema.md). Dedup on sha256. ****Versioned migrations**** via a hand-rolled runner (``muneem/migrations.py``): the v4 schema is the frozen baseline, forward migrations layer on (recorded in ``schema_migrations`` with checksums), drift is rejected ? see [schema.md](./schema.md) and feedbackJune2026 P6. A unified ``transactions`` table (plus ``holdings`` for instruments) with per-row provenance/confidence is the planned direction once the design decision lands (parser-architecture.md §9; feedbackJune2026 P3).

41 * ****The on-disk ledger is encrypted at rest**** (decision D.2) ? ****SQLCipher**** transparent AES-256 via ``sqlcipher3``, opened through ``db.connect_encrypted()``; the 256-bit key lives in the OS keychain. SQL queries are unaffected. ``memory:`` and the CI fixtures stay on plaintext ``sqlite3``. See [encryption-at-rest.md](./encryption-at-rest.md).

42 * CLI: ``ingest <pdf>`` (promo, or ``--doc-type bank_statement`` ? ****auto-unlocks encrypted PDFs****, see [ingesting-statements.md](./ingesting-statements.md)), ``offers list``, ``txns list <bank>``, ``eval <folder>`` (reconcile-rate report ? the seed?golden onboarding loop, see [onboarding-a-bank.md](./onboarding-a-bank.md)), and ``db`` encryption ops (``status``, ``encrypt``, ``rotate-key``, ``backup``, ``setup-recovery``, ``recover`` ? see [encryption-at-rest.md](./encryption-at-rest.md)).

```

at-rest.md)).
43      * Config uses platformdirs for local dirs; secrets (OAuth tokens, cached statement
passwords, **the DB encryption key**) via keyring.
44
45      ## 4. Testing & Hygiene
46      * Strong offline regression on `testdata/promo-emails/` + synthetic parser/engine
fixtures (default CI).
47      * Security guards in `test_repo_hygiene.py` + license allowlist test.
48      * **DB invariant tripwires** (`test_db_invariants.py`, default CI): a **schema lock**
(pins `SCHEMA_VERSION` + every table/column shape ? an accidental drift fails the merge; a
deliberate change is "explicitly allowed" by updating the lock in the same PR) plus **encryption
guards** (production ingest must write an *encrypted* ledger; a keyed open never silently
downgrades to plaintext; foreign keys enforced + cascade; `documents.sha256` dedup).
49      * **Local git hooks** (`.github/hooks/`, enable via `make hooks`): `pre-commit` blocks
staging sensitive files; `pre-push` runs ruff + the offline suite (incl. the invariant
tripwires) so only green code reaches GitHub. The pre-push gate is the **local** stand-in for
server-side branch protection, which GitHub gates behind Pro / public repos (muneem stays
private). Both are bypassable with `--no-verify` (the conscious "explicitly allowed" escape).
50      * Local-only tests for real statements (gitignored data; passwords via env/keyring,
never committed). The suite never touches the real OS keychain (autouse fixture in
`tests/conftest.py`) ? headless-CI safe.
51
52      ## 5. Next Steps
53      Concept-core **steps 1?3 are done**, and **password unlock is now wired into `muneem
ingest`** (C.1 ? hybrid candidates ? prompt, keychain-cached; see [ingesting-
statements.md](./ingesting-statements.md)). Real-statement precision: **HDFC 6/6, AU 4/4, Axis
6/7** (the 1 Axis miss stores nothing ? oracle-gated). In rough order from here:
54
55      * **Recover the May 2025 vintage**: the clusterer drops one transaction line ?
incomplete ledger; needs smarter line recovery. Isolated and safe (it currently stores nothing;
June 2025 was recovered via balance-delta reconstruction).
56      * **Step 4 ? L3 local-vision fallback** (pending the local-VLM deep-research run) for
layouts the deterministic ladder can't reconcile.
57      * **Step 5 ? unify storage** (`transactions` + `holdings`) and begin the **instrument
family** (equity/MF/ETF/bond ? holdings + buy/sell/dividend/fee classification) ? the next big
piece of the whole-finance vision.
58      * **Regression CI for statements** (decided 2026-06, *deferred ? not yet built*): a
smart mix of synthetic full-pipeline fixtures in cloud CI + a self-hosted runner exercising the
real HDFC/AU statements (which stay local, off GitHub). Grounded plan + rationale in
[ROADMAP.md](./ROADMAP.md) (§ "Planned ? Regression CI for statement parsing").
59
60      Parked: DESIGN §8.3 (local LLM for sensitive categories), §8.5 (merchant
canonicalization). _(Encryption-at-rest timing, D.2, is now resolved & implemented ? SQLCipher;
see [encryption-at-rest.md](./encryption-at-rest.md))._
61
62      The project stays **local-first**, secrets in the OS keychain, promo corpus safe for
CI/LLM experiments.
63

```

6. user (2026-06-10T16:13:41.902Z)

```

1      # muneem ? Parser Architecture (concept-core, verification-first)
2
3      > **Status: DRAFT for discussion.** Direction chosen 2026-06: "concept core + profiles-
as-data."
4      > The deterministic core below (L0?L2, schema, profiles, verifier, traps) is settled
enough to
5      > build against. The local-VLM research is **done**: the **L3 vision fallback** is
**DeepSeek-OCR-3B
6      > (MIT)**, with **Qwen2.5-VL-7B (Apache-2.0)** as a heavier fallback, reconcile-gated
($4, $7) ? and on
7      > the dev RTX 2070S (8 GB, Turing) it runs 4-bit + SDPA. The L3 *build* is deferred.
8      > **Steps 1?3 of the plan (§10) have landed** ? the verifier + concept schema, the
generic
9      > concept-mapping engine + profiles, and the shared savings path (Axis + AU Bank run

```

```

through it: AU
10      > 4/4, Axis 2/4 on real statements). The $11 open questions remain.
11      > Companion to [DESIGN.md](./DESIGN.md) (what/why) and [ROADMAP.md](./ROADMAP.md)
(sequencing).
12
13      ---
14
15      ## 1. Why we're rethinking this
16
17      The current parsers are brittle for two reasons:
18
19      1. They encode *syntax*, not *semantics*. HDFC hard-codes x-positions; Axis hard-
codes column
20      indices. They know where a value sits, not what it is ? so any format nudge maps
the wrong
21      column and produces wrong data.
22      2. They parse open-loop. They emit whatever they extracted with no check, so a wrong
parse looks
23      like a right one. (The Axis parser confidently produced 15 mislabeled rows ? see
ROADMAP Known
24      Issues.)
25
26      Goals for the new architecture: format-resilient (small layout/label changes don't
break us),
27      extensible (a new issuer or a changed format is mostly data, not code), semantic
(we know
28      the critical placeholders ? balance, spend, deposit, ?), and failsafe (drift is
loud, never
29      silent).
30
31      ## 2. The key idea ? reconciliation makes parsing a closed-loop search
32
33      We do not need to perfectly parse every format. We need to propose candidate
interpretations
34      and let arithmetic pick the correct one. The identity
35
36      ```
37      opening_balance + ?(deposits) ? ?(withdrawals) == closing_balance      (+ the per-row
balance walk)
38      ```
39
40      is a truth oracle that is independent of layout. That reframes everything:
41
42      - We can identify concepts aggressively / fuzzily, because a wrong guess is
rejected, not stored.
43      - It is what makes a probabilistic extractor (a vision model) safe on money ? we
never trust the
44      model, we trust the math that checks it.
45      - A changed format is not "rewrite the parser" ? it's "the search finds a mapping that
reconciles."
46
47      Principle: extraction proposes, reconciliation disposes.
48
49      ## 3. The canonical concept schema (the contract)
50
51      One bank-agnostic vocabulary that every extractor targets. These are the "critical
placeholders."
52
53      - Row concepts: `txn_date`, `value_date`, `narration`, `reference`, `debit`
(spend/withdrawal),
54      `credit` (deposit), `balance`; or `{amount, direction}` for credit cards (no running
balance).
55      - Statement concepts: `opening_balance`, `closing_balance`, `debit_count`,
`credit_count`,
56      `account_number`, `period_start/end`, `currency`.

```

```

57
58     Parsing = **map this bank's columns/fields onto these concepts, then verify.**
Provenance + a
59     confidence verdict travel with every parse (see §7).
60
61     ## 4. The extraction ladder (cheapest first; every layer gated by the verifier)
62
63     | Layer | How it identifies a concept (e.g. `balance`) | Resilience it buys |
64     |---|---|---|
65     | **L0 Header lexicon** | alias + fuzzy match: `balance` ? {Balance, Closing Bal, Running
Bal}`, `debit` ? {Withdrawal, Debit, Dr, Spend, Paid Out}`, ? | renamed/abbreviated labels |
66     | **L1 Structure & content** | dates by pattern; amounts are right-aligned numerics;
**`balance` = the numeric column whose running-balance recurrence holds** | missing/merged
headers (the Axis-June case) |
67     | **L2 Mapping search** | enumerate plausible concept?column assignments; **keep the
one(s) that reconcile** | column reorder / count drift between statements |
68     | **L3 Local vision fallback** | **DeepSeek-OCR-3B (MIT)**: page image ? rows in the
concept schema; **accepted only if it reconciles** (self-consistency re-sample on a miss) |
scans, no text layer, novel layouts (the Axis May/Jun vintages) |
69
70     The **self-identifying balance column** (L1) is the heart of it: we find "balance" not
by its label
71     but by *which column makes the math work* ? fully format-independent. L0/L1 usually
agree; L2 is the
72     fallback search when they don't; L3 is the last resort. **The research resolved the
model:**
73     **DeepSeek-OCR-3B** (MIT ? cleanest weights license) leads; **Qwen2.5-VL-7B** (Apache;
only the 7B/32B
74     sizes ? 3B/72B are research-licensed) is the heavier fallback / fine-tune base;
**PaddleOCR-VL**
75     (~1B, Apache) is an 8 GB-friendly co-pilot. Avoid LayoutLMv3 (CC-BY-NC weights), GOT-
OCR2
76     (research-only), Surya/Marker (GPL / RAIL), MinerU (AGPL dependency). On the dev **RTX
2070S** (8 GB;
77     Turing sm75 ? no bf16, no FlashAttention-2) the local pick is **DeepSeek-OCR-3B in 4-bit
+ SDPA** ;
78     Qwen-7B is a bigger-GPU / approved-cloud fine-tune target, not a local fit here.
79
80     ## 5. The verifier (reconciliation-as-oracle)
81
82     Already exists in seed form as `muneem.parsers.validation`. It grows into the component
that both
83     **selects** and **validates**:
84
85     - **Per-row walk:** `balance[i] == balance[i-1] ? debit[i] + credit[i]` (catches a
missed/misread row
86     *between* any two balances).
87     - **Total reconcile:** `opening + ?credits ? ?debits == closing` (works even when
there's no per-row
88     balance, e.g. Axis savings).
89     - **Bookend:** parsed Dr/Cr counts + opening/closing vs the bank's printed SUMMARY
(catches a missing
90     first/last row ? HDFC has this; others where available).
91     - **Selection:** among L0/L1/L2 candidate mappings, pick the reconciling one; if several
reconcile,
92     prefer the highest-confidence (header-backed) one.
93     - **Confidence verdict:** `reconciled` (per-row or total) + `bookended` ? trust; else
quarantine.
94
95     ## 6. Bank knowledge as *data* (profiles), not code
96
97     A bank profile is a small **declarative** object (likely YAML/dict): concept?label
aliases, accepted
98     date formats, anchor strings to locate the table, account-number patterns, known quirks
(multi-account

```

```

99     sections, "Other Information" column), and the password rule. **Generic detectors run
first; the
100     profile supplies priors/hints** to disambiguate ? it is not bespoke parsing logic.
Absorbing a small
101     format change = tweak a profile (often nothing ? the generic path copes). A brand-new
issuer = add a
102     profile, and only drop to custom code if the generic ladder genuinely can't cope.
103
104     ## 7. Failsafes ? the "traps" (loud drift, never silent)
105
106     The Axis lesson: a broken parse must *announce itself*. Built-in tripwires, each ? flag
+ provenance,
107     never store-and-pretend:
108
109     - header label matches no lexicon entry ? "unknown field, mapped by structure ? review"
110     - reconciliation breaks, or parsed counts ? printed summary ? **quarantine** (store
nothing or store
111     flagged; the oracle-gated parsers already refuse to persist unreconciled rows)
112     - a profile's expected anchor is missing ? "format may have changed for <bank>"
113     - _(L3)_ VLM output won't reconcile, or disagrees with a deterministic parse ? reject
the VLM
114     - **provenance + confidence on every parsed statement:** which layer/mapping produced
it, did it
115     reconcile / bookend. Trust is *recorded*, not assumed.
116
117     _Research (done):_ VLMs' top failure modes on financial numbers are **digit
transposition, decimal/
118     sign errors, dropped/duplicated rows, and column misalignment** ? which almost always
**break the
119     reconcile identity**, so the verifier rejects them. L3 traps therefore: gate every VLM
parse on
120     reconciliation; **self-consistency** re-sample (keep the pass that reconciles); use the
per-row walk
121     to localize a failing row for re-query or human review; cloud stays off unless
explicitly approved.
122
123     ## 8. Two tiers of "semantic"
124
125     - **Tier 1 ? field semantics** (this doc): what each column *means*. The resilience win.
Build first.
126     - **Tier 2 ? transaction semantics** (later): classify each row (UPI / NEFT / ATM /
charge / interest
127     / sweep), extract merchant/counterparty, assign category. Same schema, layered on top;
ties to the
128     parked merchant-canonicalization decision (DESIGN § 8.5).
129
130     ## 9. Data-model evolution
131
132     Today: per-bank raw tables (`hdfc_transactions`, `axis_transactions`, ?) ? fine for
fidelity, but the
133     concept schema points at a **unified `transactions` table** keyed on the canonical
concepts, plus
134     `account`/`document` context and **`provenance` + `confidence`** columns. Migration is
additive (the
135     per-bank tables can become views or be backfilled); sequence it after the core lands so
we don't churn
136     storage twice.
137
138     ## 10. Implementation plan (incremental, merge-as-we-go)
139
140     1. **? Core extraction ? landed.** `validation.py` is now the verifier: the per-row
walk, **total
141     reconcile** (`opening + ?credit ? ?debit == closing`), the concept-level **bookend**
(`verify`),
142     **candidate selection** (`select_reconciling`), and a recorded **confidence** verdict

```

```

143         (`Confidence`). The concept schema lives in `concepts.py` (`ConceptRow`,
`StatementConcepts`).
144         **HDFC is refactored onto it as the reference implementation** ? it maps its rows
onto the
145         concepts and verifies through the shared verifier, with **no behavior change** (every
existing
146         test passes; the relocated bookend logic is identical).
147     2. **? Concept-mapping engine (L0/L1/L2) ? landed.** `engine.py` maps a cell-matrix onto
concepts
148         via header lexicon (L0) ? profile column-order ? content search (L2), keeping the
assignment the
149         reconciliation oracle endorses (the **self-identifying balance column**). It is
**family-agnostic**
150         ? parameterized by a `FamilySpec` (concept names + typed-row builder) and an injected
oracle, so
151         an instrument family (equity/MF/ETF/bond) reuses the same control flow with its own
concepts +
152         oracle. Issuer knowledge is `profiles.py` ? **data, not code**. The engine ships
validated on
153         synthetic fixtures; HDFC consumes its profile (account pattern) while keeping its
bespoke
154         coordinate-clustering extraction (the documented `extract_tables` exception). The
first real
155         issuers driven through the engine are **Axis + AU Bank** (step 3).
156     3. **? Axis + AU Bank rewrite onto the core ? landed.** Both run the shared engine-
driven savings
157         path (`savings.py`): per-table + coordinate-clustered candidates ? engine mapping ?
reconciliation
158         oracle (per-row walk, else total reconcile), oracle-gated. Real-data precision
(local): **AU 4/4**
159         reconcile + persist; **Axis 2/4** (Aug'24, Oct'25) ? May/Jun'25 are vintages
`extract_tables`
160         mangles, so they store nothing (the safety invariant holds). HDFC stays bespoke
clustering.
161         Recovering the remaining Axis vintages (better clustering) is follow-up work.
162     4. **L3 local-vision fallback** (research done; build deferred): **DeepSeek-OCR-3B
(MIT)** , fired
163         *only* when the deterministic ladder fails to reconcile (e.g. the Axis May/Jun
vintages); output
164         accepted only if it reconciles, self-consistency re-sample on a miss, then human
review. Dev RTX
165         2070S ? 4-bit + SDPA; Qwen2.5-VL-7B (Apache) is the heavier / fine-tune option on a
bigger GPU.
166         See $4, $7, and the `l3-vision-and-eval` memory.
167     5. **Unify storage** ($9) + begin **Tier-2 transaction semantics**.
168
169     Each step is a green, reversible PR; the verifier guarantees we never regress into
storing garbage.
170
171     ## 10a. Golden eval sets (high-confidence labels)
172
173     Ground truth is built three ways, cheapest-risk first ? confidence comes from
*reconciliation +
174     consensus*, never from trusting any single extractor:
175
176     - **Local-golden (free, now):** any parse that **reconciles + bookends** is already a
gold label ?
177         HDFC 4/4 and AU 4/4 qualify today. Harvest them as the eval set; no cloud, no new
risk.
178     - **Synthetic-golden (free, CI-able):** statements authored in the real issuer layouts
with
179         fabricated PII/amounts ? safe to commit (`testdata/synthetic/`) and to run any model
against.
180     - **Cloud-golden (eval-only, approval-gated, off by default):** for a layout no local
method can

```

```

181     label, a *zero-retention* cloud doc-AI may extract a minimal real subset as one
voter**,
182     cross-checked by reconciliation ? never in the runtime path, never without explicit
per-category
183     approval (DESIGN §5).
184
185     This dovetails with the Regression CI plan (ROADMAP § "Planned ? Regression CI for
statement
186     parsing"): synthetic-golden ? the always-on cloud-CI layer; local-golden ? the self-
hosted
187     real-statement layer.
188
189     ## 11. Open questions (for discussion)
190
191     - Storage unification timing ? keep per-bank tables until the concept core is
proven, then migrate?
192     - Generic-vs-profile balance ? how much to lean on profiles as priors vs. pure
content inference.
193     *Provisional (Step 2): generic detectors run first; the profile supplies aliases /
column-order /
194     anchors / account patterns as priors. Revisit once more issuers land.*
195     - L3 scope ? *model resolved (DeepSeek-OCR-3B; §4).* Still open: rare fallback vs
routine
196     cross-check on every statement (lean rare / reconcile-fail-only, given the 2070S
budget).
197     - Tier-2 ? when do transaction classification + merchant canonicalization become in-
scope?
198     - Confidence policy ? what does the CLI/ledger do with a `quarantined` statement
(skip, show with
199         a warning, queue for the VLM)?
200

```

7. user (2026-06-10T16:13:42.333Z)

```

1     # muneem ? Database schema (living document)
2
3     > Schema version: 5 (`PRAGMA user_version`; baseline v4 in
`src/muneem/db.py:_SCHEMA` + forward
4     > migration v5 `provenance` in `src/muneem/migrations.py`). This document mirrors them
for humans. The
5     > shape is pinned by the schema-lock test (`tests/test_db_invariants.py`) ?
accidental drift fails
6     > CI, and any deliberate change must update the lock and ship a migration in the
same PR.
7     >
8     > The database is encrypted at rest (SQLCipher; see [encryption-at-
rest.md](`./encryption-at-rest.md`)).
9     > All tables below live inside the keyed connection; `:memory:` and tests use plaintext
stdlib sqlite3.
10
11     ## Current state & direction (DECIDED 2026-06)
12
13     muneem uses flat, per-issuer transaction tables (`axis_transactions`,
`axis_cc_transactions`,
14     `au_transactions`, `hdfc_transactions`, extended per issuer/instrument) as the source
of truth ?
15     *by design*, not a crutch (DESIGN §8.7, feedbackJune2026 P3). Each parser owns one
isolated table, so
16     a regression refreshes one table and backfill/rollback is trivial. Cross-source
querying is a
17     derived serving layer ? a SQL view / materialised table now, a separate query-
optimised "serving
18     DB" only when latency/cost demand it; the flat tables stay the rollback-able truth.
19
20     Provenance + confidence are first-class but a SEPARATE table (`provenance`, below),

```



```

keyed by
21     `document_id` ? every txn row reaches its provenance by the existing FK join, so it is
*not*
22     duplicated per row. `source_type` readies it for non-PDF sources (AA / SMS) should they
ever land.
23
24     ## Conventions
25     - Money columns are `REAL`. Dates are stored as `TEXT` (issuer-formatted strings, not
normalised yet).
26     - Every per-bank txn table references `documents(id)` with `ON DELETE CASCADE`, so
deleting a source
27     document removes its rows. Foreign keys are enforced (`PRAGMA foreign_keys = ON` in
`connect()`).
28     - `documents.sha256` is `UNIQUE` ? re-ingesting the same file is a no-op (dedup).
29
30     ## Tables
31
32     ### `documents` ? every ingested file
33     | column | type | notes |
34     |---|---|---|
35     | id | INTEGER | PK |
36     | source | TEXT | NOT NULL (e.g. `file`, `gmail`) |
37     | external_id | TEXT | provider id / filename |
38     | sender | TEXT | |
39     | subject | TEXT | |
40     | received_date | TEXT | |
41     | doc_type | TEXT | `promo`, `bank_statement`, ? |
42     | sha256 | TEXT | NOT NULL, **UNIQUE** (dedup key) |
43     | path | TEXT | resolved local path |
44     | status | TEXT | NOT NULL default `parsed`; parsers set `reconciled` / `unreconciled` |
45     | created_at | TEXT | NOT NULL default `datetime('now')` |
46
47     ### `offers` ? promo codes (Track A)
48     | column | type | notes |
49     |---|---|---|
50     | id | INTEGER | PK |
51     | document_id | INTEGER | NOT NULL ? `documents(id)` ON DELETE CASCADE |
52     | merchant, promo_code, description, expiry_raw, expiry_type | TEXT | |
53     | source | TEXT | NOT NULL default `text` (`text` = exact layer, `ocr` = approximate) |
54     | redeemed | INTEGER | NOT NULL default 0 |
55     | source_text | TEXT | provenance snippet |
56
57     Index: `idx_offers_document` on `offers(document_id)`.
58
59     ### Per-bank transaction tables
60
61     **`axis_transactions`, `au_transactions`** (savings; identical shape):
62     `id` PK . `document_id` NOT NULL ? documents (cascade) . `account_number` . `txn_date` .
`value_date`
63     . `particulars` . `debit` REAL . `credit` REAL . `balance` REAL.
64
65     **`hdfc_transactions`** (savings):
66     `id` PK . `document_id` NOT NULL ? documents (cascade) . `account_number` . `txn_date` .
`value_date`
67     . `particulars` . `ref` . `withdrawal` REAL . `deposit` REAL . `closing_balance` REAL.
68
69     **`axis_cc_transactions`** (credit card ? no running balance):
70     `id` PK . `document_id` NOT NULL ? documents (cascade) . `account_number` . `txn_date` .
71     `particulars` . `merchant_category` . `amount` REAL . `type` TEXT (`Cr`/`Dr`).
72
73     ### `provenance` ? where each parse came from + how confident (migration v5)
74     `id` PK . `document_id` INTEGER NOT NULL **UNIQUE** ? documents (cascade) .
75     `source_type`
76     (`pdf_statement` / `aa` / `sms` / `manual`) . `parser` . `parser_version` .
77     `mapping_layer`

```

```

76      (`deterministic` / future `l3_vision` / `aa_schema` / ?) . `confidence` (the reconcile
verdict;
77      `NULL` for sources with no running balance, e.g. credit cards) . `created_at`. **One row
per
78      document** (upserted on re-parse), reached from any txn row via its `document_id` FK ?
first-class but
79      not duplicated per row. Recorded by `db.record_parse()` / `db.record_provenance()` in
the parsers.
80
81      ### `schema_migrations` ? migration bookkeeping
82      `version` INTEGER PK . `name` TEXT NOT NULL . `checksum` TEXT NOT NULL . `applied_at`
TEXT NOT NULL.
83      Records every applied migration (the v4 `baseline` plus any forward migrations) with a
checksum.
84
85      ## Migrations
86
87      **Implemented** (feedbackJune2026 **P6**): a small **hand-rolled migration runner**
88      (`muneem/migrations.py`) ? **not Alembic** (keeps the raw-`sqlite3`, `stdlib-light
design). The
89      current `_SCHEMA` is the **frozen baseline** (version 4); a fresh DB gets it applied, a
legacy DB
90      (tables present, no `schema_migrations`) is detected and back-filled. Future changes are
**forward
91      migrations** (`MIGRATIONS`, version > 4), applied in order *inside the keyed connection*
and recorded
92      in `schema_migrations` with a checksum; the runner refuses to proceed if the frozen
baseline was
93      edited instead (drift detection). `init_db` runs it idempotently.
94
95      **Rules:** never edit the frozen baseline for a change ? add a forward migration. Update
this doc and
96      the schema-lock (`EXPECTED_SCHEMA` in `tests/test_db_invariants.py`) in the same PR;
that diff is the
97      "explicit allow." Known limitation: a crash between a migration's DDL and its record may
need manual
98      cleanup (per-migration transactions land with the first non-trivial migration).
99

```

8. user (2026-06-10T16:13:48.115Z)

```

1      # Ingesting bank statements (with password unlock)
2
3      `muneem ingest` reads a bank-statement PDF, **unlocks it if it's password-protected**,
parses it
4      into normalized transactions, and stores only a parse that **reconciles** (the running
balance
5      walks, or the totals match the printed opening/closing). Supported issuers today:
**HDFC, Axis, AU
6      Bank**.
7
8      ```bash
9      muneem ingest /path/to/statement.pdf --doc-type bank_statement
10     ```
11
12     If the PDF is encrypted, muneem resolves the password with a **hybrid strategy** (no
password is ever
13     written to the repo or logged):
14
15     1. **Cached candidates** ? passwords from earlier successful unlocks, stored in your
**OS keychain**.
16     2. **Configured candidates** ? entries in `password_rules.yaml` (see below).
17     3. **Interactive prompt** ? a hidden prompt (`getpass`). A password entered here that
works is
18         **cached in the keychain**, so the next ingest of any statement with the same

```

```

password is
19     friction-free.
20
21     On success you'll see, e.g.:
22
23     ```
24     ingested statement.pdf (doc 1); parsed 37 transaction(s) via HDFCParser [reconciled]
25     ```
26
27     `[reconciled]` means the parse is trusted; `[unreconciled]` means it couldn't be
verified, so
28     **nothing was stored** (muneem never persists mislabeled rows). Read parsed rows back
with
29     `muneem txns list hdfc` (or `axis` / `au`).
30
31     ## Enable unattended unlocking (recommended)
32
33     For batch/backfill or `--no-prompt` runs, put your statement passwords in a
**gitignored** rules
34     file so you're never prompted.
35
36     1. Find your config directory:
37
38     ```bash
39     muneem info          # ? "config dir : /home/you/.config/muneem" (varies by OS)
40     ```
41
42     2. Create `/password_rules.yaml`. **Never commit it** ? `.gitignore` already
blocks
43     `password_rules.*`, and it lives in your OS config dir (outside the repo) anyway.
44
45     ```yaml
46     # <config-dir>/password_rules.yaml  ?  NEVER commit this file.
47     # Quote every password (keeps leading zeros; avoids YAML parsing "0106" as a number).
48
49     # Per-issuer passwords:
50     hdfc: "<your-hdfc-pdf-password>"
51     axis: "<your-axis-pdf-password>"
52     aubank: "<your-au-pdf-password>"
53
54     # ?or a bank-agnostic list of passwords to try (handy when the issuer label varies):
55     candidates:
56     - "<password-a>"
57     - "<password-b>"
58     ```
59
60     muneem tries **all** of these (it has to pick the password before it can read the PDF
to know
61     the issuer), so either form works. A legacy `./password_rules.yaml` in the current
directory is
62     still read but warns ? prefer the config dir.
63
64     3. Ingest without prompts:
65
66     ```bash
67     muneem ingest statement.pdf --doc-type bank_statement --no-prompt
68     ```
69
70     `--no-prompt` uses only cached + configured candidates and **never** opens an
interactive prompt
71     (so it's safe in cron/CI). Without it, an unresolved password falls back to the
hidden prompt.
72
73     ## Security
74

```

```

75      - **Nothing sensitive is committed or logged.** `password_rules.*` is gitignored;
resolved passwords
76      live only in memory and the **OS keychain** (Windows Credential Manager / macOS
Keychain / Linux
77      Secret Service), via `muneem.secrets`.
78      - **No decrypted copy is written to disk** ? the password is handed straight to the text
extractor
79      and the parser in memory.
80      - Transaction parses are **reconciliation-gated**: an unverifiable parse stores nothing
rather than
81      risk mislabeled rows.
82
83      ## Troubleshooting
84
85      | Message | Meaning / fix |
86      |---|---|
87      | `Could not unlock <file>: no working password` | None of the cached/configured
candidates worked. Run without `--no-prompt` to type it, or add it to `password_rules.yaml`. |
88      | `No parser recognizes <file> as a supported bank statement` | The issuer isn't
supported yet (today: HDFC, Axis, AU Bank), or the text didn't match. |
89      | `? [unreconciled]` (0 transactions) | The parser couldn't verify the parse (e.g. an
Axis vintage `extract_tables` mangles); nothing was stored. See [`parser-
architecture.md`](./parser-architecture.md). |
90
91      ## Headless Linux note
92
93      Keychain storage needs a backend. Use a desktop keyring (GNOME Keyring / KWallet), or
94      `pip install keyrings.alt` for a file-based backend. Until then, `password_rules.yaml` +
`--no-prompt`
95      works without any keychain (you just won't get prompt-caching).
96

```

9. user (2026-06-10T16:13:48.555Z)

```

1      # Onboarding a new bank (seed ? golden)
2
3      A repeatable, security-respecting recipe for adding a new issuer's savings/transaction
statements,
4      built on the concept-core parser ([`parser-architecture.md`](./parser-architecture.md)).
Most of the
5      time it's **adding a profile (data), not writing a parser**.
6
7      Two principles do the heavy lifting:
8      - **Reconciliation is the oracle** ? a parse is trusted only if the running balance
walks (or the
9      totals match the printed opening/closing). So a wrong parse is *rejected*, never
stored.
10     - **Real statements never leave the machine, ever committed.** We validate on local data
and ship
11     *synthetic* fixtures (real layout, fabricated values) for cloud CI.
12
13     ## The loop
14
15     ### 1. Seed (2?3 real statements) ? build/tune the profile
16     Put a few real statements of the new bank in a **local, gitignored** folder (e.g.
17     `testdata/<bank>-statements/`). Add a `Profile` in `muneem/parsers/profiles.py` ? *data
only*:
18     concept?label aliases, accepted date patterns, account-number regex, quirks. For a
transaction-table
19     bank, that profile usually runs straight through the shared engine path (`savings.py`);
only add a
20     new parser module if `can_handle` / DB-table wiring is needed (mostly boilerplate, see
`au.py`).
21
22     Iterate with the harness until the seeds reconcile:

```

```

23
24     ```bash
25     muneem eval testdata/<bank>-statements/          # uses password_rules.yaml / keychain to
unlock
26     ```
27
28     It prints, per file, the outcome (`reconciled` / `unreconciled` / `locked` / `no-parser`
/ `error`),
29     the row count, and the parser ? then an overall reconcile rate. No statement values
are printed.
30     Tune the profile (and, if needed, the clusterer) until your seeds read `reconciled`. The
seeds
31     *prove* the parser; they stay local.
32
33     > If a vintage won't reconcile, that's the oracle doing its job ? it stores nothing
rather than
34     > mislabeled rows. Diagnose with the (masked) clustering tools; the `unreconciled` files
are safe.
35
36     ### 2. Golden (2?5 real statements) ? the high-confidence eval set
37     The reconciling parses are **self-labeling gold**: reconciliation *is* the label. Keep a
wider set of
38     real statements locally; they run in the **Layer-2 self-hosted CI** (real data stays on
your machine,
39     GitHub only orchestrates ? see ROADMAP § "Planned ? Regression CI"). `muneem eval` on
the golden
40     folder is the quick local gate.
41
42     ### 3. Synthetic fixture ? gate the parser in cloud CI
43     Add a born-digital fixture in the new bank's **layout** with **fabricated** data, using
the stdlib
44     generator `tests/synthpdf.py`, and assert a full-pipeline `reconciled` (see
45     `tests/test_regression_synth.py`). Commit it to `testdata/synthetic/` ? it's safe (no
real data) and
46     runs on every push (Layer-1), so the parser is regression-gated without exposing
statements.
47
48     ### 4. Land
49     A PR adds: the `Profile` (+ a thin parser module if required) + the synthetic fixture.
The golden
50     real statements are registered locally for Layer-2 ? **never committed** (`.gitignore` +
51     `.github/precommit_check.py` enforce this).
52
53     ## Checklist
54     - [ ] Real seeds in a gitignored folder; password in `password_rules.yaml` (config dir)
or keychain.
55     - [ ] `Profile` added (aliases / date patterns / account regex / quirks).
56     - [ ] `muneem eval <seed-folder>` ? seeds `reconciled`.
57     - [ ] `muneem eval <golden-folder>` ? golden set `reconciled` (your local high-
confidence set).
58     - [ ] Synthetic fixture in `testdata/synthetic/` + a `test_regression_synth`-style
assertion (Layer-1).
59     - [ ] PR = profile (+ parser module if needed) + synthetic fixture. No real statements
committed.
60

```

10. user (2026-06-10T16:13:56.286Z)

```

1     # Encryption at rest (the local ledger)
2
3     > **Status:** implemented 2026-05 (resolves ROADMAP **D.2**; satisfies DESIGN §5.4 rule
4).
5
6     muneem aggregates the user's entire financial life into one local SQLite file ? a
honeypot.

```

```

6      That file is **encrypted at rest** so that, on its own, it discloses nothing without the
key.
7
8      ## What we use, and why
9
10     **SQLCipher** (transparent, full-database **AES-256** page encryption) via the
**`sqlcipher3`**
11     binding. Chosen over the alternatives because it keeps the ledger **fully queryable** ?
`WHERE`,
12     `ORDER BY`, `SUM`, `GROUP BY` all work unchanged, which app-level/field encryption
breaks ? while
13     staying a near drop-in for the stdlib `sqlite3` API (PEP-249). It ships prebuilt,
permissively
14     licensed wheels for Windows + Linux (cp311?cp313), so there is no source/JVM build
burden.
15
16     | Option considered | Verdict |
17     |---|---|
18     | **SQLCipher via `sqlcipher3`** | ? **Chosen.** Transparent AES-256, queryable, drop-in
API, permissive, wheels. |
19     | SQLite3 Multiple Ciphers via `apsw-sqlite3mc` | Strong fallback (MIT, no OpenSSL, AEAD
cipher) ? but APSW is **not** PEP-249, so adopting it is a DB-layer refactor. Switch only if we
want OpenSSL out of the tree. |
20     | App-level field encryption (`cryptography`) | ? Breaks SQL queries over the numbers;
deterministic-encryption workaround leaks via frequency analysis (recurring
salary/rent/subscriptions are fingerprints). Not a primary mechanism. |
21     | OS full-disk encryption only (BitLocker/LUKS) | Necessary baseline, **not** a
substitute ? it doesn't follow the file off-disk (backups, USB, sync). Complement, documented
below. |
22
23     **Licensing** (all permissive ? clears `tests/test_dependency_licenses.py`):
`sqlcipher3` = Zlib;
24     bundled SQLCipher core = BSD-3-Clause; SQLite = Public Domain; OpenSSL = Apache-2.0.
25
26     ## Key management
27
28     - A random **256-bit** key is minted on first use (`secrets.token_bytes(32)`) and stored
in the
29     **OS keychain** via `muneem.secrets` (`muneem`/`db-key`) ? Windows Credential Manager
(DPAPI),
30     macOS Keychain, or Secret Service on Linux. Same mechanism as OAuth tokens.
31     - It is handed to SQLCipher in **raw-key form** (`PRAGMA key = "x'<64 hex>'"`), which
uses the key
32     directly and **skips the passphrase KDF** ? correct because the key is already full-
entropy.
33     - **Never** written to the repo, config, CLI args, or logs. `db.py` centralizes the
`PRAGMA key`
34     so the raw-hex form is used everywhere (mixing raw-key and passphrase forms silently
looks like
35     corruption).
36
37     **Key rotation** is implemented: `muneem db rotate-key` mints a fresh 256-bit key, re-
encrypts the
38     file (`PRAGMA rekey`), and updates the keychain ? keychain-first with rollback, so a
failed file
39     rekey leaves the keychain matching the file. Rotate on suspected compromise; no
scheduled rotation
40     needed for a single-user local key.
41
42     ### Disaster recovery ? the key-loss problem (solved)
43
44     Because the key lives only in the keychain and `db backup` encrypts backups with that
*same* key, a
45     **lost keychain** (OS reinstall, new machine, profile corruption) would otherwise
destroy the ledger

```

```

46      *and every backup*. So a passphrase-wrapped recovery file is implemented
(`muneem.recovery`):
47
48      - muneem db setup-recovery ? wraps the current DB key under a user passphrase
49      (Argon2id KDF ? AES-256-GCM) into muneem-recovery.json. The file holds only
the wrapped
50      key (salt/nonce/ciphertext + KDF params) ? never the key, never the passphrase.
Store it
51      off-machine (password manager / separate drive). The pre-commit guard blocks
recovery.json
52      from the repo.
53      - muneem db recover <file> ? prompts for the passphrase, unwraps the key (a wrong
passphrase
54      fails the GCM tag ? clean error, no oracle), verifies it opens the ledger, and
restores it to the
55      keychain. You need both the file and the passphrase.
56
57      Deferred (opt-in, ROADMAP D.2 Phase 2): a full passphrase-lock mode (the working
key itself
58      derived from a passphrase, never stored) for stronger-than-keychain protection on a
shared machine ?
59      distinct from the recovery file above, which keeps the frictionless keychain key as
primary.
60
61      ## Threat model ? stated honestly
62
63      Keychain + SQLCipher defends against: a stolen or discarded disk/file, the DB
ending up in
64      a cloud-synced backup / USB / email, and other OS users. It does not defend
against
65      malware running as the logged-in user ? that attacker can read the keychain and dump
decrypted
66      pages from RAM. Only the deferred passphrase mode raises that bar (at a usability cost).
We do not
67      oversell this. (Refs: NIST SP 800-111; OWASP Password Storage.)
68
69      ? Recommended complement: enable OS full-disk encryption (BitLocker on Windows, LUKS
on Linux).
70      Verify Windows status in an elevated shell with manage-bde -status.
71
72      ## Using it
73
74      - New ledger: born encrypted automatically ? the production CLI opens via
75      db.connect_encrypted(). Nothing to do.
76      - muneem db status ? shows whether the ledger is encrypted / plaintext / absent
and whether a
77      key is minted. Reads no statement data.
78      - muneem db encrypt ? migrates a pre-existing plaintext ledger in place (via
79      sqlcipher_export), verifying the encrypted copy (integrity_check) before swapping.
The
80      original is preserved as <name>.plaintext.bak ? it still holds cleartext; verify,
then
81      securely delete it. (The pre-commit guard blocks .plaintext.bak from ever being
committed.)
82
83      :memory: and the test/CI fixtures intentionally stay on plaintext stdlib sqlite3
(the
84      synthetic regression data is non-sensitive); the encrypted path has its own tests
85      (tests/test_db_crypto.py). The test suite never touches the real keychain (autouse
fixture in
86      tests/conftest.py), so it is safe on headless CI.
87
88      Rugged CI guards (tests/test_db_invariants.py, run on every push/PR) act as merge
tripwires:
89      production ingest must write an encrypted ledger (the seam can't be silently swapped

```

```

for a
90     plaintext open), a keyed open must *raise* rather than downgrade if the binding is
missing, foreign
91     keys are enforced + cascade, and `documents.sha256` stays unique. A companion **schema
lock** pins
92     `SCHEMA_VERSION` and the table/column shape: accidental drift fails CI, while a
deliberate schema
93     change is allowed only by updating the lock in the same PR (forcing a conscious
migration decision).
94
95     ## Operational gotchas
96
97     - **Backups must not be naïve file copies while WAL is active** (you'd desync
`-wal`/`-shm` and get
98     a corrupt snapshot). **`muneem db backup <dest>`** does this safely ? it uses the
SQLite Online
99     Backup API for a consistent snapshot (safe even with WAL active) and keys the
destination with the
100     same key, so the backup stays encrypted at rest.
101     - **`PRAGMA key` must be the first statement** on a connection; we verify it immediately
with a
102     `sqlite_master` read and raise a clean `EncryptedDatabaseError` on a wrong key.
103     - **Debugging:** an encrypted DB can't be opened by stock `sqlite3` / DB Browser without
SQLCipher.
104     Use the `sqlcipher` CLI (or DB Browser *with* SQLCipher). Don't leave a plaintext copy
lying around.
105     - **Performance:** ~5?15% page-level overhead ? negligible at personal-finance scale.
106
107     ## Sources
108
109     SQLCipher API (PRAGMA key, raw key, `sqlcipher_export`, WAL):
<https://www.zetetic.net/sqlcipher/sqlcipher-api/> .
110     `sqlcipher3` wheels/license: <https://pypi.org/project/sqlcipher3/> .
111     SQLCipher core license (BSD-3):
<https://github.com/sqlcipher/sqlcipher/blob/master/LICENSE.txt> .
112     `apsw-sqlite3mc` fallback: <https://pypi.org/project/apsw-sqlite3mc/> .
113     FDE scope: NIST SP 800-111 <https://csrc.nist.gov/pubs/sp/800/111/final> .
114     key derivation: OWASP Password Storage Cheat Sheet.
115

```

11. user (2026-06-10T16:13:56.621Z)

```

1     # feedbackJune2026 ? Architectural Review: Response & Incorporation Plan
2
3     > **Status: Approved direction (2026-06).** Avi has signed off on this analysis/plan.
The $5 open
4     > decisions (flagged **? DECISION**) remain to be settled before *their* phases are
implemented and
5     > must not be silently locked by code. The low-risk Phase-A items that carry **no** open
decision
6     > (Windows CI, `docs/schema.md`, marking Outlook/IMAP experimental) are being
implemented now; the
7     > rest waits on $5.
8     >
9     > Author's note (attribution): claims about current code are grounded in the repo as of
`main`
10    > after PRs #34?#40. Where I infer or recommend, I say so.
11
12    ---
13
14    ## 0. Framing ? what "commercial-grade" means *here*
15
16    muneem is **paid, but single-user, local-first, and lifelong**: one person's entire
financial life,
17    on their own machine, for years. So the bar is **correctness · zero-leak · auditability

```



```

18     recoverability · maintainability-by-one**. It is **not** multi-tenant SaaS. That
distinction drives
19     several push-backs below: the right move is to be *bulletproof at single-user scale*,
not to import
20     enterprise machinery (heavy ORMs, distributed anything, a web tier) that adds surface
without
21     serving the actual risk. "High bar" = fewer moving parts, each provably correct and
recoverable.
22
23     Two non-negotiables carry over from CLAUDE.md and stay load-bearing in everything below:
24     1. **No sensitive data leaves the machine** without explicit per-category, per-session
approval.
25     2. **No open architectural decision gets silently locked** by code. This doc proposes;
Avi disposes.
26
27     ---
28
29     ## 1. Corrections ? three premises in the review are already stale
30
31     The review is strong, but three items were **done in the recent code-quality pass** and
should be
32     re-scoped from "needed" to "extend / verify":
33
34     | Review claim | Reality (as of `main`) | What actually remains |
35     |---|---|---|
36     | "currently zero logging in src/" | **False now.** `muneem/log.py` (`get_logger` +
`RedactingFilter`) + `muneem/redaction.py` landed in **#36**, wired into `cli`, with masking
tests (`test_redaction`, `test_log`). | Structured (JSON/logfmt) output + **broad adoption**
across modules + per-stage leak tests. The *foundation* exists. |
37     | "muneem db backup (proper Online Backup API, not naive cp)" | **Done in #39.**
`backup_database()` uses the SQLite Online Backup API (WAL-safe), destination keyed ? encrypted
backup. | A documented restore runbook; optional *recovery-key* backups (see §2). |
38     | "key rotation scaffolding (the deferred passphrase path)" | **Rotation done in #39.**
`rekey_database()` + `muneem db rotate-key` (keychain-first with rollback). | The **passphrase
path** specifically ? but reframed as **recovery**, not just "stronger" (see §2). |
39
40     Also already in place (so the review's "table-stakes" list is largely satisfied):
SQLCipher
41     encryption-at-rest (#34), schema-lock + DB-invariant tripwires (#34), pre-commit secret
guard,
42     license allowlist, **mypy** + **ruff** + **80% coverage gate** in CI and the pre-push
hook (#38/#40),
43     verifier property-fuzz (#40). The engineering-hygiene floor is commercial-grade *today*.
44
45     ---
46
47     ## 2. ? The biggest risk the review *under-weights*: key-loss / disaster recovery (P0)
48
49     Encrypting the ledger at rest (correctly!) created a **catastrophic single point of
failure** that
50     none of the ten bullets names directly:
51
52     > The 256-bit DB key lives only in the OS keychain. **`muneem db backup` encrypts the
backup with the
53     > same key.** So if the keychain is lost ? OS reinstall, profile/Credential-Manager
corruption, dead
54     > machine, new laptop ? the ledger **and every backup** are *permanently unrecoverable*.
55
56     For a tool whose entire pitch is "hold my whole financial life for years," this outranks
most of the
57     listed features. It must be solved **before** we accumulate real, irreplaceable history.
58
59     **The right way (reframe the deferred "passphrase mode" from security ? recovery):**
60     - Keep the random DB key (fast, full-entropy) as the working key.

```

```

61      - Add a recovery wrapper: encrypt the DB key under a user passphrase (Argon2id,
OWASP params)
62      and store that wrapped blob in a `recovery.json` the user backs up off-machine
(password
63      manager / printed). Losing the keychain ? restore the key from passphrase + recovery
file.
64      - And/or `muneem db export-key` ? prints the raw key once, with loud warnings, for
the user's
65      password manager. Simpler; less safe-by-default.
66      - Optionally let `db backup --recovery-key` write a backup under the passphrase-derived
key, so a
67      backup is restorable even with the keychain gone.
68      - Ship a docs/recovery.md` runbook: "keychain lost," "move to a new machine,"
"verify a backup."
69
70      DECISION: which recovery model ? (a) passphrase-wrapped recovery file
*(recommended)*, (b)
71      one-time key export, (c) both. Pick before storing real numbers long-term.
72
73      ---
74
75      ## 3. Point-by-point: verdict · the right way · dependencies
76
77      Verdicts: Accept · Accept-with-changes · Push back · Largely
done.
78
79      ### P1 ? Structured, redacted logging (D.1) · ? Accept-with-changes
80      Premise stale (see §1). Right way: build on `log.py`/`redaction.py`; add a
structured formatter
81      (logfmt or JSON ? ? minor decision), adopt `get_logger(__name__)` across `parsers/`,
`db`, `unlock`,
82      `ingestion` at sensible levels; keep `RedactingFilter` as the backstop, not the
primary control
83      (code passes already-redacted context). Tests: feed synthetic sensitive values
through each
84      stage* and assert masking (extend the #36 tests). The other half of D.1 ? idempotent,
resumable
85      re-runs* ? is a pipeline concern (staging dir + `documents.status`), partly
separable; don't
86      conflate. Priority: high, low-risk ? Phase A.
87
88      ### P2 ? Two-layer regression CI · ? Accept (already the plan)
89      Layer 1 (synthetic full-pipeline via `synthpdf.py`) exists (`test_regression_synth.py`);
expand it
90      for more HDFC/Axis/AU header variants + bookend drift (this also absorbs the deferred
"adversarial
91      synthpdf fixtures" follow-up from the code pass). Layer 2 (self-hosted runner over real,
gitignored
92      statements; passwords from `password_rules.yaml`, never GitHub secrets) is
decided/deferred.
93      Caveat to record: a self-hosted runner executes CI on Avi's personal machine ? fine
for a
94      no-external-contributor repo, but note it runs only when the machine is on, and never
enable it for
95      forked PRs. DECISION: confirm self-hosted-runner acceptance. Phase D.
96
97      ### P3 ? Storage shape + provenance/confidence · ? DECIDED (2026-06)
98      Decision (Avi): keep flat, per-issuer source-of-truth tables ? each parser
writes its own
99      isolated table (today's `axis_` / `hdfc_` / `au_`, extended per issuer/instrument).
Rationale:
100     regression isolation (a bad parse refreshes one table), trivial backfill/rollback,
parser-faithful
101     storage. Cross-source querying is served by a derived serving layer ? a SQL view /
materialised

```

102 table** now, graduating to a separate query-optimised "serving DB" only when
 latency/cost demand it;
 103 the flat tables remain the rollback-able **source of truth**. This **supersedes** the
 earlier "core +
 104 per-family canonical table" sketch, and fits the reconcile-oracle philosophy *better*
 (each source
 105 stays independently verifiable). Endorsed ? no strong reason to override the flat-truth
 + serving-
 106 layer (bronze?silver) split.
 107
 108 **Provenance + confidence are first-class but NOT per-row** (Avi): recorded in a
 **separate
 109 `provenance` table keyed by `document\_id`** ? `source\_type` (pdf/aa/sms/manual),
 `parser`,
 110 `parser\_version`, `mapping\_layer`, `confidence`. Every txn row already FKs
 `documents(id)`, so it
 111 reaches its provenance by join with **zero redundancy**. The reconciliation oracle
 treats every
 112 source (PDF / AA / SMS) as a **hint to validate**, never ground truth. **Phase B = the
 `provenance`
 113 table (the first forward migration) + a serving view; instrument-family tables follow
 the same
 114 per-issuer-flat pattern.**
 115
 116 ### P4 ? L3 vision fallback + golden evals · ? Accept-with-changes (sequence!)
 117 Decision stands (DeepSeek-OCR-3B, MIT, 4-bit+SDPA on the 2070S, reconcile-gated, self-
 consistency
 118 resampling). **Push-back on ordering:** L3 should come **after D.3** ? the eval writeup
 that
 119 *characterizes where the deterministic ladder actually breaks*. Building a fallback
 before we know
 120 its target failure modes is guessing. So: **D.3 eval first ? then L3.** Golden sets
 (local +
 121 synthetic now; cloud eval-only, approval-gated) extend `muneem eval`. **Phase E (after
 D.3).**
 122
 123 ### P5 ? Resolve/time-box parked decisions (§8.3 LLM locality, §8.5 merchant canon) · ?
 Accept
 124 These stay parked until a milestone *forces* them, and surface to Avi with the
 hardware/latency
 125 constraints first. **Hard rule restated:** no cloud path for bank/CC/broker data, even
 126 experimentally, without per-session approval + redaction + a test gate. Merchant canon
 (§8.5) is
 127 downstream of P3+P8. **Governance item ? not a build; revisit at the phase boundary.**
 128
 129 ### P6 ? Versioned, tested schema migrations + `docs/schema.md` · ? Accept ? but ? **not
 Alembic**
 130 Accept the need; **reject Alembic** (it drags SQLAlchemy ? heavy, ORM-shaped ? against
 muneem's
 131 deliberate raw-`sqlite3`, stdlib-light design). **Right way:** a tiny hand-rolled runner
 ? numbered
 132 `migrations/NNN\_.py|sql` applied in order inside the keyed connection, recorded in a
 133 `schema\_migrations` table, each with an **idempotency/up-checksum**, gated by the
 `user\_version`
 134 pragma. Pair with the existing schema-**lock** test (drift tripwire) ? versioned +
 tested + enforced.
 135 Add a living `docs/schema.md`. Note: migrations run *inside* the encrypted connection ?
 fine, but the
 136 runbook must cover "migrate then re-verify reconciliation." **Phase A (precedes P3's
 migrations).**
 137
 138 ### P7 ? Ingestion completeness + `muneem fetch` · ? Accept ? but ? re-sequence
 139 **Push-back on "before heavy parser work":** the parser core is already *proven* (HDFC
 6/6, AU 4/4,
 140 Axis 6/7) and the user supplies statements manually today. `fetch` is

```

**automation/convenience, not
141 correctness** ? it doesn't unblock parser accuracy or the whole-finance vision;
**storage
142 unification (P3) does.** So: **P3 before P7.** What *is* cheap and should happen
immediately:
143 explicitly mark Outlook/IMAP attachment paths **experimental/deferred** in code + docs
(the mypy
144 override already treats them as such) so no one mistakes scaffolding for support. Then
build `fetch`
145 (incremental `historyId`/`deltaLink`, sender allowlist + "financial document"
heuristics, staging ?
146 `documents` rows as `locked`/`unparsed`, robust dedup) as **Phase F**. The fast bit
(mark
147 experimental) is **Phase A**.
148
149 ### P8 ? Tier-2 transaction semantics + merchant canonicalization · ? Accept
(downstream)
150 Strictly **after P3** (needs the canonical schema) and needs **real receipt data** to
expose the mess
151 (don't design merchant canon in the abstract). Counterparty/merchant extraction + fuzzy
match +
152 a user-editable **override table** ; itemized receipts (Swiggy/Amazon) likewise. **Phase
G.**
153
154 ### P9 ? Observability / ops surface · ? Accept (partly done; pull recovery forward)
155 Done: `db backup` (Online Backup API), `db rotate-key`, `db status` (#39). **Right way
for the rest:**
156 `muneem verify` (re-run reconciliation over *stored* rows ? a cheap, high-value
integrity audit that
157 also exercises provenance), `export` (CSV/JSON for the user's own analysis ? explicitly
**redaction-
158 exempt** since it's the user's own data to their own disk, but never to stdout/logs),
`txns` filters
159 (date/amount/account/status), light analytics. **Crucially, the recovery model ($2)
belongs here but
160 is P0 ? pull it into Phase A, not "later."** Document backup/restore + "keychain lost."
**Phase A
161 (recovery) + Phase H (the rest).**
162
163 ### P10 ? Packaging / distribution / reproducibility · ? Accept-with-changes
164 - **Lockfile** (uv.lock or pip-tools `requirements*.txt`): ? yes ? reproducibility is
real for a tool
165 with a compiled dep (sqlcipher3). **Phase A.**
166 - **Cross-platform CI matrix** (Windows + macOS, not just ubuntu): ? **high value,
under-rated** ?
167 the dev box is Windows, and the secret backends genuinely differ (DPAPI vs Secret
Service vs
168 Keychain) and sqlcipher3 wheels are per-OS. This directly de-risks "works on my
machine" for the
169 one machine that matters. **Phase A.**
170 - **SBOM in CI** (CycloneDX): ? reasonable for paid software but **time-box / low-
urgency** ? it
171 serves a distribution/audit story we don't have yet. Do it near first external
distribution, not
172 now. **Phase H+.**
173 - **Local web UI:** ? defer ? CLI stays authoritative; a web tier is new attack surface
+ scope for
174 no current need. Revisit only if a real UX gap appears.
175
176 ---
177
178 ## 4. Proposed sequencing (dependency-ordered)
179
180 > Rule of thumb: **recoverability + foundation first; then the one big schema decision;
then

```

```

181     > everything that depends on it.** Each phase is shippable and individually PR-able.
182
183     - **Phase A ? Foundation & recovery (low-risk, high-leverage):** $2 **recovery model
(P0)** .
184         finish structured-logging adoption (P1) . hand-rolled migration runner +
`docs/schema.md` (P6) .
185         lockfile + cross-platform CI matrix (P10) . mark Outlook/IMAP experimental (P7-fast).
186     - **Phase B ? Provenance table (first forward migration) + serving view (P3, DECIDED).**
Flat
187         per-issuer source of truth stays; add the separate `provenance` table + a cross-source
serving
188         view. Unblocks the instrument family.
189     - **Phase C ? Instrument family** (equity/MF/ETF/bond events) on the unified schema +
new
190         `FamilySpec`/oracle (P3 cont.).
191     - **Phase D ? Regression CI Layer 2 (self-hosted) + the D.3 eval writeup** (P2) ? also
tells us where
192         L3 is actually needed.
193     - **Phase E ? L3 vision fallback + golden evals (P4),** informed by D.3.
194     - **Phase F ? `muneem fetch` + ingestion completeness (P7).**
195     - **Phase G ? Tier-2 semantics + merchant canonicalization (P8),** on real receipt data.
196     - **Phase H ? Remaining ops surface (verify/export/filters/analytics) (P9) + SBOM
(P10).**
197     - **Parked/governance throughout:** $8.3, $8.5 (P5) ? surface at their forcing
milestones.
198
199     ---
200
201     ## 5. Decisions ? settled vs still open
202
203     **Settled (2026-06):**
204     1. **Recovery / key-loss model** ? **passphrase-wrapped recovery file** (Argon2id + AES-
GCM). ? shipped (#43).
205     2. **Storage shape** ? **flat per-issuer source-of-truth tables + a derived serving
layer; provenance/
206         confidence in a separate `provenance` table** (P3). Endorsed; Phase B implements the
provenance table.
207     3. **Migration mechanism** ? **hand-rolled runner** (not Alembic). ? shipped (#44).
208     8. **Open Banking / Account Aggregator** ? **AA deferred; PDF-first**, with explicit re-
adopt triggers
209         (see $7 and [`research-open-banking-india.md`](./research-open-banking-india.md)).
210
211     **Still open:**
212     4. **Structured log format** (P1) ? logfmt vs JSON.
213     5. **Self-hosted runner** (P2) ? accept the run-CI-on-my-machine trade-off?
214     6. **macOS first-class** (P10) ? Windows + Linux are first-class now (CI matrix, #42);
macOS best-effort?
215     7. **Parked:** $8.3 sensitive-doc LLM locality, $8.5 merchant canonicalization ? confirm
still parked.
216
217     ---
218
219     ## 6. Push-back summary (explicit, with reasons)
220
221     - **Alembic ? hand-rolled migrations.** Dep-minimization + raw-`sqlite3` design + the
schema-lock
222         already exists; Alembic's ORM weight buys nothing here.
223     - **"Ingestion before heavy parser work" ? storage unification (P3) first.** Parser core
is proven;
224         `fetch` is convenience, not correctness. (But mark Outlook/IMAP experimental *now* ?
cheap.)
225     - **L3 before D.3 ? reversed.** Characterize the deterministic ladder's failure modes
*first*; don't
226         build a fallback blind.
227     - **SBOM / web UI ? time-box / defer.** Low urgency vs correctness + recoverability; web

```

UI is

228 unjustified surface for a single-user CLI tool.

229 - ****Don't over-engineer for scale muneem doesn't have.**** Single-user, local-first.

Bulletproof beats

230 big.

231 - ****And the addition the review missed: key-loss recovery is P0**** ? above most of the feature list.

232

233 ---

234

235 **## 7. Open Banking / Account Aggregator (researched 2026-06) ? decided: **defer, PDF-first****

236

237 Full writeup + sources: [`research-open-banking-india.md`](./research-open-banking-india.md). Summary:

238

239 - ****Verdict:**** a single-user, local-first, no-paid-API tool ****cannot**** use India's Account Aggregator

240 (AA) today. ****FIU status requires a regulated entity**** (no individual/hobbyist path); the only

241 reachable integration rides under a licensed FIU via a ****paid TSP that decrypts your whole feed in**

242 its cloud**. So AA-primary is both **impossible** and **undesirable** under muneem's principles.

243 - ****On the security fear:**** AA is **safer in transit** (E2E-encrypted, data-blind aggregator, no

244 credential sharing) ? but ****"self-custody" is not an available AA mode****; decryption is institutional

245 (FIU/TSP cloud). For muneem's threat model (one local honeypot, zero third parties) that's a ****net**

246 privacy regression** ? so the wariness is justified, for a subtle reason (a new institutional cloud

247 tier, not weak crypto).

248 - ****Fidelity:**** AA wins on live bank rows + demat equity/MF, but ****can't cover CC, bonds, broker**

249 detail, or non-demat MF** (muneem's exact scope), caps history ~2y, and still needs muneem's

250 enrichment/oracle layer. A boost on a **subset**, not a replacement.

251 - ****Stance:**** ****PDF-first core; AA a deferred, opt-in *complement*** (import from a regulated FIU's

252 export, oracle-validated, provenance-tagged) ? never primary.** Re-adopt triggers: a TSP documents

253 ****true on-device, self-custody-key decryption**** (the key gate); CC/bonds go live on AA; a free

254 personal tier appears; or muneem's PDF coverage saturates.

255 - ****New near-term candidate it surfaced ? local SMS/email transaction-alert parsing (P11):**** RBI

256 mandates per-transaction bank SMS; mature OSS parsers exist; ****zero third party, fully local,**

257 near-real-time.** Limits: no history/balances/holdings, brittle formats, no investments ? so a

258 **freshness complement** to PDF, not a replacement, and it violates ****no**** muneem principle (echoes

259 the predecessor's email-mining DNA). Slots into Phase A (optional) / Phase F.

260

261 ---

262

263 ***Next step:** S5's settled decisions are being implemented (recovery #43, migrations #44, Windows CI

264 #42; provenance table next). Remaining open: log format, self-hosted runner, macOS. One PR per item,

265 CI green and ~80% coverage throughout.*

266

```

1      # Prior art: Email-Mining
2
3      > A short technical and conceptual writeup of the predecessor project at
   `~/Projects/Email-Mining`, captured here so muneem's contributors don't need to read that repo
   cold. Last reviewed against the Email-Mining repo at commit `4b347c6` ("Gemini changes"), April
   2025.
4
5      ## What it was
6
7      A small exploration repo from April 2025 with two halves:
8
9      1. **NLP learning notebooks** (`Tokenization/`, `Text2Vec/`) ? standard NLTK tutorial
   material: tokenization, stemming, lemmatization, NER, stopwords, bag-of-words, SMS spam corpus.
   Learning artifacts; not relevant to muneem.
10     2. **A PDF extraction prototype** (`textExtract/`) ? *this is the actual seed of
   muneem.* A working end-to-end pipeline that took a manually-saved promotional email (saved as
   PDF) and returned a short summary highlighting any promo code and expiry date.
11
12     ## Repo layout (relevant parts)
13
14     ```
15     Email-Mining/
16     ??? README.md                                # 5 lines of setup instructions
17     ??? textExtract/
18         ??? ExtractbyURL/
19         ?   ??? extract2.py                      # URL ? text + dedup'd images
   (BeautifulSoup + SSIM)
20         ??? pdfFiles/                          # ~10 saved promo emails as PDFs
21         ?   ??? Category1/                     # 2 PDFs (DataQuest, MakeMyTrip)
22         ?   ??? Category2/                     # 7 PDFs (AirAsia x3, Flipkart,
   MakeMyTrip, Swiggy, Unacademy)
23         ?   ??? TestSet/                      # 1 held-out PDF (TataNeu)
24         ??? pythonFiles/
25         ??? extract_ImgTxt.py                  # main entry: PDF ? text + images ? OCR ?
   summarize
26         ??? imageText.py                      # Gemini Vision API wrapper for image OCR
27         ??? summarize.py                      # Gemini text API wrapper for promo-code
   extraction
28         ??? extracted_content/                 # outputs from the last run
29     ```
30
31     ## The pipeline (as it existed)
32
33     ```
34     manually saved email-as-PDF
35         ?
36         ?
37     PyMuPDF (fitz).open()
38         ? ? per-page get_text("text") ? page text
39         ? ? per-page get_images(full=True)
40         ?   ?? filter by size (? 300x300)
41         ?   ?? dedupe by MD5 hash of PNG bytes
42         ?
43     for each surviving image:
44         Gemini 1.5 Flash with prompt "Extract all the available text from the image."
45         ?
46         ?
47     concatenated text = page text + "\n--- Text from imageN ---\n" + image OCR text
48         ?
49         ?
50     Gemini 1.5 Flash with prompt
51         "Give me a very short summary of this email text. If some promo code is
52         available, please include it, and its expiry date. If not available, then
53         give 1-2 line containing the main words."
54         ?

```

```

55         ?
56     stdout
57     ...
58
59     Run with: `GEMINI_API_KEY=... python extract_ImgTxt.py` after editing line 55 to point
at a PDF.
60
61     ## Why this matters to muneem
62
63     The vision is the same: **structured value sits in the user's inbox, going unread;
extract it.** Email-Mining narrowed that to "promo codes I'm forgetting." muneem widens it to
the user's full financial life, with the forgotten-deals use-case surviving as one of several
analytics surfaces.
64
65     This is not a pivot. It is a generalization.
66
67     ## Patterns to carry forward
68
69     1. **Joint text + image extraction (the approach, not the library).** The pattern is
right ? but PyMuPDF/`fitz` is **AGPL-3.0**, which muneem cannot use (commercial-safety
principle). muneem uses **pdfminer.six** (MIT) for text and will use **pypdfium2** (Apache/BSD)
for rendering. See [DEPENDENCIES.md](./DEPENDENCIES.md).
70     2. **Image deduplication by hash + SSIM.** Bank statements repeat the same
letterhead/logo on every page. Without dedup we'd OCR identical chrome 30 times per document.
Email-Mining used MD5 of PNG bytes + `skimage.metrics.structural_similarity` with a 0.95
threshold. Reasonable starting point.
71     3. **The pipeline *shape* ? *page text + per-image OCR ? unified text ? structured
extraction*. This generalizes from "free-form summary" to "structured fields per schema"
cleanly. Same pipeline, different sink.
72     4. **The `pdfFiles/` corpus.** Real-world promo PDFs from real Indian merchants are
valuable test inputs. Copied to muneem at [testdata/promo-emails/](../testdata/promo-emails/).
73
74     ## Choices NOT to inherit
75
76     1. **Manually saving emails as PDF.** The whole point of muneem is to automate this. Use
Gmail API / Microsoft Graph instead.
77     2. **Cloud LLM (Gemini) as the default for everything.** Acceptable for promo codes (low
sensitivity); unacceptable for bank balances, transaction lines, holdings. See muneem's
`DESIGN.md` § 5 (Security posture).
78     3. **One-shot stdout.** Replace with persistent normalized storage (SQLite per
`DESIGN.md` § 4 Stage 4).
79     4. **Edit-line-55-to-change-input.** The prototype hardcoded a PDF path inside
`extract_ImgTxt.py`. Obvious ? make inputs proper arguments / config.
80     5. **No schema, no validation, no error handling.** It's a prototype; muneem isn't.
81     6. **No tests.** muneem should have at minimum a regression test that runs the pipeline
against `testdata/promo-emails/` and asserts that known promo codes are recovered.
82     7. **The NLP notebooks.** Tutorial material. Leave them in Email-Mining as personal
reference.
83
84     ## What Email-Mining never had to address (but muneem does)
85
86     - **Email fetching.** OAuth, scopes, search, rate limits, incremental sync.
87     - **Password-protected PDFs.** Decryption is mechanical; password sourcing is the design
problem.
88     - **Heterogeneous formats at scale.** Email-Mining handled ~10 promo emails of broadly
similar shape. muneem will see dozens of issuer formats that all look different and change
yearly.
89     - **Persistence and normalization.** A schema spanning accounts, transactions, holdings,
merchants, items, offers ? and a canonicalization story for merchant identity.
90     - **Security as a first-class constraint.** Email-Mining was a coupon-finder; the worst-
case leak was a expired DataQuest code. muneem aggregates the user's full financial life ? the
threat model is categorically different.
91
92     ## The corpus, in muneem
93

```


94 All 10 PDFs from Email-Mining's `pdfFiles/` are preserved at [`testdata/promo-
emails/`](../testdata/promo-emails/). The original `Category1` / `Category2` / `TestSet`
directory split was not obviously documented (likely "simple" / "complex" / "held-out"); the
categorization is recorded in that directory's `README.md` for whoever wants to re-establish a
train/test split.
95

13. user (2026-06-10T16:14:03.695Z)

```
1      # Research: India Open Banking (Account Aggregator) vs native PDF parsing
2
3      > **Status: research finding (2026-06), informs roadmap ? not a commitment.** Compares
the RBI
4      > **Account Aggregator (AA)** framework against muneem's native statement-PDF parsing,
for a
5      > single-user, local-first, privacy-first, no-paid-API tool. Backed by a 6-angle
research workflow
6      > + 4 adversarial verifications. **FACT** = verified against authoritative/primary
sources;
7      > **note-sourced** = consistent across research notes but not independently re-verified;
**INFERENCE**
8      > = my judgment for muneem's specific constraints.
9
10     ## Bottom line
11
12     **A single-user, local-first, no-paid-API tool cannot realistically use AA today ? and
AA's
13     structure makes it a poor fit even where reachable.** The blocker isn't technical; it's
a hard
14     **regulatory gate**: only an entity *registered with and regulated by*
RBI/SEBI/IRDAI/PFRDA can be a
15     Financial Information User (FIU) and receive data. There is **no individual / hobbyist /
unregulated-
16     app path**. Every workaround injects a **paid, cloud-hosted third party that decrypts
your entire
17     financial feed on its servers**. **PDF-first stays correct; AA is a deferred, opt-in
complement ?
18     never AA-primary.**
19
20     ## 1. Viability ? the access reality (FACT)
21
22     - **FIU = regulated entity, full stop.** RBI's Master Direction; Sahamati's FAQ answers
the exact
23     hobbyist question ? a great consumer use-case still **cannot** be an FIU if
unregulated. An
24     individual cannot register.
25     - **No self-serve / personal-use exemption** in the framework, DEPA, or the roadmap as
of mid-2026.
26     - **Sandbox ? production.** Finvu/Setu/Anumati sandboxes serve **mock FIP data only**;  
real accounts
27     require proving you are/will be a regulated FIU.
28     - **Production is paid, per-fetch, no free tier.** ~?1??25 per fetch via B2B contracts
(Setu, Finvu,
29     Anumati, INK); the *end-user* pays nothing but the *FIU* always pays ? and muneem
would be standing
30     in the FIU's shoes. RBI mandates a published pricing policy, **not** a free tier.
31     - **The only two real paths:** (a) become an **NBFC-AA-FIU** ? *note-sourced:* ?2 cr
net-owned funds,
32     fit-and-proper governance, biennial CISA audits, 12?18 mo ? facially disqualifying for
a personal
33     tool; (b) **ride under a licensed FIU via a TSP** (Technology Service Provider needs
no license but
34     operates under a regulated FIU that legally "consumes" the data). There is **no**
arrangement where
35     muneem-as-software is the consenting party pulling its owner's own data outside a
```

regulated FIU.

36

37 ## 2. Fidelity ? concrete delta vs PDF

38

39 **Where AA genuinely wins (FACT, ReBIT schema):** typed schema-locked transactions
 (`txnId`, `type`,
 40 `mode`, `amount`, `currentBalance`, `valueDate`, `narration`, ?) ? **no OCR, no
 coordinate
 41 clustering, no format-drift** (kills muneem's single biggest fragility);
 `currentBalance` per row
 42 (removes the fragile opening-balance + running-walk reconstruction); **fresher /
 incremental** pulls
 43 (~10?15 s) vs waiting for a monthly emailed PDF; **demat equity/MF/ETF holdings** via
 **CDSL (live
 44 Apr 2023) + NSDL** (FACT, verify-3).
 45

46 **Where PDF still wins / AA's investment story is overstated:**
 47 - **Credit cards: NOT live on AA** (proposed) ? muneem's CC stays PDF. *(note-sourced)*
 48 - **Bonds / G-secs / PPF / EPF: NOT live** (proposed) ? squarely in muneem's scope ?
 PDF-only. *(note-sourced)*
 49 - **Brokers (Zerodha/Groww/Kuvera) are NOT direct FIPs** ? holdings surface only
 indirectly via
 50 depositories, losing broker detail (order IDs, brokerage/charges, corporate-action
 specifics). The
 51 richer artifact is still the broker PDF/CAS. **(FACT, verify-3)**
 52 - **RTA (CAMS/KFintech) direct-FIP status is UNCONFIRMED/overstated** by primary
 sources; **MFCentral
 53 was taken offline Sept 2025** ? non-demat MF holdings via AA are *not* cleanly
 established. **(FACT, verify-3)**
 54 - **History capped ~2 years** *(note-sourced)* vs archived PDFs going back arbitrarily
 far.
 55 - **No categorization / merchant canonicalization** ? AA `narration` is one bank-
 dependent string;
 56 muneem's enrichment/ML layer is **still required either way**.
 57 - **Coverage/reliability:** *note-sourced* ~38% of accounts AA-enabled (~62% still need
 PDFs);
 58 joint / NRE-NRO generally unsupported; real deployments report high partial-pull
 failure rates.
 59 PDF covers ~100% of issuers the user actually receives mail from, **offline**.
 60

61 **Net:** AA is materially higher-fidelity **for live bank-deposit rows + demat
 equity/MF** ? but it
 62 **cannot cover muneem's full scope today**, and even where it wins it does **not**
 remove muneem's
 63 reconciliation/enrichment layer. A fidelity boost on a *subset*, not a replacement.
 64

65 ## 3. Security / privacy ? does AA add "deeper risk"? (the honest answer)

66

67 **AA genuinely gives (FACT):** end-to-end encryption with a **data-blind AA** ? ECDH
 (Curve25519) ?
 68 AES-256-GCM; the AA routes **ciphertext** and cannot read/store/decrypt it (verify-2 P1,
 CONFIRMED).
 69 No credential sharing (vs screen-scraping). Granular, revocable, audited consent. In-
 India storage +
 70 regulated audits.
 71

72 **The decisive caveat (FACT, verify-2):** **"self-custody" does not exist in AA ? it is
 custodial by
 73 design.** On-device / local-only decryption with a user-held key is **NOT a documented
 or available
 74 mode** (verify-2 Part 4: *no primary-source evidence found*). In real deployments,
 **decryption
 75 happens on the FIU/TSP backend** ? **"a TSP's first job is to decrypt data." Vendor
 "local" language
 76 means the **FIU's servers**, not your machine. Once data leaves the AA ecosystem, **no

technical

77 safeguard prevents FIU misuse** (Dvara Research, note-sourced) ? the FIU's security posture becomes

78 the single point of control.

79

80 **Direct verdict on the user's fear:**

81 - **In transit / credential exposure ? AA is clearly *safer*.* Here the fear is *misplaced*.

82 - **For muneem's actual threat model** (one local honeypot, no third party, offline, SQLCipher at

83 rest) ? **adopting AA the normal way is a *deeper* risk, and the fear is *justified*** ? the only

84 reachable integration (TSP-under-FIU) **inserts a paid cloud tier that decrypts your whole feed on

85 its servers**, plus consent/metadata logs and a new token/key surface. PDF-via-email keeps **zero

86 third parties in the path and all plaintext on one machine you control**.

87 - **Honest nuance:** AA *removes* email-interception and PDF-password-in-email weaknesses. So it's not

88 "AA worse on every axis" ? it's ***"AA trades a local-only surface for an institutional/cloud

89 surface."** For a privacy-first solo tool, that's a **net regression on the dimensions muneem

90 optimizes for** ? which is exactly why the instinct to be wary is correct. *(INFERENCE, grounded.)*

91

92 ## 4. Hybrid / local-preserving options (ranked)

93

94 Hard ceiling (verify-2): **truly on-device, self-custody-key decryption is not an available AA mode.**

95 Within that:

96

97 1. **PDF-default + opt-in AA via a regulated FIU's *export*, imported locally.** The user pulls their

98 data through an existing licensed FIU app and **imports the resulting statement/JSON into muneem**,

99 which reconciles it via the oracle. **muneem stays unlicensed, unpaid, local.** Cost: data passes

100 through a third-party FIU first (not minimal-trust), manual-export friction. *The only option that

101 keeps muneem itself unlicensed/unpaid/local ? #1.*

102 2. **TSP "transport-only, decrypt-locally"*** (the ideal you're reaching for). *No primary-source

103 evidence any TSP offers local-key custody* ? standard is cloud-side decryption. Theoretically

104 viable, non-standard, bespoke, still paid, still under a regulated FIU. **Watch, don't build.**

105 3. **muneem becomes its own NBFC-AA-FIU** (true self-custody) ? ?2cr NOF + audits + 12?18 mo,

106 FIU-purpose-only. **Disproportionate; effectively a non-option.**

107 4. **Skip AA ? use SMS/email transaction-alert parsing.** RBI mandates per-transaction bank SMS;

108 mature OSS parsers exist; **zero third party, fully local, near-real-time.** Limits: no history

109 backfill, no balances/holdings, brittle formats, no investments. **Not AA ? but the genuinely local

110 "fresher data" lever, and a better near-term complement to PDF than anything AA offers here.**

111

112 Guardrails for any AA path muneem ever adopts: **opt-in per account; provenance-tag every row

113 (PDF/AA/SMS); decrypt on-device if/when a TSP ever supports it; treat AA as a hint the reconciliation

114 oracle validates ? never as unverifiable ground truth.**

115

```

116    ## 5. Recommendation ? decisive & phased
117
118    **PDF-first core, AA as a deferred opt-in complement, never AA-primary.** Not a close
call: AA-primary
119    is *impossible* today (regulatory gate) and *undesirable* (mandatory paid cloud tier
that decrypts
120    everything) even if it were possible.
121
122    - **Phase 1 (now) ? do not touch AA.** Keep building the PDF pipeline + reconciliation
oracle + local
123    L3 vision fallback. **Optionally add local SMS/email-alert parsing** as a zero-trust
freshness
124    signal (advances "less manual / fresher" without violating a single principle, and
echoes the
125    predecessor's email-mining DNA).
126    - **Phase 2 ? opt-in AA *complement* (Rank 1), only when a trigger fires.** An "Import
AA data" path
127    via a regulated FIU export, into the same schema, **provenance-tagged, oracle-
validated, per-account
128    opt-in**, PDF remaining default + fallback. Use where AA is strongest: live bank-
deposit rows +
129    demat equity/MF.
130    - **Phase 3 ? re-evaluate TSP transport-only (Rank 2)** *only if* the market offers true
131    local-key/on-device decryption at hobbyist cost. **Never** pursue own-FIU.
132
133    **Triggers that move AA from "defer" ? "adopt as complement":**
134    1. **CC and/or bonds go live as AA FI-types** (closes the scope gaps that force PDF
anyway).
135    2. **A TSP/AA documents true on-device, self-custody-key decryption** ? verify-2 P4
flips from "not
136    confirmed" to "available." *The single most important gate for muneem's privacy
model.*
137    3. **A genuinely free / flat near-zero personal production tier appears** (today: none).
138    4. **muneem's PDF coverage saturates** (HDFC/Axis/AU hardened, broker/MF parsers
shipped) so AA is
139    pure upside, not a substitute for unfinished work.
140    5. **Coverage/reliability matures** past ~38% and the joint-/NRE-/partial-pull gaps for
the user's
141    specific institutions.
142
143    ## Sources (anchors)
144
145    RBI Master Direction on NBFC-AA + **Sahamati FIU FAQ** (the regulatory gate). **ReBIT
NBFC-AA API
146    spec / Sahamati security standards** (E2E, data-blind crypto). **Moneymone data-in-
transit docs +
147    MediaNama / CAMSfinserv** (decryption is institutional/cloud, *not* self-custody).
Aggregator pricing
148    (Setu / Anumati / INK) (no free production tier). **CDSL/NSDL FIP pages + CASParser**
(live investment
149    data, and the RTA/MFCentral gap).
150

```

14. user (2026-06-10T16:14:08.885Z)

```

1    # muneem ? Testing Strategy
2
3    > Status: **active.** Companion to [`.ROADMAP.md`](./ROADMAP.md) (sequencing) and
[`.DESIGN.md`](./DESIGN.md) (architecture + threat model). This doc owns *how we know muneem
works* ? and, just as importantly, *how we know it isn't leaking or hallucinating*.
4    >
5    > Created 2026-05-30. Grounded in a direct inspection of the committed promo corpus
(`testdata/promo-emails/`), not in the abstract.
6
7    ---

```

```

8
9     ## 1. Principles
10
11     1. **Grounded, not aspirational.** Every test tier below maps to a failure mode actually
12     observed in the real corpus (§ 2), not a hypothetical one.
13     2. **Precision ranks with recall.** A bookkeeper that invents a promo code or a
14     transaction is worse than one that misses it. Negative fixtures (no code present) are first-
15     class.
16     3. **Test on safe data by default.** The committed promo corpus is non-sensitive and
17     drives CI. Sensitive paths (statements) are tested on **synthetic/redacted fixtures** in CI and
18     on **local-only, gitignored** real data behind an opt-in marker ? never real statements in CI or
19     the repo.
20     4. **CI is offline, deterministic, secretless.** No network, no cloud LLM, no OAuth, no
21     keychain in the default CI run. Non-deterministic steps (OCR, LLM) are mocked/cassette-backed by
22     default and exercised live only behind explicit markers.
23     5. **Security posture is enforced by tests, not just docs.** Redaction, category-gating
24     of cloud LLMs, and "no sensitive paths tracked in git" are assertions that fail the build (§ 9).
25     6. **Adding a corpus file means adding its answer-key entry.** The corpus and its ground
26     truth move together (§ 4).
27
28     ---
29
30     ## 2. What the corpus actually contains (grounded findings)
31
32     Direct inspection of all 10 PDFs ? PyMuPDF text-layer extraction (`get_text`) + large-
33     image count (`get_images`, ?300×300 px, matching the Email-Mining dedup filter) + visual
34     rendering of a sample. This is the empirical basis for everything below.
35
36     | PDF | Issuer (from content) | Pages | Text chars | Big imgs | Promo code | Code
37     location | Expiry / validity | Verified |
38     |---|---|---|---|---|---|---|---|
39     | `dataquest.pdf` | Dataquest | 3 | 2526 | 0 | `JS231sz` | **text layer** | "Ends Today"
40     + live countdown (relative to send date 2024-07-29) | ? text + visual |
41     | `makemytrip-2.pdf` | MakeMyTrip | 3 | 925 | 7 | `EARLYBIRDDEAL` | **text layer** |
42     "?15 days in advance" (conditional, not a date) | ? text |
43     | `swiggy.pdf` | Swiggy × ClearTax | 4 | 774 | 3 | `CTSWIGGYUX` | **image only** (OCR) |
44     "before July 31st" (in image) | ? visual |
45     | `flipkart.pdf` | Flipkart | 2 | 555 | 1 | ? none ? | n/a | "GOAT SALE, 20?25 Jul" (in
46     image) | ? negative |
47     | `airasia-2.pdf` | AirAsia | 2 | 695 | 1 | TBD | image? | "Flat 50% Off Seats, 25% Off
48     Meals" (subject/text) | ?? unverified |
49     | `airasia-3.pdf` | AirAsia | 2 | 710 | 1 | TBD | image? | "Final Day! Save 15% Off
50     Flights?" (relative) | ?? unverified |
51     | `airasia-1.pdf` | **Air India Express** (? filename) | 3 | 834 | 4 | TBD | image? |
52     "Special Offers" | ?? unverified |
53     | `makemytrip-1.pdf` | MakeMyTrip | 2 | 1051 | 2 | TBD | image? | "travel dates & save
54     up?" | ?? unverified |
55     | `unacademy.pdf` | Unacademy | 2 | 555 | 3 | TBD | image? | "Last few hours left
56     Ayush!" (relative, personalized) | ?? unverified |
57     | `tataneu.pdf` | Tata Neu | 4 | 1581 | 10 | TBD | image? | ? | ?? unverified |
58
59     **`Verified`** distinguishes ground truth we have *confirmed* (text layer and/or visual
60     read) from entries still **TBD**. We do **not** invent codes for the unverified rows ?
61     establishing them (via OCR + a human glance) is an explicit task in ROADMAP A.4. Attribution
62     discipline (CLAUDE.md): if it isn't verified, it's marked unverified.
63
64     ### Gotchas the real data exposes (each becomes a test)
65
66     - **Ligatures in the text layer.** PyMuPDF returns `0?er` / `0?` (U+FB00 `?`), not
67     `Offer`/`Off`. Naïve substring/regex matching for "Off"/"Offer" silently fails. ? **NFKC
68     normalization** before matching, with a dedicated unit test (`airasia-2`, `airasia-3`).
69     - **Codes live in images about half the time.** 6 of 10 PDFs carry the offer in
70     rasterized banners (`bigimgs` ? 1 and an empty/near-empty text layer). OCR is not optional for
71     the corpus; it is the path for ~half of it.
72     - **Expiry is rarely a clean date.** Observed forms: absolute-ish ("before July 31st"),

```

```

relative ("Ends Today", "Final Day!", "Last few hours left"), conditional ("?15 days in
advance"), and a **live countdown timer**. ? Expiry parsing must resolve relative phrases
against the email's *received date* and tolerate "no hard date."
44 - **Filename ? issuer.** `airasia-1.pdf` is an **Air India Express** mailer. ? Issuer
detection must derive from sender/content, never the filename. (Also: fix the mislabel in the
corpus README.)
45 - **Third-party PII in fixtures.** Recipient is `Ayush Raina <ayushraina767@gmail.com>`
(the prototype's tester, *not* the muneem user), and bodies are personalized ("?left Ayush!").
The corpus README says "no PII beyond the recipient's email address"; in reality the recipient
is a *different person* and appears in the body. Treated as **non-sensitive** (promo, already
committed, no financial data), but extraction tests must not surface or log it as financial PII,
and golden files (§ 5) intentionally retain it as committed non-sensitive content.
46
47 ---
48
49 ## 3. Fixture tiers (derived from § 2)
50
51 The corpus self-stratifies into capability tiers. Tests are gated by tier so CI stays
green and honest about what's proven.
52
53 | Tier | Capability under test | Members | CI behavior |
54 |---|---|---|---|
55 | **T1 ? text-layer codes** | Deterministic extraction, no OCR | `dataquest`,
`makemytrip-2` | **Must pass** in default CI (recall + precision = 100%). |
56 | **T2 ? image-only codes** | OCR (Tesseract) ? code recovery | `swiggy` (+ any
unverified row confirmed image-coded) | Behind `@pytest.mark.ocr`; runs in the optional OCR job,
skipped if the Tesseract binary is absent. |
57 | **T3 ? no-code negatives** | Precision / no hallucination | `flipkart` (+ confirmed
no-code rows) | **Must pass** in default CI (assert *zero* codes extracted). |
58 | **T4 ? unverified** | Answer key not yet established | the 6 ?? rows |
`@pytest.mark.xfail(strict=False)` until verified in A.4; never block CI on guesses. |
59
60 ---
61
62 ## 4. The answer-key file (ground truth)
63
64 A single committed file is the source of truth; tests read it, they don't hardcode
expectations.
65
66 `testdata/promo-emails/expected.yaml` ? one entry per PDF:
67
68 ```yaml
69 # Seed (created during ROADMAP A.4). verified:false rows are placeholders, NOT
assertions.
70 dataquest.pdf:
71   issuer: Dataquest
72   codes: ["JS231sz"]
73   code_location: text # text | image | none
74   discount: "$706 / lifetime"
75   expiry: {type: relative, raw: "Ends Today", resolved_against: received_date}
76   verified: true
77 makemytrip-2.pdf:
78   issuer: MakeMyTrip
79   codes: ["EARLYBIRDDEAL"]
80   code_location: text
81   discount: "up to 50% off"
82   expiry: {type: conditional, raw: "?15 days in advance"}
83   verified: true
84 swiggy.pdf:
85   issuer: Swiggy # offer brand: ClearTax
86   codes: ["CTSWIGGYUX"]
87   code_location: image
88   discount: "up to 70% off"
89   expiry: {type: absolute, raw: "before July 31st"}
90   verified: true

```

```

91     flipkart.pdf:
92         issuer: Flipkart
93         codes: []          # negative fixture ? must extract nothing
94         code_location: none
95         discount: "GOAT Sale"
96         expiry: {type: relative, raw: "20-25 Jul"}
97         verified: true
98         # airasia-1/2/3, makemytrip-1, unacademy, tataneu: verified:false ? establish in A.4
(OCR + manual glance).
99     ``
100
101     **Process to verify a TBD row (A.4):** run OCR on its large images ? propose codes ? a
human confirms/edits ? flip `verified:true`. Until then it stays T4/xfail. New corpus PDF ? new
entry, same flow.
102
103     ---
104
105     ## 5. Test layers (the pyramid)
106
107     **Unit (fast, deterministic, the bulk):**
108     - PDF text extraction wrapper (pages ? text), large-image extraction + size filter.
109     - **Image dedup**: MD5-of-PNG-bytes equality and SSIM ? 0.95 near-duplicate detection
(port + test Email-Mining's logic on synthetic image pairs).
110     - **Text normalization**: NFKC ligature folding (`?'`ff`), whitespace, case ? table-
driven, seeded by § 2 gotchas.
111     - **Code detection**: regex/heuristics over normalized text; positive cases (`JS231sz`,
`EARLYBIRDDEAL`) and adversarial negatives (order numbers, prices like `$706`, dates, the
literal word "code" with no token) so precision is pinned.
112     - **Expiry parsing**: absolute / relative / conditional / none, each resolved against a
fixed `received_date`.
113     - **Schema + migrations**: apply v1?vN on a temp SQLite file; assert idempotent re-
apply.
114
115     **Component / integration:**
116     - `ingest(pdf)` end-to-end on a single corpus PDF ? assert exact `documents` + `offers`
rows (via the answer key), against an in-memory/temp SQLite.
117
118     **Regression (corpus):** run the pipeline over `testdata/promo-emails/`, assert per-tier
recall/precision from `expected.yaml`. T1+T3 in default CI; T2 in the OCR job; T4 xfail.
119
120     **Golden-file (drift detection):** snapshot the unified extracted text per PDF under
`testdata/promo-emails/golden/`. A diff flags upstream library/behavior drift. (Non-sensitive
promo text only ? safe to commit.)
121
122     **CLI:** drive `muneem ingest` / `muneem offers list` via subprocess (or the framework's
test runner); assert exit codes + stdout shape, not internals.
123
124     **Determinism / idempotency:** ingest the same PDF twice ? exactly one `documents` row
(sha256 dedup), stable output.
125
126     ---
127
128     ## 6. Testing the non-deterministic parts (OCR & LLM)
129
130     - **OCR (Tesseract, local):** Compare against golden text with a **normalized fuzzy
match** (e.g. token-set ratio ? threshold), never exact equality. Marker `@pytest.mark.ocr`;
auto-skip if the binary is missing so contributors without Tesseract still get a green default
suite.
131     - **LLM extraction (promo only ? cloud is policy-permitted for this *non-sensitive*
category):**
132         - Default CI uses **recorded cassettes** (VCR-style) ? deterministic, offline, free.
133         - Live calls only behind `@pytest.mark.llm` + an env flag (`MUNEEM_LLM_LIVE=1`); never
in default CI.
134         - The cassette/fixtures must contain **only** promo-corpus content ? enforced by the
guard in § 9.

```

```

135
136 ---
137
138 ## 7. The sensitive path (statements) ? coverage without real data in CI
139
140 The corpus is promo-only by design. Statement parsing (ROADMAP Milestone C) is tested
two ways:
141
142 1. Synthetic fixtures, committed. A small generator emits fake HDFC-layout
statement PDFs with faker-generated names/amounts/dates and a known derived password. These
exercise the parser and the unlocker in CI with zero real data. Stored under
`testdata/synthetic/` (committed; clearly fake).
143 2. Real statements, local-only. Behind `@pytest.mark.local_statements`, reading
`testdata/statements/` (gitignored). Skipped in CI; run by the user locally.
144
145 Unlocker tests (hybrid strategy, C.1): encrypt a synthetic fixture with a password
derived by a known rule, then assert the resolver tries dictionary ? rule ? prompt in order,
caches the winner, and ? critically ? that no password material appears in logs or
exceptions (log-capture assertion).
146
147 Parser tests (C.2): synthetic HDFC statement ? assert normalized `transactions`
(date, amount, direction, balance) match the generator's known ledger.
148
149 ---
150
151 ## 8. CI shape (GitHub Actions)
152
153 - Default job (`push` / `pr`): `setup-python@3.13` ? install ? `ruff check` ?
`pytest -m "not ocr and not llm and not local_statements"`. Offline, no secrets. Dependency
cache on.
154 - OCR job (manual / scheduled): installs Tesseract, runs `pytest -m ocr`.
155 - Guard job (always): the security assertions in § 9.
156 - Coverage reported but thresholds start modest and ratchet up; we don't gate on a
vanity number before the code exists.
157
158 ---
159
160 ## 9. Security assertions as tests (muneem-specific, must-pass)
161
162 These encode the DESIGN § 5 standing rules as failing-build checks:
163
164 - No sensitive paths tracked. A test runs `git ls-files` and fails if anything under
`testdata/statements/`, `testdata/private/`, `*.db`, `*.sqlite`, `token*`, `.env`, or
`password_rules.*` is tracked.
165 - Cloud-LLM category gate. Calling the cloud-LLM client with a category in {bank,
cc, broker, mf, insurance} raises/refuses; only {promo, ?explicitly-approved} are allowed. A
test asserts the refusal.
166 - Log redaction. Feed synthetic account numbers / balances / a known password
through the logging path; assert they are masked in captured output.
167 - Token storage. Token persistence goes through the `keyring` abstraction (mocked in
tests); a test asserts nothing is written to a plaintext file on disk.
168
169 ---
170
171 ## 10. Action items surfaced by this grounding
172
173 Small, real follow-ups discovered while building the answer key (tracked, not silently
fixed):
174
175 1. Establish the 6 unverified answer-key rows (A.4) ? OCR + manual confirmation.
176 2. Fix the corpus README mislabel: `airasia-1.pdf` is Air India Express, and the
"no PII beyond recipient email" note understates that the recipient (`Ayush Raina`) is a third
party present in message bodies.
177 3. Ligature normalization is a hard requirement, not a nicety ? wire NFKC into the
extractor before code matching.

```



```

178
179 ---
180
181 ## 11. Definition of done (testing) for Track A
182
183 `expected.yaml` committed with every verified row; **T1 recall + precision = 100%** in
default CI; **T3 negatives extract nothing**; **T2 passes locally with OCR**; idempotency + the
§ 9 security guards green. That is the bar for calling the promo "forgotten deals" feature
trustworthy.
184

```

15. user (2026-06-10T16:14:09.321Z)

```

1 # muneem ? Dependency policy & license allowlist
2
3 muneem must be **safe to use commercially** and must **minimize paid dependencies**.
4 These are standing principles (DESIGN.md § 5, rules 8?9). This doc is the allowlist;
5 `tests/test_dependency_licenses.py` enforces it (a dependency not listed here fails CI).
6
7 ## Rules
8
9 1. Only permissive, commercial-safe licenses: **MIT, BSD (2/3-clause), Apache-2.0, ISC,
HPND**.
10
11 2. **Banned:** GPL, LGPL, **AGPL**, and source-available / non-commercial licenses.
12 ? **PyMuPDF / `fitz` is AGPL-3.0 ? do not use it.** (Its own metadata reads
13 **"Dual Licensed - GNU AFFERO GPL 3.0 or Artifex Commercial"; the non-AGPL option
14 is a *paid* license, which also violates rule 4.)
15
16 3. **MPL-2.0** (weak, file-level copyleft) only as a fallback when no permissive option
17 exists, and flagged here.
18
19 4. Prefer **local / self-hosted / open-source** runtimes over paid APIs. Paid APIs
20 (cloud LLMs, paid OCR/vision) are a **fallback only**, gated behind explicit
21 approval.
22
23 We are willing to **train our own** image/OCR models rather than take a paid
24 dependency.
25
26 5. Adding a dependency requires updating **both** this table **and** the allowlist in
27 `tests/test_dependency_licenses.py` ? otherwise the build fails.
28
29 ## Current dependencies
30
31 | Package | License | Role |
32 |---|---|---|
33 | typer | MIT | CLI |
34 | platformdirs | MIT* | OS config/data dirs |
35 | keyring | MIT | OS keychain (secrets) |
36 | pdfminer.six | MIT | PDF text extraction (replaces AGPL PyMuPDF) |
37 | pypdfium2 | Apache-2.0 / BSD-3 | PDF page rendering for OCR |
38 | pyesseract | Apache-2.0 | local OCR (Tesseract) wrapper |
39 | Pillow | HPND | image handling for OCR |
40 | pypdf | BSD-3-Clause | decrypt password-protected statements (Stage 2) |
41 | google-api-python-client | Apache-2.0 | Gmail API (read-only) |
42 | google-auth | Apache-2.0 | OAuth credentials |
43 | google-auth-oauthlib | Apache-2.0 | installed-app OAuth consent flow |
44 | msal | MIT | Microsoft Graph API OAuth |
45 | requests | Apache-2.0 | HTTP library for Outlook provider |
46 | pdfplumber | MIT | text/table + coordinate extraction for statements (HDFC/Axis/AU) |
47 | sqlalchemy | Zlib | at-rest DB encryption (SQLCipher, transparent AES-256) ? see below
48 |
49 | cryptography | Apache-2.0 / BSD-3 | Argon2id + AES-GCM for the passphrase-wrapped DB-
key recovery file (PyCA) |
50 | pytest *(dev)* | MIT | tests |
51 | pytest-cov *(dev)* | MIT | coverage measurement + the >80% CI gate (coverage.py is
Apache-2.0) |
52 | ruff *(dev)* | MIT | lint / format |
53 | mypy *(dev)* | MIT | static type checking (CI + pre-push gate) |
54 | PyYAML *(dev)* | MIT* | read the test answer key |

```

```

47
48 \* PyMuPDF (AGPL), PyYAML (MIT) and platformdirs (MIT) were confirmed directly from
49 installed package metadata; the rest are MIT per their upstream projects.
50
51 ## System tools (not Python packages)
52
53 - **Tesseract OCR** (Apache-2.0) ? local OCR engine, invoked via `pytesseract`. Install
54 separately (e.g. `winget install UB-Mannheim.TesseractOCR`). muneem finds it on PATH,
55 at the Windows default location, or via the `MUNEEM_TESSERACT_CMD` env var.
56
57 ## Current / recently added (permissive)
58
59 - **sqlcipher3** (Zlib) ? at-rest encryption of the local ledger (SQLCipher, transparent
60 AES-256). Its prebuilt wheels **bundle** native code whose licenses are all
permissive:
61 **SQLCipher core = BSD-3-Clause, SQLite amalgamation = Public Domain, OpenSSL =
Apache-2.0**.
62 No copyleft anywhere in the chain. The `test_dependency_licenses.py` allowlist records
the
63 binding as `Zlib` (the upstream `LICENSE`; PyPI metadata mislabels it `MIT` ? both
permissive).
64 Documented fallback (not adopted): `apsw-sqlite3mc` (APSW = permissive, SQLite3MC =
MIT, no
65 OpenSSL) ? switch only to drop the OpenSSL dep, at the cost of an APSW-API refactor.
See
66 [`.encryption-at-rest.md`](./encryption-at-rest.md).
67 - **pypdf** (BSD-3) ? PDF decryption (Stage 2, used by unlock.py and parsers). Chosen
over
68 pikepdf (MPL-2.0 flagged fallback) for license simplicity.
69 - **scikit-image / numpy** (BSD) ? planned for image dedup (SSIM) in future; not yet in
70 pyproject (Pillow covers basic image needs for OCR today).
71
72 ## Planned (future, permissive, no paid)
73
74 - A **purpose-trained OCR model** for stylized promo codes (Tesseract is approximate on
75 some banners like Swiggy). Aligns with "train our own" (DESIGN §5 rule 9).
76
77 **Optional deps** (outlook extra): msal + requests ? only loaded when OutlookProvider is
78 used; guarded by ImportError in code.
79

```

16. user (2026-06-10T16:14:14.018Z)

```

1 # testdata/promo-emails
2
3 A small regression corpus of **promotional email PDFs** carried over from the
predecessor project `~/Projects/Email-Mining`. These are non-sensitive marketing emails (no
account numbers, no balances, no PII beyond the recipient's email address), safe to keep in the
repo.
4
5 ## Why these are here
6
7 These are real-world Indian-merchant promo emails saved as PDFs. Each contains some
combination of:
8 - promo codes (the original extraction target)
9 - expiry dates
10 - terms & conditions
11 - banner imagery with text rasterized into the image (the reason image-OCR was needed in
the predecessor)
12
13 They serve as the **first regression corpus** for muneem's "forgotten deals" extraction
target ? the use-case carried forward from Email-Mining. A passing test on this corpus should
recover the known promo codes from each PDF.
14
15 ## Contents

```

```

16
17 | Filename | Issuer | Original category | Notes |
18 |---|---|---|---|
19 | `airasia-1.pdf` | AirAsia | Category2 | 3.7 MB ? large image content |
20 | `airasia-2.pdf` | AirAsia | Category2 | |
21 | `airasia-3.pdf` | AirAsia | Category2 | |
22 | `dataquest.pdf` | DataQuest | Category1 | Smallest (57 KB) ? likely mostly text |
23 | `flipkart.pdf` | Flipkart | Category2 | |
24 | `makemytrip-1.pdf` | MakeMyTrip | Category2 | |
25 | `makemytrip-2.pdf` | MakeMyTrip | Category1 | 3.8 MB ? heavy banner imagery |
26 | `swiggy.pdf` | Swiggy | Category2 | |
27 | `tataneu.pdf` | TataNeu | TestSet | 13.9 MB ? largest; was held out in predecessor |
28 | `unacademy.pdf` | Unacademy | Category2 | |
29
30 The original `Category1` / `Category2` / `TestSet` split in Email-Mining was not
explicitly documented but appears to have been a rough "simple / complex / held-out" partition.
Preserved here in the table above in case anyone wants to re-establish a train/test split.
31
32 ## Provenance
33
34 Carried forward verbatim from `Projects/Email-
Mining/textExtract/pdfFiles/{Category1,Category2,TestSet}/`. Only the filenames were normalized
(lowercased, kebab-cased).
35
36 ## What is *not* in this directory
37
38 - **No bank statements.** Real statements are sensitive and must never enter the repo.
See `.gitignore` and `CLAUDE.md` § Standing rules.
39 - **No credit-card bills, brokerage statements, mutual-fund reports, or utility bills.**
Same reason.
40 - The intended location for those (locally, gitignored) is `testdata/statements/` ? a
directory that should exist on the user's machine but never be committed.
41

```

17. user (2026-06-10T16:14:16.114Z)

C:/Users/avidu/Projects/muneem/docs:

```

CURRENT_STATE.md
DEPENDENCIES.md
ROADMAP.md
TESTING.md
encryption-at-rest.md
feedbackJune2026.md
ingesting-statements.md
onboarding-a-bank.md
parser-architecture.md
prior-art-email-mining.md
research-open-banking-india.md
schema.md

```

C:/Users/avidu/Projects/muneem/src/muneem:

```

__init__.py
__pycache__
cli.py
config.py
db.py
db_key.py
errors.py
extract.py
ingestion
log.py
migrations.py
ocr.py
offers.py
parsers

```

```
password_rules.py
recovery.py
redaction.py
secrets.py
statement_unlock.py
unlock.py
```

```
C:/Users/avidu/Projects/muneem/src/muneem/ingestion:
__init__.py
__pycache__
base.py
gmail.py
imap.py
outlook.py
```

```
C:/Users/avidu/Projects/muneem/src/muneem/parsers:
__init__.py
__pycache__
au.py
axis.py
base.py
clustering.py
concepts.py
engine.py
hdfc.py
profiles.py
savings.py
validation.py
```

```
C:/Users/avidu/Projects/muneem/testdata:
aubank-statements
axis-statements
credentials
hdfc-statements
promo-emails
snapshots
```

```
C:/Users/avidu/Projects/muneem/tools:
gmail_auth.py
imap_auth.py
outlook_auth.py
```

18. user (2026-06-10T16:14:22.652Z)

```
1      """The ``muneem`` command-line interface.
2
3      Track A ships the promo spine: ``ingest`` a PDF into the local store and
4      ``offers list`` to read back what was found. ``--ocr`` adds local image OCR for
5      codes rasterized into banners (approximate; flagged for verification). Pipeline
6      subcommands (``fetch``, ``txns``) land with their milestones.
7      """
8
9      from __future__ import annotations
10
11     from contextlib import contextmanager
12     from pathlib import Path
13
14     import typer
15
16     from muneem import __version__
17     from muneem import db as dbmod
18     from muneem.config import load_config
19     from muneem.log import get_logger
20     from muneem.redaction import redact_filename
21
```

```

22     _log = get_logger(__name__)
23
24     app = typer.Typer(
25         add_completion=False,
26         help="muneem ? a local-first personal-finance bookkeeper.",
27         no_args_is_help=True,
28     )
29     offers_app = typer.Typer(no_args_is_help=True, help="Work with extracted offers.")
30     app.add_typer(offers_app, name="offers")
31
32     txns_app = typer.Typer(no_args_is_help=True, help="Work with parsed transactions.")
33     app.add_typer(txns_app, name="txns")
34
35     db_app = typer.Typer(no_args_is_help=True, help="Manage the local encrypted ledger.")
36     app.add_typer(db_app, name="db")
37
38
39     @contextmanager
40     def _ledger(db_path: Path):
41         """Open the encrypted ledger, turning an un-migrated *plaintext* file into a clean
42         error (pointing at ``muneem db encrypt``) rather than a raw traceback."""
43         try:
44             with dbmod.connect_encrypted(db_path) as conn:
45                 yield conn
46         except dbmod.EncryptedDatabaseError as exc:
47             typer.echo(str(exc), err=True)
48             raise typer.Exit(1) from exc
49
50
51     # Dispatch table for supported banks (extensible as new parsers land).
52     BANK_DISPATCH = {
53         "axis": (dbmod.list_axis_transactions, "debit", "credit"),
54         "axis_cc": (dbmod.list_axis_cc_transactions, "amount", "type"),
55         "au": (dbmod.list_au_transactions, "debit", "credit"),
56         "hdfc": (dbmod.list_hdfc_transactions, "withdrawal", "deposit"),
57     }
58
59
60     def _version_callback(value: bool) -> None:
61         if value:
62             typer.echo(f"muneem {__version__}")
63             raise typer.Exit()
64
65
66     @app.callback()
67     def main(
68         version: bool = typer.Option(
69             False,
70             "--version",
71             callback=_version_callback,
72             is_eager=True,
73             help="Show the muneem version and exit.",
74         ),
75     ) -> None:
76         """muneem command-line interface."""
77
78
79     @app.command()
80     def info() -> None:
81         """Show resolved local paths (no secrets, no sensitive data)."""
82         cfg = load_config()
83         present = "present" if cfg.config_file.is_file() else "absent"
84         typer.echo(f"muneem {__version__}")
85         typer.echo(f"config dir : {cfg.config_dir}")

```

```

86     typer.echo(f"data dir    : {cfg.data_dir}")
87     typer.echo(f"db path    : {cfg.db_path}")
88     typer.echo(f"staging    : {cfg.staging_dir}")
89     typer.echo(f"config file: {cfg.config_file} ({present})")
90
91
92     def _ingest_bank_statement(pdf: Path, db_path: Path, doc_type: str, *, interactive:
bool) -> None:
93         """Unlock (if encrypted) ? extract ? select parser ? store reconciled transactions.
94
95         The hybrid password unlock (cached/configured candidates ? prompt) runs *before*
96         extraction,
97         since the text needed to pick a parser is itself encrypted. The resolved password is
98         forwarded
99         to extraction and the parser, used in-memory only ? never logged.
100         """
101         from muneem import db as dbmod
102         from muneem.extract import extract
103         from muneem.parsers import get_parser
104         from muneem.statement_unlock import resolve_password
105         from muneem.unlock import is_encrypted
106
107         password: str | None = ""
108         if is_encrypted(pdf):
109             password = resolve_password(pdf, interactive=interactive)
110             if password is None:
111                 hint = " (run without --no-prompt to enter it)" if not interactive else ""
112                 typer.echo(f"Could not unlock {pdf.name}: no working password{hint}.",
err=True)
113                 raise typer.Exit(1)
114
115         ex = extract(pdf, password=password or "") # resolved above; `or ""` keeps the type
116         str
117         parser = get_parser(ex.text)
118         if parser is None:
119             typer.echo(f"No parser recognizes {pdf.name} as a supported bank statement.",
err=True)
120             raise typer.Exit(1)
121
122         with _ledger(db_path) as conn:
123             dbmod.init_db(conn)
124             existing = dbmod.get_document_by_sha(conn, ex.sha256)
125             if existing is not None:
126                 typer.echo(f"already ingested (doc {existing['id']}): {pdf.name}")
127                 return
128             doc_id = dbmod.insert_document(
129                 conn,
130                 source="file",
131                 external_id=pdf.name,
132                 doc_type=doc_type,
133                 sha256=ex.sha256,
134                 path=str(pdf.resolve()),
135                 status="parsed",
136             )
137             count = parser.parse_and_insert(pdf, conn, doc_id, password=password)
138             doc = dbmod.get_document_by_sha(conn, ex.sha256)
139             status = doc["status"] if doc else "parsed"
140
141         typer.echo(
142             f"ingested {pdf.name} (doc {doc_id}); parsed {count} transaction(s) "
143             f"via {parser.__class__.__name__} [{status}]"
144         )
145
146     def _eval_statement(pdf: Path, *, interactive: bool) -> tuple[str, int, str | None]:

```

```

145         """Run one statement through the full parse pipeline into an in-memory DB.
146
147     Returns `` (outcome, row_count, parser_name)`` where ``outcome`` is the document
status
148     (``reconciled`` / ``unreconciled``) or a failure tag (``locked`` / ``no-parser`` /
``error``).
149     Nothing is persisted and no statement value is surfaced ? only the verdict and
counts.
150     """
151     from muneem import db as dbmod
152     from muneem.extract import extract
153     from muneem.parsers import get_parser
154     from muneem.statement_unlock import resolve_password
155     from muneem.unlock import is_encrypted
156
157     password = ""
158     if is_encrypted(pdf):
159         password = resolve_password(pdf, interactive=interactive) or ""
160         if not password:
161             return ("locked", 0, None)
162     try:
163         ex = extract(pdf, password=password)
164         parser = get_parser(ex.text)
165     except Exception as exc: # eval must not crash on one bad file; record the redacted
reason
166         _log.warning(
167             "eval: extract/route failed for %s (%s)", redact_filename(pdf),
type(exc).__name__
168         )
169         return ("error", 0, None)
170     if parser is None:
171         return ("no-parser", 0, None)
172     name = parser.__class__.__name__
173     try:
174         with dbmod.connect(":memory:") as conn:
175             dbmod.init_db(conn)
176             doc_id = dbmod.insert_document(
177                 conn, source="file", external_id=pdf.name, sha256=ex.sha256,
status="parsed"
178             )
179             count = parser.parse_and_insert(pdf, conn, doc_id, password=password)
180             doc = dbmod.get_document_by_sha(conn, ex.sha256)
181             status = doc["status"] if doc else "parsed"
182     except Exception as exc:
183         _log.warning(
184             "eval: parse/store failed for %s (%s)", redact_filename(pdf),
type(exc).__name__
185         )
186         return ("error", 0, name)
187     return (status, count, name)
188
189
190 @app.command("eval")
191 def eval_statements(
192     folder: Path = typer.Argument(
193         ...,
194         exists=True,
195         file_okay=False,
196         readable=True,
197         help="A LOCAL folder of statement PDFs to evaluate (e.g. your seed or golden
set).",
198     ),
199     prompt: bool = typer.Option(
200         False,
201         "--prompt/--no-prompt",

```

```

202             help="Prompt for a password when cached/configured candidates don't unlock a
file.",
203         ),
204     ) -> None:
205         """Evaluate the parsers against a folder of statements and report the reconcile
rate.
206
207         The seed?golden onboarding loop (see docs/onboarding-a-bank.md): point it at a few
*seed*
208         statements while tuning a profile, then at a *golden* set to confirm. Everything
runs locally in
209         an in-memory DB; only counts / statuses / parser names print ? never statement
values.
210         """
211         pdfs = sorted(folder.glob("*.pdf"))
212         if not pdfs:
213             typer.echo(f"no .pdf files in {folder}")
214             return
215         reconciled = 0
216         for pdf in pdfs:
217             outcome, count, name = _eval_statement(pdf, interactive=prompt)
218             reconciled += outcome == "reconciled"
219             typer.echo(f" {outcome:12} {count:4d} row(s) {name or '?':20} {pdf.name}")
220         typer.echo(f"\n{reconciled}/{len(pdfs)} reconciled")
221
222
223     @app.command()
224     def ingest(
225         pdf: Path = typer.Argument(
226             ..., exists=True, dir_okay=False, readable=True, help="Path to a PDF to ingest."
227         ),
228         doc_type: str = typer.Option("promo", help="Document category."),
229         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
230         ocr: bool = typer.Option(
231             False, "--ocr", help="Also OCR images locally (Tesseract) for codes in banners."
232         ),
233         no_prompt: bool = typer.Option(
234             False,
235             "--no-prompt",
236             help="Never prompt for a statement password; use only cached/configured
candidates.",
237         ),
238     ) -> None:
239         """Store a PDF's offers (promo) or transactions (bank_statement) in the local DB."""
240         from muneem.extract import extract
241         from muneem.offers import extract_offer
242
243         db_path = db or load_config().db_path
244
245         if doc_type == "bank_statement":
246             _ingest_bank_statement(pdf, db_path, doc_type, interactive=not no_prompt)
247             return
248
249         from muneem import db as dbmod
250
251         ex = extract(pdf)
252         text_fields = extract_offer(ex.text)
253         # (code, source) pairs; text-layer codes are exact, OCR codes are approximate.
254         found: list[tuple[str, str]] = [(c, "text") for c in text_fields.codes]
255         expiry_raw, expiry_type = text_fields.expiry_raw, text_fields.expiry_type
256
257         if ocr:
258             from muneem.ocr import ocr_pdf, tesseract_available
259

```



```

260         if not tesseract_available():
261             typer.echo("warning: --ocr requested but Tesseract not found; skipping
OCR.", err=True)
262         else:
263             ocr_fields = extract_offer(ocr_pdf(pdf))
264             known = {c for c, _ in found}
265             for code in ocr_fields.codes:
266                 if code not in known:
267                     found.append((code, "ocr"))
268                     known.add(code)
269             if expiry_raw is None:
270                 expiry_raw, expiry_type = ocr_fields.expiry_raw, ocr_fields.expiry_type
271
272     with _ledger(db_path) as conn:
273         dbmod.init_db(conn)
274         existing = dbmod.get_document_by_sha(conn, ex.sha256)
275         if existing is not None:
276             typer.echo(f"already ingested (doc {existing['id']}): {pdf.name}")
277             return
278         doc_id = dbmod.insert_document(
279             conn,
280             source="file",
281             external_id=pdf.name,
282             doc_type=doc_type,
283             sha256=ex.sha256,
284             path=str(pdf.resolve()),
285             status="parsed",
286         )
287         for code, src in found:
288             dbmod.insert_offer(
289                 conn,
290                 document_id=doc_id,
291                 promo_code=code,
292                 expiry_raw=expiry_raw,
293                 expiry_type=expiry_type,
294                 source=src,
295                 source_text=ex.text[:2000],
296             )
297
298         summary = ", ".join(f"{c} [{s}]" for c, s in found) if found else "none"
299         typer.echo(f"ingested {pdf.name} (doc {doc_id}); {len(found)} offer(s): {summary}")
300
301
302 @offers_app.command("list")
303 def offers_list(
304     db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
305 ) -> None:
306     """List extracted offers."""
307     from muneem import db as dbmod
308
309     db_path = db or load_config().db_path
310     if not Path(db_path).exists():
311         typer.echo("no offers yet (database not created).")
312         return
313
314     with _ledger(db_path) as conn:
315         dbmod.init_db(conn)
316         rows = dbmod.list_offers(conn)
317
318     if not rows:
319         typer.echo("no offers found.")
320         return
321     for row in rows:
322         who = row["merchant"] or row["external_id"] or "?"

```

```

323         when = f" ? expires {row['expiry_raw']}" if row["expiry_raw"] else ""
324         tag = " [ocr ? verify]" if row["source"] == "ocr" else ""
325         typer.echo(f"[{row['id']}] {who}: {row['promo_code']}{when}{tag}")
326
327
328     @txns_app.command("list")
329     def txns_list(
330         bank: str = typer.Argument(..., help="Which bank's transactions to list (e.g. hdfc,
axis)."),
331         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
332         limit: int = typer.Option(50, help="Max number of transactions to show."),
333     ) -> None:
334         """List parsed transactions for a specific bank."""
335         from muneem import db as dbmod
336
337         db_path = db or load_config().db_path
338         if not Path(db_path).exists():
339             typer.echo("database not created yet.")
340             return
341
342         bank = bank.lower()
343         if bank not in BANK_DISPATCH:
344             typer.echo(f"Unknown bank: {bank}", err=True)
345             return
346
347         list_fn, neg_key, pos_key = BANK_DISPATCH[bank]
348         with _ledger(db_path) as conn:
349             dbmod.init_db(conn)
350             rows = list_fn(conn, limit=limit)
351
352         if not rows:
353             typer.echo(f"no {bank} transactions found.")
354             return
355         for row in rows:
356             if bank == "axis_cc":
357                 amount = f"+{row[pos_key]}" if row["type"] == "Cr" else f"-{row[neg_key]}"
358             else:
359                 amount = f"+{row[pos_key]}" if row[pos_key] else f"-{row[neg_key]}"
360             part = (row["particulars"] or "")[:50]
361             typer.echo(f"[{row['account_number']}] {row['txn_date']} : {part:<50} |
{amount:>10}")
362
363
364     @db_app.command("status")
365     def db_status(
366         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
367     ) -> None:
368         """Show the ledger's encryption state (no statement data is read or printed)."""
369         from muneem import db as dbmod
370         from muneem.db_key import peek_key
371
372         db_path = db or load_config().db_path
373         key_state = "present in keychain" if peek_key() else "not yet minted"
374         if not Path(db_path).exists():
375             typer.echo(f"ledger : {db_path} (does not exist yet)")
376             typer.echo(f"key : {key_state}")
377             return
378         if dbmod.is_plaintext_sqlite(db_path):
379             state = "PLAINTEXT ? run 'muneem db encrypt' to harden it"
380         else:
381             state = "encrypted (SQLCipher)"
382         typer.echo(f"ledger : {db_path}")
383         typer.echo(f"state : {state}")

```

```

384         typer.echo(f"key      : {key_state}")
385
386
387     @db_app.command("encrypt")
388     def db_encrypt(
389         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
390 dir)."),
391     ) -> None:
392         """Migrate an existing *plaintext* ledger to an encrypted one (SQLCipher), in
393 place."""
394         from muneem import db as dbmod
395         from muneem.db_key import get_or_create_key
396
397         db_path = db or load_config().db_path
398         if not Path(db_path).exists():
399             typer.echo(
400                 f"nothing to encrypt: {db_path} does not exist (a new ledger is born
401 encrypted)."
402             )
403             return
404         if not dbmod.is_plaintext_sqlite(db_path):
405             typer.echo(
406                 f"{Path(db_path).name} is already encrypted (or not a SQLite file); nothing
407 to do."
408             )
409             return
410         n = dbmod.encrypt_database(db_path, get_or_create_key())
411         typer.echo(
412             f"encrypted {Path(db_path).name}: {n} table(s) migrated. A plaintext backup
413 remains at "
414             f"{Path(db_path).name}.plaintext.bak ? verify, then securely delete it."
415         )
416
417
418     def _require_encrypted_ledger(db_path: Path) -> str:
419         """Return the ledger's keychain key, or exit with a clear message if it can't be
420 used."""
421         from muneem import db as dbmod
422         from muneem.db_key import peek_key
423
424         if not Path(db_path).exists() or dbmod.is_plaintext_sqlite(db_path):
425             typer.echo("no encrypted ledger here (run 'muneem db encrypt' first).",
426 err=True)
427             raise typer.Exit(1)
428         key = peek_key()
429         if not key:
430             typer.echo("no encryption key in the keychain for this ledger.", err=True)
431             raise typer.Exit(1)
432         return key
433
434
435     @db_app.command("rotate-key")
436     def db_rotate_key(
437         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
438 dir)."),
439     ) -> None:
440         """Rotate the ledger's encryption key (SQLCipher rekey) and update the keychain."""
441         from muneem import db as dbmod
442         from muneem import db_key
443
444         db_path = db or load_config().db_path
445         current = _require_encrypted_ledger(db_path)
446         fresh = db_key.new_key()
447         # Keychain-first with rollback: if the file rekey fails, the keychain still matches
448 the file.

```

```

440     db_key.set_key(fresh)
441     try:
442         dbmod.rekey_database(db_path, current, fresh)
443     except dbmod.EncryptedDatabaseError as exc:
444         db_key.set_key(current) # roll back; the file is still under the old key
445         typer.echo(str(exc), err=True)
446         raise typer.Exit(1) from exc
447     typer.echo(f"rotated the encryption key for {Path(db_path).name}.")
448
449
450     @db_app.command("backup")
451     def db_backup(
452         dest: Path = typer.Argument(..., help="Destination path for the encrypted backup
copy."),
453         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
454     ) -> None:
455         """Write an encrypted online-backup copy of the ledger (consistent even with WAL
active)."""
456         from muneem import db as dbmod
457
458         db_path = db or load_config().db_path
459         key = _require_encrypted_ledger(db_path)
460         try:
461             dbmod.backup_database(db_path, dest, key)
462         except dbmod.EncryptedDatabaseError as exc:
463             typer.echo(str(exc), err=True)
464             raise typer.Exit(1) from exc
465         typer.echo(f"backed up {Path(db_path).name} to {dest} (encrypted).")
466
467
468     @db_app.command("setup-recovery")
469     def db_setup_recovery(
470         out: Path | None = typer.Option(
471             None, help="Recovery file path (default: <data dir>/muneem-recovery.json)."
472         ),
473     ) -> None:
474         """Create a passphrase-wrapped recovery file for the ledger's encryption key.
475
476         Lets you restore the key (and thus the ledger + backups) if the OS keychain is ever
lost. The
477         file holds only the *wrapped* key ? useless without the passphrase.
478         """
479         import getpass
480
481         from muneem import recovery
482         from muneem.db_key import peek_key
483
484         key = peek_key()
485         if not key:
486             typer.echo("no encryption key in the keychain yet (create the ledger first).",
err=True)
487             raise typer.Exit(1)
488         out_path = out or (load_config().data_dir / "muneem-recovery.json")
489         passphrase = getpass.getpass("New recovery passphrase: ")
490         if passphrase != getpass.getpass("Confirm passphrase: "):
491             typer.echo("passphrases did not match.", err=True)
492             raise typer.Exit(1)
493         try:
494             recovery.write_recovery_file(out_path, key, passphrase)
495         except recovery.RecoveryError as exc:
496             typer.echo(str(exc), err=True)
497             raise typer.Exit(1) from exc
498         typer.echo(
499             f"wrote recovery file to {out_path}. Store it OFF this machine (password manager

```

```

/ "
500         "separate drive) and remember the passphrase ? you need BOTH to recover. Without
them, a "
501         "lost keychain means the ledger and its backups are unrecoverable."
502     )
503
504
505     @db_app.command("recover")
506     def db_recover(
507         file: Path = typer.Argument(..., exists=True, dir_okay=False, help="Recovery file
path."),
508         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
509     ) -> None:
510         """Restore the ledger's encryption key from a recovery file + passphrase (into the
keychain)."""
511         import getpass
512
513         from muneem import db as dbmod
514         from muneem import db_key, recovery
515
516         passphrase = getpass.getpass("Recovery passphrase: ")
517         try:
518             key = recovery.read_recovery_file(file, passphrase)
519         except recovery.RecoveryError as exc:
520             typer.echo(str(exc), err=True)
521             raise typer.Exit(1) from exc
522         db_path = db or load_config().db_path
523         # If the ledger is present, verify the recovered key actually opens it before
committing it.
524         if Path(db_path).exists() and not dbmod.is_plaintext_sqlite(db_path):
525             try:
526                 with dbmod.connect(db_path, key=key) as conn:
527                     conn.execute("SELECT 1")
528             except dbmod.EncryptedDatabaseError as exc:
529                 typer.echo("recovered a key, but it does not open the current ledger.",
err=True)
530                 raise typer.Exit(1) from exc
531         db_key.set_key(key)
532         typer.echo("recovered the encryption key into the keychain.")
533
534
535     if __name__ == "__main__":
536         app()
537

```

19. user (2026-06-10T16:14:35.448Z)

```

1     """Local SQLite store: the ``documents`` and ``offers`` slice of the schema.
2
3     This is the minimal slice the promo spine needs (ROADMAP A.2). The full schema
4     (accounts, transactions, holdings, merchants, items) grows with later milestones.
5     ``documents.sha256`` is UNIQUE so re-ingesting the same file is a no-op.
6     ``offers.source`` records provenance ('text' = exact text layer, 'ocr' = approximate).
7     """
8
9     from __future__ import annotations
10
11     import os
12     import sqlite3
13     from collections.abc import Iterator
14     from contextlib import contextmanager
15     from pathlib import Path
16
17     from muneem.errors import EncryptedDatabaseError # re-exported so

```

```

db.EncryptedDatabaseError works
18
19     SCHEMA_VERSION = 4
20
21     # Unencrypted SQLite files start with this 16-byte magic header; an encrypted
(SQLCipher)
22     # file does not, so we can tell them apart without a key.
23     _SQLITE_MAGIC = b"SQLite format 3\x00"
24
25     _SCHEMA = ""
26     CREATE TABLE IF NOT EXISTS documents (
27         id            INTEGER PRIMARY KEY,
28         source        TEXT NOT NULL,
29         external_id   TEXT,
30         sender        TEXT,
31         subject       TEXT,
32         received_date TEXT,
33         doc_type      TEXT,
34         sha256        TEXT NOT NULL UNIQUE,
35         path          TEXT,
36         status        TEXT NOT NULL DEFAULT 'parsed',
37         created_at    TEXT NOT NULL DEFAULT (datetime('now'))
38     );
39
40     CREATE TABLE IF NOT EXISTS offers (
41         id            INTEGER PRIMARY KEY,
42         document_id   INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
43         merchant      TEXT,
44         promo_code    TEXT,
45         description   TEXT,
46         expiry_raw    TEXT,
47         expiry_type   TEXT,
48         source        TEXT NOT NULL DEFAULT 'text',
49         redeemed      INTEGER NOT NULL DEFAULT 0,
50         source_text   TEXT
51     );
52
53     CREATE INDEX IF NOT EXISTS idx_offers_document ON offers(document_id);
54
55     -- We use separate tables for each bank (e.g. axis_transactions, hdfc_transactions)
56     -- instead of a unified table to preserve raw schema fidelity of the source PDFs.
57     -- Normalization into a unified view layer is planned for a later phase
58     -- once all bank schemas are stabilized.
59     CREATE TABLE IF NOT EXISTS axis_transactions (
60         id INTEGER PRIMARY KEY,
61         document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
62         account_number TEXT,
63         txn_date TEXT,
64         value_date TEXT,
65         particulars TEXT,
66         debit REAL,
67         credit REAL,
68         balance REAL
69     );
70
71     CREATE TABLE IF NOT EXISTS axis_cc_transactions (
72         id INTEGER PRIMARY KEY,
73         document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
74         account_number TEXT,
75         txn_date TEXT,
76         particulars TEXT,
77         merchant_category TEXT,
78         amount REAL,
79         type TEXT
80     );

```

```

81
82 CREATE TABLE IF NOT EXISTS au_transactions (
83     id INTEGER PRIMARY KEY,
84     document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
85     account_number TEXT,
86     txn_date TEXT,
87     value_date TEXT,
88     particulars TEXT,
89     debit REAL,
90     credit REAL,
91     balance REAL
92 );
93
94 CREATE TABLE IF NOT EXISTS hdfc_transactions (
95     id INTEGER PRIMARY KEY,
96     document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
97     account_number TEXT,
98     txn_date TEXT,
99     value_date TEXT,
100    particulars TEXT,
101    ref TEXT,
102    withdrawal REAL,
103    deposit REAL,
104    closing_balance REAL
105 );
106 """
107
108
109 def _raw_key_pragma(key: str) -> str:
110     """The SQLCipher raw-key PRAGMA: use the 256-bit ``key`` (64 hex chars) directly, no
KDF."""
111     return f"PRAGMA key = \"x'{key}'\""
112
113
114 def is_plaintext_sqlite(db_path: Path | str) -> bool:
115     """True iff the file exists and begins with the *unencrypted* SQLite magic header.
116
117     Lets us refuse to silently treat a plaintext ledger as encrypted (and vice-versa)
118     without needing the key.
119     """
120     try:
121         with open(db_path, "rb") as fh:
122             return fh.read(16) == _SQLITE_MAGIC
123     except OSError:
124         return False
125
126
127 def _harden_permissions(path: str) -> None:
128     """Best-effort: restrict the DB file to its owner (0600). No-op on failure or
`:memory:`."""
129     if path == ":memory:":
130         return
131     try:
132         os.chmod(path, 0o600)
133     except OSError:
134         pass
135
136
137 def _open_encrypted(path: str, key: str) -> sqlite3.Connection:
138     """Open an encrypted connection via SQLCipher, verifying the key before returning.
139
140     A keyed open NEVER silently downgrades to plaintext: if the binding is missing, or
the
141     key is wrong (or the file is not an encrypted muneem ledger), this raises.
142     """

```

```

143     try:
144         import sqlcipher3
145     except ImportError as exc: # pragma: no cover - exercised only without the binding
146         raise EncryptedDatabaseError(
147             "database encryption requested but the 'sqlcipher3' package is not
installed"
148         ) from exc
149
150     conn = sqlcipher3.connect(path)
151     conn.execute(_raw_key_pragma(key)) # MUST be the first statement on the connection
152     try:
153         conn.execute("SELECT count(*) FROM sqlite_master") # forces key verification
154     except sqlcipher3.DatabaseError as exc:
155         conn.close()
156         raise EncryptedDatabaseError(
157             f"could not open encrypted database {Path(path).name}: wrong key, or not an
"
158             "encrypted muneem ledger"
159         ) from exc
160     conn.row_factory = sqlcipher3.Row
161     return conn
162
163
164     @contextmanager
165     def connect(db_path: Path | str, *, key: str | None = None) ->
Iterator[sqlite3.Connection]:
166         """Open a connection (commit on success, always close). Parent dirs created.
167
168         With ``key`` (a 64-hex-char SQLCipher key) the database is **encrypted at rest** via
169         SQLCipher. Without it, a plaintext stdlib :mod:`sqlite3` connection is used ? the
path for
170         ``:memory:`` and tests. A keyed open never silently falls back to plaintext (see
171         :func:`_open_encrypted`); the production ledger is opened via
:func:`connect_encrypted`.
172         """
173         path = str(db_path)
174         if path != ":memory:":
175             Path(path).parent.mkdir(parents=True, exist_ok=True)
176         if key is not None:
177             conn = _open_encrypted(path, key)
178             _harden_permissions(path)
179         else:
180             conn = sqlite3.connect(path)
181             conn.row_factory = sqlite3.Row
182             conn.execute("PRAGMA foreign_keys = ON")
183         try:
184             yield conn
185             conn.commit()
186         finally:
187             conn.close()
188
189
190     @contextmanager
191     def connect_encrypted(db_path: Path | str) -> Iterator[sqlite3.Connection]:
192         """Open the real on-disk ledger, encrypted with the keychain-managed key.
193
194         The production path: resolves (or mints, on first use) the 256-bit key from the OS
195         keychain and opens with SQLCipher. If ``db_path`` is a pre-existing *plaintext*
SQLite
196         file, refuses with a clear pointer to ``muneem db encrypt`` rather than a cryptic
197         "file is not a database" error.
198         """
199         from muneem.db_key import get_or_create_key
200
201         path = str(db_path)

```



```

202     if is_plaintext_sqlite(path):
203         raise EncryptedDatabaseError(
204             f"{Path(path).name} is an unencrypted SQLite database; run 'muneem db
encrypt' "
205             "to migrate it to an encrypted ledger"
206         )
207     with connect(path, key=get_or_create_key()) as conn:
208         yield conn
209
210
211     def encrypt_database(db_path: Path | str, key: str) -> int:
212         """Migrate a plaintext SQLite ledger to an encrypted one in place, via
`sqlcipher_export`.
213
214         The original is preserved as `.plaintext.bak` for the caller to verify and
then
215         securely delete (it still holds the cleartext data). Returns the number of user
tables
216         copied. Raises :class:`EncryptedDatabaseError` if `db_path` is not a plaintext
SQLite file.
217         """
218         import sqlcipher3
219
220         path = Path(db_path)
221         if not is_plaintext_sqlite(path):
222             raise EncryptedDatabaseError(
223                 f"{path.name} is not a plaintext SQLite database (already encrypted or
missing)"
224             )
225
226         enc_tmp = path.with_name(path.name + ".enc.tmp")
227         enc_tmp.unlink(missing_ok=True)
228         try:
229             src = sqlcipher3.connect(str(path)) # open the plaintext source (no key)
230             try:
231                 src_tables = src.execute(
232                     "SELECT count(*) FROM sqlite_master WHERE type = 'table'"
233                 ).fetchone()[0]
234                 target = str(enc_tmp).replace("\\", "/") # SQL string literal: forward
slashes
235                 src.execute(f"ATTACH DATABASE '{target}' AS encrypted KEY '{key}'")
236                 src.execute("SELECT sqlcipher_export('encrypted')")
237                 src.execute("DETACH DATABASE encrypted")
238             finally:
239                 src.close()
240
241             # Verify the encrypted copy BEFORE the destructive swap: it must open with the
key, pass
242             # an integrity check, and carry every table across. (The plaintext .bak is kept
either way,
243             # so a failed check is recoverable ? but we refuse to swap in a bad file.)
244             with connect(str(enc_tmp), key=key) as check:
245                 integrity = check.execute("PRAGMA integrity_check").fetchone()[0]
246                 ntables = check.execute(
247                     "SELECT count(*) FROM sqlite_master WHERE type = 'table'"
248                 ).fetchone()[0]
249             if integrity != "ok" or ntables != src_tables:
250                 raise EncryptedDatabaseError(
251                     f"encryption migration verification failed for {path.name} "
252                     f"(integrity={integrity!r}, tables {ntables}/{src_tables}); original
left untouched"
253                 )
254
255         backup = path.with_name(path.name + ".plaintext.bak")
256         backup.unlink(missing_ok=True)

```

```

257         path.rename(backup)
258         enc_tmp.rename(path) # consumed on success
259         _harden_permissions(str(path))
260     finally:
261         # Never leave a partial temp behind: no-op on success (already renamed), cleans
up on error.
262         enc_tmp.unlink(missing_ok=True)
263     return int(ntables)
264
265
266     def rekey_database(db_path: Path | str, current_key: str, new_key: str) -> None:
267         """Re-encrypt the ledger under ``new_key`` in place (SQLCipher ``PRAGMA rekey``).
268
269         Opens with ``current_key`` first (which verifies it), then rekeys. Raises
270         :class:`EncryptedDatabaseError` if ``current_key`` is wrong or the file is not an
encrypted
271         muneem ledger. The caller is responsible for updating the keychain entry to
``new_key`` only
272         after this returns successfully.
273         """
274         conn = _open_encrypted(str(db_path), current_key) # validates current_key
275         try:
276             conn.execute(f"PRAGMA rekey = '{new_key}'")
277             conn.commit()
278         finally:
279             conn.close()
280         _harden_permissions(str(db_path))
281
282
283     def backup_database(db_path: Path | str, dest: Path | str, key: str) -> None:
284         """Write an **encrypted** online-backup copy of the ledger to ``dest`` (same key).
285
286         Uses the SQLite Online Backup API for a consistent snapshot ? safe even with WAL
active, so no
287         naive file copy and no manual checkpoint needed. The destination is keyed with the
same key, so
288         it stays encrypted at rest. Raises :class:`EncryptedDatabaseError` if ``key`` is
wrong.
289         """
290         import sqlcipher3
291
292         dest_path = Path(dest)
293         dest_path.parent.mkdir(parents=True, exist_ok=True)
294         dest_path.unlink(missing_ok=True)
295         source = _open_encrypted(str(db_path), key) # validates key
296         try:
297             target = sqlcipher3.connect(str(dest_path))
298             try:
299                 target.execute(_raw_key_pragma(key)) # key the destination so the backup is
encrypted
300                 source.backup(target) # consistent snapshot of the live DB
301             finally:
302                 target.close()
303         finally:
304             source.close()
305         _harden_permissions(str(dest_path))
306
307
308     def init_db(conn: sqlite3.Connection) -> None:
309         """Bring the schema up to date via the migration runner (creates tables on a fresh
DB).
310
311         The current ``_SCHEMA`` is the frozen *baseline* (version ``SCHEMA_VERSION``);
future changes
312         are forward migrations in :mod:`muneem.migrations`, recorded in

```

```

``schema_migrations``.
313     """
314     from muneem import migrations
315
316     migrations.run_migrations(conn, baseline_sql=_SCHEMA,
baseline_version=SCHEMA_VERSION)
317
318
319     def get_document_by_sha(conn: sqlite3.Connection, sha256: str) -> sqlite3.Row | None:
320         return conn.execute("SELECT * FROM documents WHERE sha256 = ?",
(sha256,)).fetchone()
321
322
323     def _insert_row(conn: sqlite3.Connection, table: str, fields: dict) -> int:
324         """Internal helper for the simple INSERT pattern used by documents, offers, and bank
txns."""
325         cols = ", ".join(fields)
326         placeholders = ", ".join("?" for _ in fields)
327         cur = conn.execute(
328             f"INSERT INTO {table} ({cols}) VALUES ({placeholders})",
329             tuple(fields.values()),
330         )
331         rowid = cur.lastrowid # always set after a successful INSERT; guard for the type-
checker
332         if rowid is None: # pragma: no cover - defensive; INSERT always sets lastrowid
333             raise RuntimeError(f"INSERT into {table} did not return a row id")
334         return int(rowid)
335
336
337     def insert_document(conn: sqlite3.Connection, **fields: object) -> int:
338         """Insert into documents table. Delegates to the shared helper for DRY."""
339         return _insert_row(conn, "documents", dict(fields))
340
341
342     def record_provenance(
343         conn: sqlite3.Connection,
344         *,
345         document_id: int,
346         source_type: str,
347         parser: str | None = None,
348         parser_version: str | None = None,
349         mapping_layer: str | None = None,
350         confidence: str | None = None,
351     ) -> None:
352         """Upsert the provenance row for a parse (one per document ? see DESIGN §8.7).
353
354         First-class but *not* per transaction row: every txn table FKs ``documents(id)`, so
a single
355         provenance record per document is reachable by join. Idempotent on ``document_id``
(a re-parse
356         overwrites rather than duplicating).
357         """
358         conn.execute(
359             "INSERT INTO provenance "
360             "(document_id, source_type, parser, parser_version, mapping_layer, confidence) "
361             "VALUES (?, ?, ?, ?, ?, ?) "
362             "ON CONFLICT(document_id) DO UPDATE SET "
363             "source_type=excluded.source_type, parser=excluded.parser, "
364             "parser_version=excluded.parser_version, mapping_layer=excluded.mapping_layer, "
365             "confidence=excluded.confidence",
366             (document_id, source_type, parser, parser_version, mapping_layer, confidence),
367         )
368
369
370     def record_parse(

```

```

371     conn: sqlite3.Connection,
372     document_id: int,
373     *,
374     confidence: str,
375     parser: str,
376     source_type: str = "pdf_statement",
377     parser_version: str | None = None,
378     mapping_layer: str = "deterministic",
379 ) -> None:
380     """Record a parse outcome: set ``documents.status`` and upsert provenance (one per
parse)."""
381     conn.execute("UPDATE documents SET status = ? WHERE id = ?", (confidence,
document_id))
382     record_provenance(
383         conn,
384         document_id=document_id,
385         source_type=source_type,
386         parser=parser,
387         parser_version=parser_version,
388         mapping_layer=mapping_layer,
389         confidence=confidence,
390     )
391
392
393     def get_provenance(conn: sqlite3.Connection, document_id: int) -> sqlite3.Row | None:
394         return conn.execute("SELECT * FROM provenance WHERE document_id = ?",
(document_id,)).fetchone()
395
396
397     def insert_offer(conn: sqlite3.Connection, **fields: object) -> int:
398         return _insert_row(conn, "offers", fields)
399
400
401     def list_offers(conn: sqlite3.Connection) -> list[sqlite3.Row]:
402         return conn.execute(
403             """
404             SELECT o.id, o.merchant, o.promo_code, o.expiry_raw, o.description, o.source,
405                    d.external_id, d.path
406             FROM offers o
407             JOIN documents d ON d.id = o.document_id
408             ORDER BY o.id
409             """
410         ).fetchall()
411
412
413     def insert_axis_transaction(conn: sqlite3.Connection, **fields: object) -> int:
414         return _insert_row(conn, "axis_transactions", fields)
415
416
417     def insert_axis_cc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
418         return _insert_row(conn, "axis_cc_transactions", fields)
419
420
421     def list_axis_cc_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
422         return conn.execute(
423             "SELECT * FROM axis_cc_transactions ORDER BY id DESC LIMIT ?", (limit,)
424         ).fetchall()
425
426
427     def list_axis_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
428         return conn.execute(
429             "SELECT * FROM axis_transactions ORDER BY id DESC LIMIT ?", (limit,)
430         ).fetchall()

```

```

431
432
433     def insert_hdfc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
434         return _insert_row(conn, "hdfc_transactions", fields)
435
436
437     def list_hdfc_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
438         return conn.execute(
439             "SELECT * FROM hdfc_transactions ORDER BY id DESC LIMIT ?", (limit,)
440         ).fetchall()
441
442
443     def insert_au_transaction(conn: sqlite3.Connection, **fields: object) -> int:
444         return _insert_row(conn, "au_transactions", fields)
445
446
447     def list_au_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
448         return conn.execute(
449             "SELECT * FROM au_transactions ORDER BY id DESC LIMIT ?", (limit,)
450         ).fetchall()
451

```

20. user (2026-06-10T16:14:41.201Z)

```

1     """Hand-rolled, versioned, checksummed schema migrations (feedbackJune2026 P6).
2
3     Deliberately not Alembic ? muneem stays raw-`sqlite3` / stdlib-light. Migrations are
an ordered
4     list; each is applied exactly once, inside the (possibly encrypted) connection, and
recorded in a
5     ``schema_migrations`` table with a checksum. The schema-lock test
(`tests/test_db_invariants.py`)
6     pins the resulting shape, so accidental drift and deliberate changes both surface in
review.
7
8     Model:
9     - Baseline. The current schema (`db._SCHEMA`, version `db.SCHEMA_VERSION`) is the
frozen
10     baseline ? never edit it for new changes; add a migration instead. A fresh DB gets the
baseline
11     applied; a legacy DB created before this framework (tables present, no
`schema_migrations`) is
12     detected and its baseline row back-filled (no DDL re-run).
13     - Forward migrations. Entries in `MIGRATIONS` with ``version > baseline`` run in
order. (None
14     yet ? the first lands with the unified-schema work, feedbackJune2026 P3.)
15
16     Known limitation (single-user, reviewed migrations): a crash between applying a
migration's DDL
17     and recording it could require manual cleanup; per-migration transactions can be added
when the
18     first non-trivial migration lands.
19     """
20
21     from __future__ import annotations
22
23     import hashlib
24     import sqlite3
25     from dataclasses import dataclass
26
27     from muneem.errors import MigrationError
28
29     _SCHEMA_MIGRATIONS = """

```

```

30     CREATE TABLE IF NOT EXISTS schema_migrations (
31         version    INTEGER PRIMARY KEY,
32         name       TEXT NOT NULL,
33         checksum   TEXT NOT NULL,
34         applied_at TEXT NOT NULL DEFAULT (datetime('now'))
35     );
36     """
37
38
39     @dataclass(frozen=True)
40     class Migration:
41         """One forward migration: a version, a name, and the DDL to apply (run via
42         executescript)."""
43         version: int
44         name: str
45         sql: str
46
47         @property
48         def checksum(self) -> str:
49             return hashlib.sha256(self.sql.encode("utf-8")).hexdigest()
50
51
52     # --- Forward migrations (version > baseline), applied in ascending order
53     -----
54     # v5: provenance/confidence as a FIRST-CLASS but SEPARATE table (DESIGN §8.7,
55     # feedbackJune2026 P3).
56     # Keyed by document_id (1:1) so every txn row reaches its provenance by the existing FK
57     # join ? no
58     # per-row redundancy. source_type readies it for non-PDF sources (aa/sms) we may add
59     # later.
60     _M5_PROVENANCE = """
61     CREATE TABLE IF NOT EXISTS provenance (
62         id            INTEGER PRIMARY KEY,
63         document_id   INTEGER NOT NULL UNIQUE REFERENCES documents(id) ON DELETE CASCADE,
64         source_type    TEXT NOT NULL,
65         parser        TEXT,
66         parser_version TEXT,
67         mapping_layer TEXT,
68         confidence    TEXT,
69         created_at    TEXT NOT NULL DEFAULT (datetime('now'))
70     );
71     """
72
73
74     MIGRATIONS: list[Migration] = [
75         Migration(version=5, name="provenance", sql=_M5_PROVENANCE),
76     ]
77
78
79     def _checksum(sql: str) -> str:
80         return hashlib.sha256(sql.encode("utf-8")).hexdigest()
81
82
83     def _user_tables(conn: sqlite3.Connection) -> list[str]:
84         return [
85             r[0]
86             for r in conn.execute(
87                 "SELECT name FROM sqlite_master WHERE type='table' "
88                 "AND name NOT LIKE 'sqlite_%' AND name != 'schema_migrations'"
89             )
90         ]
91
92
93     def _record(conn: sqlite3.Connection, version: int, name: str, checksum: str) -> None:

```

```

90         conn.execute(
91             "INSERT INTO schema_migrations (version, name, checksum) VALUES (?, ?, ?)",
92             (version, name, checksum),
93         )
94
95
96     def run_migrations(
97         conn: sqlite3.Connection,
98         *,
99         baseline_sql: str,
100         baseline_version: int,
101         migrations: list[Migration] | None = None,
102     ) -> int:
103         """Bring ``conn``'s schema up to date. Returns the resulting ``user_version``.
104
105         Idempotent: re-running applies nothing already recorded. Raises
106         :class:`MigrationError` if the
107         frozen baseline's checksum no longer matches what was recorded (i.e. someone edited
108         the baseline
109         instead of adding a migration).
110         """
111         migrations = MIGRATIONS if migrations is None else migrations
112         conn.executescript(_SCHEMA_MIGRATIONS) # bookkeeping table first (idempotent)
113         applied = {r[0] for r in conn.execute("SELECT version FROM schema_migrations")}
114         current = conn.execute("PRAGMA user_version").fetchone()[0]
115         baseline_checksum = _checksum(baseline_sql)
116
117         if baseline_version in applied:
118             recorded = conn.execute(
119                 "SELECT checksum FROM schema_migrations WHERE version = ?",
120                 (baseline_version,)
121             ).fetchone()
122             if recorded and recorded[0] != baseline_checksum:
123                 raise MigrationError(
124                     "baseline schema changed without a migration ? add a migration instead
125                     of editing "
126                     "the frozen baseline (feedbackJune2026 P6)"
127                 )
128         else:
129             # Fresh DB ? create the baseline; legacy DB (tables already present) ? just
130             record it.
131             if not _user_tables(conn):
132                 conn.executescript(baseline_sql)
133             _record(conn, baseline_version, "baseline", baseline_checksum)
134             if current < baseline_version:
135                 conn.execute(f"PRAGMA user_version = {baseline_version}")
136                 current = baseline_version
137
138         for migration in sorted(migrations, key=lambda m: m.version):
139             if migration.version <= baseline_version or migration.version in applied:
140                 continue
141             conn.executescript(migration.sql)
142             _record(conn, migration.version, migration.name, migration.checksum)
143             conn.execute(f"PRAGMA user_version = {migration.version}")
144             current = migration.version
145
146         return int(current)

```

21. user (2026-06-10T16:14:41.504Z)

```

1     """Outlook / Microsoft Graph email provider ? ?? EXPERIMENTAL / INCOMPLETE.
2
3     Device-flow auth + basic search are implemented; attachment/header/download are
4     ``NotImplementedError``. This provider is **not wired into any CLI command** and is not

```

```

on a
5      supported path yet (see docs/CURRENT_STATE.md §1). Do not rely on it; it is excluded
from the
6      strict mypy gate (pyproject ``[[tool.mypy.overrides]]``) until the ``fetch`` command
lands
7      (ROADMAP B.4 / feedbackJune2026 P7). Requires the optional ``outlook`` extra (msal +
requests).
8      """
9
10     from __future__ import annotations
11
12     import json
13     from pathlib import Path
14
15     from .base import EmailProvider, SearchCriteria
16
17     _TOKEN_KEY = "outlook_token"
18     SCOPES = ["Mail.Read"]
19
20
21     class OutlookProvider(EmailProvider):
22         def __init__(self, client_id: str, authority: str =
"https://login.microsoftonline.com/common"):
23             self.client_id = client_id
24             self.authority = authority
25             self.access_token = None
26
27         def authorize(self) -> None:
28             try:
29                 from msal import PublicClientApplication
30             except ImportError:
31                 msg = "msal is required for OutlookProvider. Run pip install msal"
32                 raise RuntimeError(msg) from None
33
34             from muneem import secrets
35
36             app = PublicClientApplication(self.client_id, authority=self.authority)
37
38             # Try to find a cached token (simplified)
39             raw = secrets.get_secret(_TOKEN_KEY)
40             if raw:
41                 token_data = json.loads(raw)
42                 self.access_token = token_data.get("access_token")
43                 # Ideally, handle refresh logic here via msal TokenCache
44                 return
45
46             # Interactive login
47             flow = app.initiate_device_flow(scopes=SCOPES)
48             if "user_code" not in flow:
49                 raise ValueError("Failed to create device flow")
50
51             print(flow["message"]) # Instructs user to go to a URL and enter code
52             result = app.acquire_token_by_device_flow(flow)
53
54             if "access_token" in result:
55                 self.access_token = result["access_token"]
56                 secrets.set_secret(_TOKEN_KEY, json.dumps({"access_token":
self.access_token}))
57             else:
58                 raise RuntimeError(f"Failed to authenticate:
{result.get('error_description')}")
59
60         def _build_filter(self, criteria: SearchCriteria) -> str:

```



```

1      """IMAP email provider ? ?? EXPERIMENTAL / INCOMPLETE.
2
3      Basic auth + search are implemented; ``get_headers`` and the attachment methods are
4      ``NotImplementedError``. This provider is **not wired into any CLI command** and is not
on a
5      supported path yet (see docs/CURRENT_STATE.md §1). Do not rely on it; it is excluded
from the
6      strict mypy gate (pyproject ``[[tool.mypy.overrides]]``) until the ``fetch`` command
lands
7      (ROADMAP B.4 / feedbackJune2026 P7).
8      """
9
10     from __future__ import annotations
11
12     import imaplib
13     from pathlib import Path
14
15     from muneem import secrets
16
17     from .base import EmailProvider, SearchCriteria
18
19     _PASS_KEY = "imap_password"
20
21
22     class IMAPProvider(EmailProvider):
23         def __init__(self, host: str, username: str):
24             self.host = host
25             self.username = username
26             self.mail = None
27
28         def authorize(self) -> None:
29             password = secrets.get_secret(f"{_PASS_KEY}_{self.username}")
30             if not password:

```

23. user (2026-06-10T16:14:51.768Z)

```

1      from __future__ import annotations
2
3      import base64
4      import json
5      from pathlib import Path
6
7      from .base import EmailProvider, SearchCriteria
8
9      SCOPES = ["https://www.googleapis.com/auth/gmail.readonly"]
10     _TOKEN_KEY = "gmail_token"
11
12
13     class GmailProvider(EmailProvider):
14         def __init__(self, credentials_path: Path | str, open_browser: bool = True):
15             self.credentials_path = credentials_path
16             self.open_browser = open_browser
17             self.service = None
18
19         def authorize(self) -> None:
20             from google.auth.transport.requests import Request
21             from google.oauth2.credentials import Credentials
22             from google_auth_oauthlib.flow import InstalledAppFlow
23             from googleapiclient.discovery import build
24
25             from muneem import secrets
26
27             creds = None
28             raw = secrets.get_secret(_TOKEN_KEY)
29             if raw:

```

```

30         creds = Credentials.from_authorized_user_info(json.loads(raw), SCOPES)
31     if creds and creds.valid:
32         pass
33     elif creds and creds.expired and creds.refresh_token:
34         creds.refresh(Request())
35     else:
36         flow = InstalledAppFlow.from_client_secrets_file(str(self.credentials_path),
SCOPES)
37         creds = flow.run_local_server(port=0, open_browser=self.open_browser)
38     secrets.set_secret(_TOKEN_KEY, creds.to_json())
39     self.service = build("gmail", "v1", credentials=creds, cache_discovery=False)
40
41     def _build_query(self, criteria: SearchCriteria) -> str:
42         parts = []
43         if criteria.sender:
44             parts.append(f"from:{criteria.sender}")
45         if criteria.has_attachment:
46             parts.append("has:attachment")
47         if criteria.newer_than_days:
48             parts.append(f"newer_than:{criteria.newer_than_days}d")
49         return " ".join(parts)
50
51     def search_messages(self, criteria: SearchCriteria, max_results: int = 100) ->
list[str]:
52         if not self.service:
53             self.authorize()
54
55         query = self._build_query(criteria)
56         ids: list[str] = []
57         page_token = None
58         while True:
59             resp = (
60                 self.service.users()
61                 .messages()
62                 .list(userId="me", q=query, pageToken=page_token,
maxResults=min(max_results, 100))
63                 .execute()
64             )
65             ids.extend(m["id"] for m in resp.get("messages", []))
66             page_token = resp.get("nextPageToken")
67             if not page_token or len(ids) >= max_results:
68                 break
69         return ids[:max_results]
70
71     def _get_message(self, message_id: str) -> dict:
72         if not self.service:
73             self.authorize()
74         return (
75             self.service.users().messages().get(userId="me", id=message_id,
format="full").execute()
76         )
77
78     def get_headers(self, message_id: str) -> dict[str, str]:
79         message = self._get_message(message_id)
80         return {

```

24. user (2026-06-10T16:14:52.619Z)

```

src\muneem\db.py:342: def record_provenance(
src\muneem\db.py:370: def record_parse(
src\muneem\db.py:382:     record_provenance(
src\muneem\parsers\axis.py:84:         dbmod.record_parse(
src\muneem\parsers\axis.py:105:         dbmod.record_parse(
src\muneem\parsers\axis.py:193:         dbmod.record_provenance(
src\muneem\parsers\hdfc.py:455:         dbmod.record_parse(

```

```
src\muneeem\parsers\au.py:51:         dbmod.record_parse(  
src\muneeem\parsers\au.py:72:         dbmod.record_parse(  

```

25. user (2026-06-10T16:14:59.299Z)

```
1      """The generic concept-mapping engine ? columns in, concepts out, arithmetic decides.  
2  
3      This is the *generic-first* half of the concept-core architecture (see  
4      [`docs/parser-architecture.md`](../.../docs/parser-architecture.md) §4, §6).  
5      Bank-agnostic code that takes a **cell-matrix** (the shape `pdfplumber.extract_tables()`  
returns,  
6      or what a coordinate-clustering extractor emits) and maps its columns onto the canonical  
concepts  
7      via a cheap-first ladder, with a profile supplying priors:  
8  
9      * **L0 ? header lexicon.** Match header-row labels against the profile's concept?label  
aliases.  
10     * **profile-order.** If there is no header, fall back to the profile's known column  
order (a prior).  
11     * **L1/L2 ? content + search.** Classify columns by content (dates, money), then  
enumerate the  
12     plausible amount-column ? {debit, credit, balance} assignments and **keep the one the  
13     reconciliation oracle endorses** ? the self-identifying balance column, found by  
*which mapping  
14     makes the math work* rather than by its label.  
15  
16     The engine is **family-agnostic**: it is parameterized by a :class:`FamilySpec` (the  
target concept  
17     names, which are date/amount/balance, and a typed-row builder) and an injected `oracle`  
18     (`rows -> bool`). Only the *bank* family ships today; an instrument family  
(equity/MF/ETF/bond ? see  
19     the project scope) would reuse this control flow with its own concepts + oracle. Nothing  
here logs a  
20     value; provenance is the winning layer + column map.  
21     """  
22  
23     from __future__ import annotations  
24  
25     import itertools  
26     import re  
27     from collections.abc import Callable, Sequence  
28     from dataclasses import dataclass, field  
29  
30     from .concepts import ConceptRow  
31     from .validation import parse_amount  
32  
33     Cell = object # a table cell: str | None (we coerce defensively)  
34     Row = Sequence[Cell]  
35  
36     # Generic Indian-statement date shapes; a profile can pin its own to avoid false  
positives.  
37     _GENERIC_DATE_PATTERNS: tuple[re.Pattern, ...] = (  
38         re.compile(r"^\d{2}[/-]\d{2}[/-]\d{4}$"), # dd/mm/yyyy or dd-mm-yyyy  
39         re.compile(r"^\d{2}[- ][A-Za-z]{3}[- ]\d{2,4}$"), # dd-Mon-yyyy / dd Mon yy  
40     )  
41     # "Money" requires a decimal point so bare integers (e.g. a long reference number) are  
NOT mistaken  
42     # for an amount column during content classification.  
43     _MONEY_RE = re.compile(r"(?i)^\d{1,2}(\.\d{1,2})?(?:s*(?:cr|dr))?$")  
44     _BALANCE_MARKERS = ("opening balance", "closing balance", "balance b/f", "balance c/f")  
45  
46     # The generic bank lexicon. Profiles extend/override per issuer; keys are concept names,  
values are  
47     # lower-cased header labels matched exactly (not by substring) during L0.  
48     _BANK_ALIASES: dict[str, tuple[str, ...]] = {
```

```

49     "txn_date": ("date", "txn date", "tran date", "transaction date", "posting date"),
50     "value_date": ("value date", "val date", "value dt"),
51     "narration": (
52         "narration",
53         "particulars",
54         "description",
55         "transaction details",
56         "details",
57         "remarks",
58         "transaction",
59         "transaction remarks",
60     ),
61     "reference": (
62         "ref",
63         "reference",
64         "ref no",
65         "reference no",
66         "cheque no",
67         "chq no",
68         "instrument no",
69     ),
70     "debit": ("debit", "withdrawal", "withdrawals", "dr", "paid out", "debit amount"),
71     "credit": ("credit", "deposit", "deposits", "cr", "paid in", "credit amount"),
72     "balance": (
73         "balance",
74         "closing balance",
75         "running balance",
76         "closing bal",
77         "available balance",
78     ),
79 }
80

```

26. user (2026-06-10T16:14:59.664Z)

```

1     """Per-issuer profiles ? bank knowledge expressed as **data, not code** (architecture
2     $6).
3     A profile carries the priors the generic engine needs to disambiguate one issuer: extra
4     concept?label aliases, the accepted date shape, a positional fallback when there's no
5     header,
6     table anchors, account-number patterns, quirks, and (parked) the password rule. Adding
7     an issuer is
8     ideally *adding a profile*, not writing a parser.
9     `HDFC_PROFILE` is data consumed by HDFC's parser, whose *extraction* stays bespoke
10    coordinate-
11    clustering ? the documented exception where `extract_tables()` fails on a semi-ruled
12    table (see the
13    `hdfc-parser-tech-decision` memory). The Axis and AU Bank profiles land with their
14    engine-driven
15    parsers (coordinate clustering ? generic mapping ? total reconcile).
16    """
17
18    from __future__ import annotations
19
20    import re
21
22    from .engine import Profile
23
24    AXIS_PROFILE = Profile(
25        name="axis",
26        aliases={
27            # Real Axis savings header: Tran/Txn Date | Transaction | Withdrawals | Deposits
28            | Balance |

```

```

24         # Other Information. "Transaction" is the narration column (not the date).
25         "narration": ("transaction", "particulars"),
26         "debit": ("withdrawals", "withdrawal", "debit"),
27         "credit": ("deposits", "deposit", "credit"),
28         "balance": ("balance",),
29     },
30     date_patterns=(re.compile(r"^\d{2}[-/]\d{2}[-/]\d{4}$"),), # dd-mm-yyyy or
dd/mm/yyyy
31     # Layout / column-count varies between vintages, so there is NO reliable
column_order ? the
32     # generic content search + total reconcile do the work. Broadened account pattern:
the old
33     # `Account No:` regex missed real statements.
34     account_patterns=(
35         re.compile(r"Account\s*(?:No|Number)\s*[:\.-]?\s*(\d{6,})", re.IGNORECASE),
36         re.compile(r"A/c\s*No\.\s*[:\s]*(\d{6,})", re.IGNORECASE),
37     ),
38     quirks=frozenset({"layout-varies", "other-information-column", "balance-optional"}),
39 )
40
41 AUBANK_PROFILE = Profile(
42     name="aubank",
43     aliases={
44         "narration": (
45             "transaction",
46             "particulars",
47             "description",
48             "narration",
49             "transaction details",
50         ),
51         "debit": ("withdrawal", "withdrawals", "debit", "withdrawal amount", "dr"),
52         "credit": ("deposit", "deposits", "credit", "deposit amount", "cr"),
53         "balance": ("balance", "closing balance", "running balance"),
54     },
55     # AU vintages use dd-mm-yyyy / dd/mm/yyyy or dd-Mon-yyyy; verified on the real
statements.
56     date_patterns=(
57         re.compile(r"^\d{2}[-/]\d{2}[-/]\d{4}$"),
58         re.compile(r"^\d{2}-[A-Za-z]{3}-\d{4}$"),
59     ),
60     account_patterns=(
61         re.compile(r"Account\s*(?:No|Number)\s*[:\.-]?\s*(\d{6,})", re.IGNORECASE),
62         re.compile(r"A/c\s*No\.\s*[:\s]*(\d{6,})", re.IGNORECASE),
63     ),
64     quirks=frozenset({"engine-driven"}),
65 )
66
67 HDFC_PROFILE = Profile(
68     name="hdfc",
69     aliases={
70         "debit": ("withdrawal", "withdrawals"),
71         "credit": ("deposit", "deposits"),
72         "balance": ("closing balance",),
73     },
74     date_patterns=(re.compile(r"^\d{2}/\d{2}/\d{4}$"),), # dd/mm/yyyy
75     table_anchors=("Narration", "Closing Balance"),
76     account_patterns=(re.compile(r"Account\s*Number\s*[:.]?\s*(\d{6,})",
re.IGNORECASE),),
77     # HDFC's transaction table is semi-ruled; extract_tables() is unreliable on it, so
HDFC extracts
78     # by word-position clustering rather than through the generic cell-matrix engine.
79     quirks=frozenset({"semi-ruled-table", "coordinate-clustering"}),
80 )

```

```
testdata/promo-emails/README.md
testdata/promo-emails/airasia-1.pdf
testdata/promo-emails/airasia-2.pdf
testdata/promo-emails/airasia-3.pdf
testdata/promo-emails/dataquest.pdf
testdata/promo-emails/expected.yaml
testdata/promo-emails/flipkart.pdf
testdata/promo-emails/makemytrip-1.pdf
testdata/promo-emails/makemytrip-2.pdf
testdata/promo-emails/swiggy.pdf
testdata/promo-emails/tataneu.pdf
testdata/promo-emails/unacademy.pdf
---ALL-TRACKED-SENSITIVE-CHECK---
docs/ingesting-statements.md
src/muneem/password_rules.py
src/muneem/statement_unlock.py
tests/test_password_rules.py
tests/test_statement_unlock.py
---LOG-CHECK---
done
```

28. user (2026-06-10T16:15:10.617Z)

```
LICENSE
PRIVACY.md
README.md
ROADMAP.md
appsscript.json
docs
esbuild.config.mjs
eslint.config.mjs
html
jest.config.js
package-lock.json
package.json
src
tests
tsconfig.json
```

29. user (2026-06-10T16:15:18.117Z)

```
ARCHITECTURE.md
VISION.md
---
```

Gmail Attachments ? Google Drive

A Google Apps Script that copies your Gmail attachments into a `GmailAttachments` folder in Google Drive and keeps it updated as new mail arrives. You choose how the attachments are organised (by content type, by month, or by year) and how far back the first sync should reach.

Originally a 2013 Chrome Web Store app; this is a modernised rebuild on the **V8 runtime** with a TypeScript source, an esbuild build, and a local Jest test suite. The web app runs **as each accessing user** (``executeAs: USER_ACCESSING``), so it can serve just you, a small group, or a Google Workspace org ? each person's attachments are copied within their own account. See [Multi-user deployment / publishing](#multi-user-deployment--publishing).

```
> Where this is headed: attachment archiving is the hardened core of a larger
> goal ? turning email into a structured, queryable record of one's life
> (receipts, statements, travel, ?) via an ingest ? classify ? extract ? handoff
> pipeline, with domain processing isolated in downstream services. See
> [`docs/VISION.md`](./docs/VISION.md) and [`docs/ARCHITECTURE.md`](./docs/ARCHITECTURE.md).
```

How it works

- ``doGet`` serves a setup form (``html/index.html``).
- ``processInput`` stores your choices and schedules the first background run.
- ``syncUserDrives`` is a self-rescheduling time-based trigger that walks Gmail in 30-day windows, copying allowed attachments into Drive while respecting a daily file cap (``MAX_FILES_ADD_PER_DAY``) and a per-run wall-clock budget.

State lives in ``PropertiesService`` (user properties). Existing Drive files are never modified ? re-running setup only resets the script's own bookkeeping.

Project layout

```
| Path | Purpose |
| --- | --- |
| `src/` | TypeScript source. Pure logic (`query`, `naming`, `quota`, `contentTypes`) is
separated from the Google-service shells (`drive`, `sync`, `ui`, `state`). |
| `src/entry.ts` | Bundle entry; esbuild exposes `doGet` / `processInput` / `syncUserDrives` as
globals. |
| `html/index.html` | The setup form (pushed to Apps Script as `index`). |
| `appsscript.json` | Manifest: V8 runtime, OAuth scopes, web-app settings. |
| `tests/` | Jest unit tests. |
| `dist/` | Build output that `clasp` pushes (git-ignored). |
```

Develop

```
```bash
npm install
npm run typecheck # tsc --noEmit
npm run lint # eslint
npm test # jest
npm run build # bundle src -> dist/Code.js (+ manifest + html)
```
```

Deploy (requires your Google account)

1. ``npm install -g @google/clasp && clasp login``
2. The project is already linked via the committed ``clasp.json`` (``scriptId`` + ``rootDir: dist``). To target a different project, edit ``scriptId``.
3. ``npm run push`` ? builds and ``clasp push``es ``dist/``.
4. In the Apps Script editor, deploy as a **Web app**. The manifest already sets ***Execute as: User accessing*** and ***Who has access: Anyone***, so each visitor

30. user (2026-06-10T16:15:22.777Z)

```
1      # Vision ? Email as a smart way to organize your life
2
3      > Status: living document. The overall product direction. Architectural detail
4      > and the decision log live in [`ARCHITECTURE.md`](./ARCHITECTURE.md).
5
6      ## North star
7
8      Turn a person's email into a **high-fidelity, structured, queryable record of
9      their life** ? receipts, bank statements, flights & travel, coupons, shopping ?
10     built automatically, privately, and reliably. Email becomes the source of truth
11     that organizes the rest.
12
13     ## What this repo is ? and is not
14
15     This repository (`gmail-attachments-to-gdrive`) is the **extraction / ingest
16     layer**. It is the reliable, quota-aware "front door" that:
17
18     - watches a user's Gmail **in their own account**,
```

```

19 - archives raw artifacts (attachments) ? still valuable on its own,
20 - **classifies** messages and **extracts** a clean, normalized signal,
21 - **redacts/minimizes** that signal, and **hands it off** to downstream
22   processors.
23
24 It is **not** the domain parser, the datastore, or the analytics engine. That
25 logic is deliberately **isolated in separate downstream services**:
26
27 | Concern | Owner |
28 | --- | --- |
29 | Gmail watching, archiving, classification, extraction, redaction, handoff | **this
repo** |
30 | Bank-statement & financial parsing, indexing, encrypted storage, analytics |
**Muneem** (`avidullu/muneem`) |
31 | Grocery / receipts, coupons, travel processing | future siblings of Muneem |
32
33 The boundary is intentional: **extraction (this repo) is decoupled from
34 processing (downstream).** This repo never imports domain-parsing logic;
35 downstream services never read Gmail. They meet only at a versioned handoff
36 contract.
37
38 ## Foundation first
39
40 Attachment archiving is the **hardened core**. Everything else is built on its
41 proven strengths, which we preserve and extend, never bypass:
42
43 - per-user isolation (`executeAs: USER_ACCESSING`),
44 - a per-user lock and self-healing trigger fan-out,
45 - the hybrid `history.list` + date-window sync,
46 - "mark a message done only on clean completion,"
47 - pure-logic / GAS-shell separation with heavy unit testing,
48 - ruthless respect for Apps Script quotas and the 6-minute boundary.
49
50 ## Audience & rollout
51
52 - We **build for public** ? the long-term goal is a wide launch.
53 - **For now, stay under 100 users** (OAuth "Testing" mode) to harden the
54   system, learn the value proposition, and make an *educated* call on whether to
55   financially invest (the annual CASA security assessment, hosted infra) **once
56   there is a substantial user base**.
57 - Until that call, we stay in Google's no-verification zone (see
58   [`.//README.md`](./README.md) ? Multi-user deployment / publishing).
59
60 ## Privacy north star
61
62 - **Minimize egress.** Only **non-PII** signal ever leaves the user's Google
63   account, and only to **our own isolated, hosted service** ? **never third
64   parties**.
65 - **Scrub + encrypt** as required, applied *before* anything leaves.
66 - Continuously **reduce** what leaves while maintaining product health.
67 - The in-account baseline needs **zero** external dependency; external
68   processing is a deliberate, **separately-consented** upgrade with a privacy
69   policy that names exactly what is sent, where, and why.
70
71 ### The auditability invariant
72
73 **This repo is the only code that ever touches a user's Gmail, and it stays
74 auditable and self-deployable by a power user at any time.** It is intended to be
75 open-sourced precisely so that a privacy-conscious user can read ? and run ? the
76 exact code that reads their mail. Muneem and every downstream service operate
77 only on minimized, non-PII signal and never hold the Gmail grant. This is a
78 binding constraint on all future work; the enforceable form (and a per-change
79 checklist) lives in [`.//ARCHITECTURE.md`](./ARCHITECTURE.md#auditability-invariant-
binding).
80

```



```

81     ## First verticals
82
83     1. Grocery receipts ? high volume, forgiving fidelity bar; proves the whole
84       ingest ? classify ? extract ? index pipeline cheaply.
85     2. Bank statements ? high-fidelity path; output feeds Muneem, proving the
86       redact ? handoff ? downstream-processing seam.
87
88     ## Success criteria (proposed)
89
90     - The user's index auto-populates with correct entities at high precision;
91       low-confidence items are surfaced for review, never silently dropped.
92     - Re-running an improved extractor over history corrects past entities without
93       duplicates and without re-copying blobs.
94     - Egress stays minimal and non-PII; the in-account baseline works with no
95       external service available.
96
97     ## Phased plan (high level)
98
99     Detailed status is tracked in [`../ROADMAP.md`](../ROADMAP.md); the technical
100    shape of each phase is in [`../ARCHITECTURE.md`](../ARCHITECTURE.md).
101
102    0. Ingest model ? a rich, stable `EmailMessage`; the copier consumes it
103      (no behavior change).
104    1. Classify + extract + index (GAS-only, in-account) ? classifier, pluggable
105      extractors, normalized entities, a user-owned index. No egress.
106    2. Coverage + re-extraction ? more extractors, body/OCR handling, extraction
107      cursors & status, "re-extract for new rules," review queues.
108    3. Handoff to hosted services ? redact/minimize, then feed Muneem & siblings;
109      separate consent + privacy update; extraction/egress budgets.
110    4. Surfaces & ops ? dashboards, Calendar/Tasks automation, a richer review &
111      configuration UI, Shared Drives, schema migration.
112

```

31. user (2026-06-10T16:15:34.619Z)

```

testdata/credentials:
client_secret_207319377548-j0vbfmqak4cq6kqa4s9ikq6l5sgore.apps.googleusercontent.com.json

```

```

testdata/snapshots:
April Credit Card Statement.json
lder Credit Card Statement.json
---GITIGNORE---

```

--- Sensitive data ? NEVER commit ---

Real financial documents stay local only. Everything under testdata/ is sensitive by default ? ONLY the non-sensitive promo corpus and committed synthetic fixtures are

Real statements in ANY subdir (e.g. testdata/hdfc-statements/) must never be committed

```

testdata/*
!testdata/promo-emails/
!testdata/synthetic/
*.db
*.sqlite
*.sqlite3
muneem.db*

```

Secrets and credentials

```

.env
.env.*
!.env.example
*.pem
*.key
credentials.json

```

token.json
token.pickle
oauth_token*
password_rules.yaml
password_rules.yml
password_rules.json
passwords.txt

Logs (may contain extracted statement text ? sanitize before committing any log)

*.log
logs/

--- Python ---

__pycache__/
*.py[cod]
*\$py.class
*.so
.Python
build/
develop-eggs/
dist/
downloads/
eggs/
.eggs/
lib/
lib64/
parts/
sdist/
var/
wheels/
*.egg-info/
.installed.cfg
*.egg
MANIFEST
.venv/
venv/
env/
ENV/
.pytest_cache/
.mypy_cache/
.ruff_cache/
.coverage
htmlcov/
.tox/

--- Go ---

*.exe
*.exe~
*.dll
*.dylib
*.test
*.out
vendor/

--- Node (if a UI later lands) ---

node_modules/
npm-debug.log*
yarn-debug.log*
yarn-error.log*
.next/
.nuxt/
dist/

--- IDE / OS ---

```
.vscode/  
.idea/  
*.swp  
*.sw0  
*~  
.DS_Store  
Thumbs.db  
desktop.ini
```

--- Jupyter (in case notebooks are used for exploration) ---

```
.ipynb_checkpoints/
```

--- Scratch / local-only ---

```
scratch/  
local/  
tmp/  
ruff-bin/  
ruff.zip
```

32. user (2026-06-10T16:15:36.288Z)

```
"""Authorize Gmail (read-only) and list the statement emails found.
```

Run by *you* ? it opens a browser for Google consent, caches the token in your OS keychain, then prints the statement emails it can see. Nothing is uploaded; this only reads your inbox. Use it to confirm your OAuth client + consent + search work before the full import pipeline.

```
python tools/gmail_auth.py --credentials testdata/credentials/client_secret*.json  
"""
```

```
from __future__ import annotations
```

```
import argparse  
import glob  
import sys  
from pathlib import Path
```

```
def _find_credentials(arg: str | None) -> Path | None:  
    if arg:  
        return Path(arg)  
    matches = sorted(glob.glob("testdata/credentials/*.json"))  
    return Path(matches[0]) if matches else None
```

```
def main() -> int:  
    parser = argparse.ArgumentParser(  
        description="Authorize Gmail (read-only) and list statement emails."  
    )  
    parser.add_argument("--credentials", help="Path to the OAuth client secret JSON.")  
    parser.add_argument("--sender", default="hdfcbank.net", help="Sender email domain to search.")  
    parser.add_argument("--max", type=int, default=25, help="Max emails to list.")  
    args = parser.parse_args()  
  
    creds_path = _find_credentials(args.credentials)  
    if not creds_path or not creds_path.exists():  
        print(  
            "No OAuth client secret found. Pass --credentials <path>. "  
            "(Create a Desktop OAuth client in Google Cloud Console and download the JSON; "  
            "see README § Getting started.)",
```

```

        file=sys.stderr,
    )
    return 2

from muneem.ingestion.base import SearchCriteria
from muneem.ingestion.gmail import SCOPES, GmailProvider

print(f"Client secret : {creds_path.name}")
print(f"Scope          : {SCOPES[0]} (read-only ? cannot modify or delete mail)")

provider = GmailProvider(creds_path)
provider.authorize()

criteria = SearchCriteria(sender=args.sender, has_attachment=True, newer_than_days=None)
print(f"Searching      : from:{args.sender} has:attachment")

ids = provider.search_messages(criteria, max_results=args.max)
print(f"Found {len(ids)} message(s) with a PDF attachment:\n")
for mid in ids:
    hdrs = provider.get_headers(mid)
===
8:## Auditability invariant (binding)
17:definition, a decision to break the invariant and needs explicit owner sign-off.
20:  account (`executeAs: USER_ACCESSING`). No other component ? Muneem included ?
21:  ever holds the Gmail OAuth grant or sees raw mail. Muneem may *orchestrate*
65: Gmail ??? INGEST ??? CLASSIFY ??? EXTRACT ??? REDACT/MINIMIZE ??? HANDOFF ??? Muneem /
125:- Downstream services (Muneem for finance; siblings for groceries/coupons) own
128: Muneem-facing schema and transport are **TBD and aligned with that repo**
130: here and reconciled with Muneem separately).
141:| `RemoteStore` (our hosted DB) / Muneem-API sink | later | the durable system of record |
194:2. **Hosted, isolated service** (Muneem pattern) for heavy/ML extraction,
202:| ADR-002 | Extraction (this repo) is decoupled from processing (Muneem & siblings) |
Isolate domain logic + encrypted storage + analytics downstream; clean contract seam |
203:| ADR-003 | Storage via a `Store`/`Sink` interface ? Sheets v1, remote/Muneem later |
Extensible by design; never baked into the code |
206:| ADR-006 | First verticals: grocery receipts + bank statements (? Muneem) | Volume + the
high-fidelity handoff path |
208:| ADR-008 | **Option A:** this repo is the sole Gmail-touching component ? in-account,
auditable, self-deployable; Muneem orchestrates but never holds the Gmail grant. Codified as the
binding [auditability invariant](#auditability-invariant-binding). | Verifiable privacy is the
product's moat and shrinks future verification scope; it concentrates the trust-critical surface
in one open, in-account place. |
212:- **Muneem handoff contract:** schema + transport (e.g. `UrlFetch` to a signed
213: endpoint vs. write-to-a-shared-store) ? define jointly with `avidullu/muneem`.
===

```

Roadmap

Next steps for the modernised project, grouped by effort and grounded in the current Apps Script / Gmail platform constraints.

Platform constraints that shape the design

For consumer (gmail.com) accounts:

- ****6 minutes**** max per execution; ****90 minutes/day**** total trigger runtime; ****20 triggers**** per script per user; **~**20,000**** Gmail read calls/day. ([Apps Script quotas](https://developers.google.com/apps-script/guides/services/quotas))
- There is ****no clearly documented daily Drive file-create quota****, so `MAX_FILES_ADD_PER_DAY` (240)` is a self-imposed heuristic, not an API limit.
- Gmail's `[`users.history.list`](https://developers.google.com/workspace/gmail/api/guides/sync)` with a stored `startHistoryId`` is the purpose-built incremental sync (~2 quota units vs 5 for a list call) and returns only changes. A `historyId`` can expire (HTTP 404 ? fall back to full sync), so it complements the date-window backfill rather than replacing it.

- clasp can deploy from CI using a `CLASPRC\_JSON` secret (contents of `~/clasprc.json`); service accounts do **not** work with the Apps Script API, so use a dedicated owner account's token.
([clasp #225](https://github.com/google/clasp/issues/225))

Quick wins

- [x] **Cover the orchestration shells with tests.** `sync.ts` / `ui.ts` had no coverage ? the trickiest logic (trigger fan-out, quota reset, catch-up loop, `processInput` state reset). **(done in this branch; see `tests/sync.test.ts`, `tests/ui.test.ts`)**
- [x] **Raise the per-run wall-clock budget.** Was capped at 1 minute against a 6-minute ceiling, making backfill ~6x slower than necessary. Now `MAX\_RUNTIME\_MS` (5 min) in `src/config.ts`, threaded through `hasTime`.
- [x] **Guard against trigger leakage.** A `LockService` script lock prevents concurrent runs from stacking triggers, deletion is scoped to the sync handler, and a cap prunes any excess. **(done in this branch; `src/sync.ts`)**
- [x] **Add a Jest `coverageThreshold`** so CI fails if coverage regresses. **(done; `jest.config.js`, gated at 85% stmts/lines/funcs, 75% branches, `entry.ts` bundler glue excluded)**

Medium

- [x] **CI/CD auto-deploy on merge to `master`.** A gated `deploy` job writes `CLASPRC\_JSON` ? `~/clasprc.json` and runs `clasp push` / `clasp deploy` against the committed `clasp.json`. **(done; `.github/workflows/ci.yml`. Requires only the `CLASPRC\_JSON` repo secret; skips cleanly if unset.)**
- [x] **Cross-run dedup by message/attachment ID.** A bounded FIFO of processed Gmail message ids (`src/processed.ts`, capped at `MAX\_PROCESSED\_IDS`) is persisted in user properties; `updateDrive` skips messages already in the set and reports newly-completed ids back to `sync.ts`. A message is only marked done when all its attachments were copied cleanly (not truncated by the daily cap or a transient error), so re-scanned windows skip the Drive `getFilesByName` probe. **(done; `src/processed.ts`, `src/drive.ts`, `src/sync.ts`, `src/state.ts`)**
- [x] **Observability.** Level-prefixed structured logging (`src/logger.ts`), per-day run stats accumulator (`src/stats.ts` ? runs/files/errors/last error, persisted via `PROP\_RUN\_STATS`), and an opt-in daily summary (`src/notify.ts`) logged on each new-day rollover. The summary email is OFF by default; enabling it needs `ENABLE\_SUMMARY\_EMAIL` + the `script.send\_mail` scope. **(done)**

Larger / architectural

- [x] **Hybrid incremental sync.** Keep the date-window scan for the initial backfill; once caught up (`isToday`), switch the steady state to the Gmail advanced service + `history.list`/`startHistoryId`, storing `historyId` in properties and falling back to full sync on 404. **(done; pure logic in `src/historySync.ts`, advanced-service shell in `src/gmailHistory.ts`, wired into `src/sync.ts`. Gated behind `ENABLE\_INCREMENTAL\_SYNC` (OFF by default) for gradual rollout over the proven backfill. The Gmail advanced service is declared in `appsscript.json`; no new OAuth scope ? `gmail.readonly` covers history reads. Backfill captures the mailbox `historyId` before processing so mail arriving mid-run is still caught; expiry (HTTP 404) clears the cursor and rewinds to a bounded date-window re-scan.)**
- [] **More structure options & Shared Drive support** ? by-sender, nested `Year/Month`, label-driven folders; currently My Drive only.

Open-source & auditability readiness

Concrete prep so the [auditability invariant](docs/ARCHITECTURE.md#auditability-invariant-binding)

33. user (2026-06-10T16:16:18.161Z)

```
__pycache__
conftest.py
ingestion
parsers
synthpdf.py
test_cli.py
test_db.py
test_db_crypto.py
test_db_crypto_ops.py
test_db_invariants.py
test_dependency_licenses.py
test_errors.py
test_eval_cli.py
test_extract.py
test_log.py
test_migrations.py
test_ocr.py
test_offers.py
test_password_rules.py
test_precommit_hook.py
test_promo_regression.py
test_provenance.py
test_recovery.py
test_redaction.py
test_regression_synth.py
test_repo_hygiene.py
test_secrets.py
test_smoke.py
test_statement_unlock.py
test_unlock.py
test_verifier_properties.py
===
===(empty means none tracked)
1: ""Layer-1 regression CI ? full-pipeline fixtures on synthetic born-digital PDFs.
5: covers the extraction / coordinate-clustering / engine layers end-to-end, the gap the
synthetic
94:     assert parser is not None, "no parser routed this synthetic statement"
===
tests/test_repo_hygiene.py:15:FORBIDDEN_PREFIXES = ("testdata/statements/", "testdata/private/")
tests/test_unlock.py:57:@pytest.mark.local_statements
pyproject.toml:84:# Default runs are offline and deterministic: the ocr/llm/local_statements
tiers
pyproject.toml:86:adopto = "-ra -m 'not ocr and not llm and not local_statements'"
pyproject.toml:91:     "local_statements: reads gitignored real statements (local only)",
```

34. user (2026-06-10T16:16:32.145Z)

```
1     """Security guard: no sensitive paths may be tracked in git.
2
3     Encodes DESIGN.md section 5 ("Nothing sensitive ever in git") and docs/TESTING.md
4     section 9 as a failing-build assertion. If this test fails, something that must
5     stay local has been committed.
6     """
7
8     from __future__ import annotations
9
10    import subprocess
11    from pathlib import Path
12
13    REPO_ROOT = Path(__file__).resolve().parents[1]
14
15    FORBIDDEN_PREFIXES = ("testdata/statements/", "testdata/private/")
16    FORBIDDEN_SUFFIXES = (".db", ".sqlite", ".sqlite3", ".env", ".pem", ".key")
```

```

17     FORBIDDEN_NAMES = (
18         "token.json",
19         "token.pickle",
20         "credentials.json",
21         "password_rules.yaml",
22         "password_rules.yml",
23         "password_rules.json",
24         "passwords.txt",
25     )
26
27
28     def _tracked_files() -> list[str]:
29         out = subprocess.run(
30             ["git", "ls-files"],
31             cwd=REPO_ROOT,
32             capture_output=True,
33             text=True,
34             check=True,
35         )
36         return [line.strip() for line in out.stdout.splitlines() if line.strip()]
37
38
39     def test_no_sensitive_paths_tracked():
40         offenders = []
41         for path in _tracked_files():
42             low = path.lower()
43             if (
44                 low.startswith(FORBIDDEN_PREFIXES)
45                 or low.endswith(FORBIDDEN_SUFFIXES)
46                 or Path(path).name.lower() in FORBIDDEN_NAMES
47             ):
48                 offenders.append(path)
49         assert not offenders, f"sensitive paths must never be tracked: {offenders}"
50
51
52     def test_only_corpus_pdfs_are_tracked():
53         """Statements are PDFs ? the only PDFs git may track are the non-sensitive promo
54 corpus.
55
56 This is the guard that would have caught real HDFC statements being committed to
57 testdata/hdfc-statements/.
58 """
59         offenders = [
60             path
61             for path in _tracked_files()
62             if path.lower().endswith(".pdf") and not path.startswith("testdata/promo-
63 emails/")
64         ]
65         assert not offenders, f"only testdata/promo-emails/ PDFs may be tracked; found:
66 {offenders}"
67
68
69     def test_no_noncorpus_testdata_is_tracked():
70         """Mirror of the pre-commit guard: under testdata/, only the non-sensitive
71 promo-emails/ and synthetic/ corpora may be tracked ? nothing else (real
72 statements, zips, credentials, snapshots all stay local)."""
73         allowed = ("testdata/promo-emails/", "testdata/synthetic/")
74         offenders = [
75             path
76             for path in _tracked_files()
77             if path.startswith("testdata/") and not path.startswith(allowed)
78         ]
79         assert not offenders, (
80             f"only promo-emails/ and synthetic/ may be tracked under testdata/; found:
81 {offenders}"

```

78)
79

35. user (2026-06-10T16:16:34.400Z)

```
#!/usr/bin/env python3
"""muneem sensitive-file guard (invoked by the pre-commit hook).
```

```
Refuses to *stage* files that policy says must never be committed: real
statements (any PDF, or anything under ``testdata/`` outside the allowed
corpora), OAuth secrets, password rules, local databases, key material, and
``.env`` files. This is the PRE-commit backstop to ``.gitignore`` ? it stops the
``git add -f`` / ``git add -A`` mistake before it becomes a commit (the very
thing that caused an earlier statement leak). The post-commit complement is
``tests/test_repo_hygiene.py``.
```

```
Exit 0 when nothing sensitive is staged, 1 otherwise. A deliberate, reviewed
exception can be committed with ``git commit --no-verify``.
"""
```

```
from __future__ import annotations
```

```
import fnmatch
import os
import subprocess
import sys
```

Only these subtrees of testdata/ are non-sensitive and may be committed.

```
ALLOWED_PREFIXES = ("testdata/promo-emails/", "testdata/synthetic/")
SECRET_NAMES = {"credentials.json", "token.json", "token.pickle", "passwords.txt"}
```

```
def is_sensitive(path: str) -> str | None:
    """Return a human-readable reason if ``path`` must not be committed, else None."""
    p = path.replace("\\", "/").strip()
    if not p or p.startswith(ALLOWED_PREFIXES):
        return None
    name = p.rsplit("/", 1)[-1]
    low = p.lower()
    nlow = name.lower()
    if p.startswith("testdata/"):
        return "under testdata/ ? sensitive by default (only promo-emails/ and synthetic/
allowed)"
    if low.endswith(".pdf"):
        return "PDF outside the allowed corpora (real statements must stay local)"
    if low.endswith((".db", ".sqlite", ".sqlite3")):
        return "local database (may contain parsed statement data)"
    if low.endswith(".plaintext.bak"):
        return "plaintext database backup (cleartext from 'muneem db encrypt' migration)"
    if fnmatch.fnmatch(nlow, "*recovery*.json"):
        return "DB-key recovery file (passphrase-wrapped key ? keep it off the repo)"
    if low.endswith((".pem", ".key")):
        return "key material"
    if nlow in SECRET_NAMES:
        return "OAuth / credential secret"
    if nlow.startswith("oauth_token"):
        return "OAuth token"
    # Block the rules *data* file (password_rules.yaml/.yaml/.json/?) but not the loader module
    # (src/muneem/password_rules.py is code, not a secret).
    if fnmatch.fnmatch(nlow, "password_rules.*") and not nlow.endswith(".py"):
        return "password rules"
    if (nlow == ".env" or nlow.startswith(".env.")) and nlow != ".env.example":
        return ".env file (may contain secrets)"
    return None
```



```

===
15:from synthpdf import at, build_pdf, right
103:def test_hdfc_layout_full_pipeline_reconciles(tmp_path):
104:    name, n, status = _run_pipeline(_hdfc_pdf(tmp_path / "hdfc.pdf"))
110:def test_savings_layout_full_pipeline_reconciles(tmp_path):
111:    name, n, status = _run_pipeline(_au_savings_pdf(tmp_path / "au.pdf"))
===
ci.yml
12:  runs-on: ${ matrix.os }
15:  matrix:
16:    os: [ubuntu-latest]
21:      - os: windows-latest
27:    python-version: ${ matrix.python-version }

```

36. user (2026-06-10T16:16:47.148Z)

```

1      """Central, redaction-aware logging.
2
3      Use ``get_logger(__name__)`` instead of the stdlib ``logging.getLogger`` so every muneem
logger
4      carries a :class:`RedactingFilter`. The filter scrubs money-/account-like substrings
from each
5      record as a backstop ? code should still pass already-sanitised context (see
6      :mod:`muneem.redaction`); this is defense-in-depth so an accidental sensitive f-string
cannot leak.
7
8      muneem stays library-style: we attach the filter but do not configure handlers or
levels (the
9      application / CLI owns that), so importing muneem never hijacks the root logger.
10     """
11
12     from __future__ import annotations
13
14     import logging
15
16     from muneem.redaction import redact_text
17
18
19     class RedactingFilter(logging.Filter):
20         """A logging filter that masks sensitive substrings in the formatted message."""
21
22         def filter(self, record: logging.LogRecord) -> bool:
23             try:
24                 message = record.getMessage()
25                 except Exception: # never drop a record just because formatting failed
26                     return True
27                 redacted = redact_text(message)
28                 if redacted != message:
29                     record.msg = redacted
30                     record.args = ()
31                 return True
32
33
34     _filtered: set[str] = set()
35
36
37     def get_logger(name: str) -> logging.Logger:
38         """Return a logger that redacts sensitive substrings from its records
(idempotent)."""
39         logger = logging.getLogger(name)
40         if name not in _filtered:
41             logger.addFilter(RedactingFilter())
42             _filtered.add(name)
43         return logger
44

```

37. user (2026-06-10T16:16:49.990Z)

98481ee feat(db): provenance table (first forward migration) + record parse provenance (#46)
e285b98 docs: Open Banking (AA) research + decisions (AA deferred, flat-schema, SMS) (#45)
22a34de Add hand-rolled, checksummed schema migration runner (P6 / decision #3) (#44)
7fed5d4 Add passphrase-wrapped disaster recovery for the ledger key (resolves P0) (#43)
4fb78c0 ci: test on Windows too (the dev platform) (#42)
cc20aa0 docs: land feedbackJune2026 plan + schema.md; mark Outlook/IMAP experimental (#41)
3a659f7 Add coverage gate (>80%) + verifier property/fuzz tests (#40)
0f8d099 Strengthen DB crypto: rekey/rotate, encrypted backup, migration cleanup (#39)
e884196 Add mypy type-checking (CI + pre-push) and fix surfaced type errors (#38)
9f9de66 Harden secrets / password handling + attribute external calls (#37)
87b8271 Add error hierarchy + redaction + redacting logger; clean parser errors (#36)
dcfd7a7 Add pre-push gate running lint + tests (local enforcement) (#35)
e9c6244 Encrypt the local ledger at rest (SQLCipher) ? resolves D.2 (#34)
9c13e69 feat(cli): `muneem eval` seed?golden onboarding harness + playbook (#33)
02cc372 feat(parsers): balance-delta reconstruction recovers Axis June 2025 (5/7 -> 6/7) (#32)

===

68

===

6:subcommands (`fetch`, `txns`) land with their milestones.
src/muneem/ingestion/gmail.py
src/muneem/ingestion/__init__.py
src/muneem/ingestion/__pycache__/gmail.cpython-313.pyc
src/muneem/ingestion/__pycache__/__init__.cpython-313.pyc
tools/gmail_auth.py
tests/ingestion/test_gmail.py
tests/ingestion/test_providers.py
tests/ingestion/__pycache__/test_gmail.cpython-313-pytest-9.0.3.pyc
tests/ingestion/__pycache__/test_providers.cpython-313-pytest-9.0.3.pyc

38. user (2026-06-10T16:17:01.093Z)

Architecture

> Status: living document. Captures the **target architecture** for the
> extraction pivot and the **decisions** behind it. Product framing is in
> [`VISION.md`](./VISION.md). This supersedes the original "just copy
> attachments" mental model; the copier survives as one extractor at the leaf.

Auditability invariant (binding)

This repository is the only component that reads or processes a user's Gmail, and it must remain auditable and self-deployable by a power user at any time. This is the product's core privacy guarantee and a deliberate constraint on every future decision (see ADR-008). It is why the project is built to be open-sourced: **don't trust us ? read, and run, the code.**

Every change **MUST** preserve all of the following. A change that cannot is, by definition, a decision to break the invariant and needs explicit owner sign-off.

1. **Sole Gmail surface.** Gmail is read/processed *only* here, in the user's own account (`executeAs: USER_ACCESSING`). No other component ? Muneem included ? ever holds the Gmail OAuth grant or sees raw mail. Muneem may *orchestrate* this app; it is never *itself* the Gmail reader.
2. **Egress == the audited boundary.** Nothing derived from a user's mail leaves their Google account except through the in-repo **REDACT/MINIMIZE** stage, carrying only minimized, non-PII signal, only to the user's own hosted service, never to third parties. The set of egress destinations is explicit and reviewable in source.
3. **Run-your-own-from-source.** A power user can build and deploy their own instance from the published source and obtain an equivalent app; the build is reproducible and the verification steps are documented ? "trust by running the code you read," not merely "read the code."

4. ****No secrets in source.**** Credentials exist only as deploy-time secrets; the repository contains nothing sensitive.
5. ****Transparent over clever**** at the read and egress/redaction boundaries ? prefer obviously-correct, reviewable code even at some cost to brevity.

****PR review checklist (apply to every change):****

- [] Gmail access stays entirely within this repo, in-account?
- [] If any data leaves the account, does it pass through REDACT/MINIMIZE and go only to the user's own service (non-PII)?
- [] Build still reproducible and self-deployable from source?
- [] No new secrets, third-party endpoints, or network calls? (Default: none ? enforced by `tests/auditability.test.ts`.)

Principles (preserved from today's codebase)

1. ****Shells around pure functions.**** GAS-facing code is a thin shell; all logic is pure and unit-tested.
2. ****Quota discipline.**** Respect the 6-minute execution limit, the 90-min/day trigger budget, and per-user caps. Self-healing trigger fan-out; per-user lock; "mark done only on clean completion."
3. ****Per-user isolation.**** Runs as the accessing user; state is per-user.
4. ****Versioning & provenance everywhere.**** Every persisted shape carries a `schemaVersion`; every extracted entity carries where it came from and which extractor (and version) produced it.

The pipeline

This repo owns the left half; downstream services own the right half. They meet at the ****handoff contract**** only.

```

...
    ?????????????????????? this repo ?????????????????????? ?? downstream ??
Gmail ??? INGEST ??? CLASSIFY ??? EXTRACT ??? REDACT/MINIMIZE ??? HANDOFF ??? Muneem /
(rich      (type +      (normalize (strip PII,      (sink)      grocery /
EmailMsg) confidence) light) encrypt) coupon svc
?
???? RAW ARCHIVE (Drive folders ? optional per type later) encrypted
???? LOCAL INDEX (Store interface; Sheets now, remote later) store,
=====
nonPiiFields, // minimized, scrubbed signal only
// encrypted in transit; PII stripped or tokenized
}
...

```

- ****Entity keys**** give content-stable dedup (re-mailed receipts; cross-source fusion of an email confirmation + a PDF ticket ? one entity).
- ****Attachment `sha256`**** enables content-addressable dedup of re-sent files.
- ****Provenance + `extractorVersion`**** make "this extraction is stale ? re-run" possible and auditable.

The extraction ? processing boundary

- This repo produces a normalized, classified, ****redacted**** `HandoffPayload` per detected document/entity and routes it to a downstream ****sink****.
- Downstream services (Muneem for finance; siblings for groceries/coupons) own domain parsing, indexing, ****encrypted storage****, and analytics.
- ****Contract** is owned jointly, defined from our side first.****** The exact Muneem-facing schema and transport are ****TBD** and aligned with that repo****** (this repo's GitHub tooling is scoped to itself, so the contract is specified here and reconciled with Muneem separately).
- Hard rule: this repo imports ****no**** domain-parsing logic; downstream reads ****no**** Gmail.

Storage abstraction (decision: extensible, not baked-in)

A single `Store` / `Sink` interface; code depends only on the interface.

```
Impl	When	Properties
`GoogleSheetsStore` (user-owned "LifeIndex" sheet)	v1	in-account, queryable, zero infra,
doubles as a first dashboard		
`RemoteStore` (our hosted DB) / Muneem-API sink	later	the durable system of record
```

`PropertiesService` stays only for small per-user **control state** (cursors, flags, counters) ? never the entity index.

Privacy & egress controls

- **Egress** only to our own isolated, hosted service. No third parties.
 - A **Redactor/Minimizer** stage runs before any egress: strip/tokenize PII, keep only non-PII signal, encrypt in transit (encrypted at rest downstream).
- =====
- Per-**classifier** and per-**extractor** success / failure / ambiguity rates ? not just "files added."
 - Surfaced **review queues**: "matched no extractor" and "low confidence."
 - Structured logs carrying `messageId`; extraction & handoff metrics.

Platform spectrum

1. **Pure-GAS conservative extractors** (regex, known formats `.ics`/`.csv`, native Drive OCR via Doc conversion) for the easy ~70%, **in-account**.
2. **Hosted, isolated service** (Muneem pattern) for heavy/ML extraction, storage, and analytics ? fed only the minimized, non-PII handoff.

Decision log

```
#	Decision	Rationale
ADR-001	Pipeline (ingest ? classify ? extract ? redact ? handoff), not a copy loop	Treat mail as structured data; make extraction re-runnable over a durable record
ADR-002	Extraction (this repo) is decoupled from processing (Muneem & siblings)	Isolate domain logic + encrypted storage + analytics downstream; clean contract seam
ADR-003	Storage via a `Store`/`Sink` interface ? Sheets v1, remote/Muneem later	Extensible by design; never baked into the code
ADR-004	Egress: non-PII only, to our own isolated hosted service, no third parties; scrub + encrypt; minimize	Privacy north star; preserves trust; in-account baseline needs no egress
ADR-005	Audience: <100 (Testing) now, public later; CASA deferred	Harden + learn value before investing in assessment/infra
ADR-006	First verticals: grocery receipts + bank statements (? Muneem)	Volume + the high-fidelity handoff path
ADR-007	Preserve current strengths (pure/shell, quotas, triggers, per-user, versioning/provenance)	The recent rewrite is exactly why this pivot is feasible
ADR-008	Option A: this repo is the sole Gmail-touching component ? in-account, auditable, self-deployable; Muneem orchestrates but never holds the Gmail grant. Codified as the binding [auditability invariant](#auditability-invariant-binding).	Verifiable privacy is the product's moat and shrinks future verification scope; it concentrates the trust-critical surface in one open, in-account place.
```

Open items for the next iteration

- **Muneem handoff contract:** schema + transport (e.g. `UrlFetch` to a signed endpoint vs. write-to-a-shared-store) ? define jointly with `avidullu/muneem`.
- **Per-vertical "non-PII" definition** for receipts and bank statements (what is sent vs. tokenized vs. kept in-account).

```
a764030 2026-06-03 Codify the auditability invariant (Option A) + enforce no egress (#14)
90c85ba 2026-06-03 Merge pull request #13 from avidullu/docs/vision-and-architecture
4975254 2026-06-02 docs: capture extraction-pivot vision and architecture
172b3b3 2026-06-02 Validate setup input + traceable attachment-copy errors (#12)
964b58c 2026-06-02 Portable attachment names + robust content-type matching (#11)
5e934b1 2026-05-31 Make the web app multi-user-ready for small-group / org deployment (#10)
e4362a3 2026-05-31 Add hybrid history.list incremental sync (opt-in) (#9)
6d0b10e 2026-05-31 ci: trigger deploy to verify CLASPRC_JSON (v2 format)
===
(empty = no mention)
```

40. user (2026-06-10T16:19:08.866Z)

```
Exit code 2
/usr/bin/bash: line 1: $null: ambiguous redirect
src/muneem/statement_unlock.py
src/muneem/unlock.py
tests/test_statement_unlock.py
tests/test_unlock.py
===
3:on:
4- # All-branch pushes so feature branches are gated even before a PR exists
5- # (the regression fixtures in tests/test_regression_synth.py run here, secret-free).
6- push:
7-   pull_request:
8-   workflow_dispatch:
9-
10-
11-
12-   runs-on: ${{ matrix.os }}
13-   strategy:
14-     fail-fast: false
15-     matrix:
16-
17-
18-
19-
20-
21-
22-
23-
24-
25-
26-
27-
28-
29-
30-
31-
32-
33-
34-
35-
36-
37-
38-
39-
40-
41-
42-
43-
44-
45-
46-
47-
48-
49-
50-
51-
52-
53-
54-
55-
56-
57-
58-
59-
60-
61-
62-
63-
64-
65-
66-
67-
68-
69-
70-
71-
72-
73-
74-
75-
76-
77-
78-
79-
80-
81-
82-
83-
84-
85-
86-
87-
88-
89-
90-
91-
92-
93-
94-
95-
96-
97-
98-
99-
100-
101-
102-
103-
104-
105-
106-
107-
108-
109-
110-
111-
112-
113-
114-
115-
116-
117-
118-
119-
120-
121-
122-
123-
124-
125-
126-
127-
128-
129-
130-
131-
132-
133-
134-
135-
136-
137-
138-
139-
140-
141-
142-
143-
144-
145-
146-
147-
148-
149-
150-
151-
152-
153-
154-
155-
156-
157-
158-
159-
160-
161-
162-
163-
164-
165-
166-
167-
168-
169-
170-
171-
172-
173-
174-
175-
176-
177-
178-
179-
180-
181-
182-
183-
184-
185-
186-
187-
188-
189-
190-
191-
192-
193-
194-
195-
196-
197-
198-
199-
200-
201-
202-
203-
204-
205-
206-
207-
208-
209-
210-
211-
212-
213-
214-
215-
216-
217-
218-
219-
220-
221-
222-
223-
224-
225-
226-
227-
228-
229-
230-
231-
232-
233-
234-
235-
236-
237-
238-
239-
240-
241-
242-
243-
244-
245-
246-
247-
248-
249-
250-
251-
252-
253-
254-
255-
256-
257-
258-
259-
260-
261-
262-
263-
264-
265-
266-
267-
268-
269-
270-
271-
272-
273-
274-
275-
276-
277-
278-
279-
280-
281-
282-
283-
284-
285-
286-
287-
288-
289-
290-
291-
292-
293-
294-
295-
296-
297-
298-
299-
300-
301-
302-
303-
304-
305-
306-
307-
308-
309-
310-
311-
312-
313-
314-
315-
316-
317-
318-
319-
320-
321-
322-
323-
324-
325-
326-
327-
328-
329-
330-
331-
332-
333-
334-
335-
336-
337-
338-
339-
340-
341-
342-
343-
344-
345-
346-
347-
348-
349-
350-
351-
352-
353-
354-
355-
356-
357-
358-
359-
360-
361-
362-
363-
364-
365-
366-
367-
368-
369-
370-
371-
372-
373-
374-
375-
376-
377-
378-
379-
380-
381-
382-
383-
384-
385-
386-
387-
388-
389-
390-
391-
392-
393-
394-
395-
396-
397-
398-
399-
400-
401-
402-
403-
404-
405-
406-
407-
408-
409-
410-
411-
412-
413-
414-
415-
416-
417-
418-
419-
420-
421-
422-
423-
424-
425-
426-
427-
428-
429-
430-
431-
432-
433-
434-
435-
436-
437-
438-
439-
440-
441-
442-
443-
444-
445-
446-
447-
448-
449-
450-
451-
452-
453-
454-
455-
456-
457-
458-
459-
460-
461-
462-
463-
464-
465-
466-
467-
468-
469-
470-
471-
472-
473-
474-
475-
476-
477-
478-
479-
480-
481-
482-
483-
484-
485-
486-
487-
488-
489-
490-
491-
492-
493-
494-
495-
496-
497-
498-
499-
500-
501-
502-
503-
504-
505-
506-
507-
508-
509-
510-
511-
512-
513-
514-
515-
516-
517-
518-
519-
520-
521-
522-
523-
524-
525-
526-
527-
528-
529-
530-
531-
532-
533-
534-
535-
536-
537-
538-
539-
540-
541-
542-
543-
544-
545-
546-
547-
548-
549-
550-
551-
552-
553-
554-
555-
556-
557-
558-
559-
560-
561-
562-
563-
564-
565-
566-
567-
568-
569-
570-
571-
572-
573-
574-
575-
576-
577-
578-
579-
580-
581-
582-
583-
584-
585-
586-
587-
588-
589-
590-
591-
592-
593-
594-
595-
596-
597-
598-
599-
600-
601-
602-
603-
604-
605-
606-
607-
608-
609-
610-
611-
612-
613-
614-
615-
616-
617-
618-
619-
620-
621-
622-
623-
624-
625-
626-
627-
628-
629-
630-
631-
632-
633-
634-
635-
636-
637-
638-
639-
640-
641-
642-
643-
644-
645-
646-
647-
648-
649-
650-
651-
652-
653-
654-
655-
656-
657-
658-
659-
660-
661-
662-
663-
664-
665-
666-
667-
668-
669-
670-
671-
672-
673-
674-
675-
676-
677-
678-
679-
680-
681-
682-
683-
684-
685-
686-
687-
688-
689-
690-
691-
692-
693-
694-
695-
696-
697-
698-
699-
700-
701-
702-
703-
704-
705-
706-
707-
708-
709-
710-
711-
712-
713-
714-
715-
716-
717-
718-
719-
720-
721-
722-
723-
724-
725-
726-
727-
728-
729-
730-
731-
732-
733-
734-
735-
736-
737-
738-
739-
740-
741-
742-
743-
744-
745-
746-
747-
748-
749-
750-
751-
752-
753-
754-
755-
756-
757-
758-
759-
760-
761-
762-
763-
764-
765-
766-
767-
768-
769-
770-
771-
772-
773-
774-
775-
776-
777-
778-
779-
780-
781-
782-
783-
784-
785-
786-
787-
788-
789-
790-
791-
792-
793-
794-
795-
796-
797-
798-
799-
800-
801-
802-
803-
804-
805-
806-
807-
808-
809-
810-
811-
812-
813-
814-
815-
816-
817-
818-
819-
820-
821-
822-
823-
824-
825-
826-
827-
828-
829-
830-
831-
832-
833-
834-
835-
836-
837-
838-
839-
840-
841-
842-
843-
844-
845-
846-
847-
848-
849-
850-
851-
852-
853-
854-
855-
856-
857-
858-
859-
860-
861-
862-
863-
864-
865-
866-
867-
868-
869-
870-
871-
872-
873-
874-
875-
876-
877-
878-
879-
880-
881-
882-
883-
884-
885-
886-
887-
888-
889-
890-
891-
892-
893-
894-
895-
896-
897-
898-
899-
900-
901-
902-
903-
904-
905-
906-
907-
908-
909-
910-
911-
912-
913-
914-
915-
916-
917-
918-
919-
920-
921-
922-
923-
924-
925-
926-
927-
928-
929-
930-
931-
932-
933-
934-
935-
936-
937-
938-
939-
940-
941-
942-
943-
944-
945-
946-
947-
948-
949-
950-
951-
952-
953-
954-
955-
956-
957-
958-
959-
960-
961-
962-
963-
964-
965-
966-
967-
968-
969-
970-
971-
972-
973-
974-
975-
976-
977-
978-
979-
980-
981-
982-
983-
984-
985-
986-
987-
988-
989-
990-
991-
992-
993-
994-
995-
996-
997-
998-
999-
1000-
1001-
1002-
1003-
1004-
1005-
1006-
1007-
1008-
1009-
1010-
1011-
1012-
1013-
1014-
1015-
1016-
1017-
1018-
1019-
1020-
1021-
1022-
1023-
1024-
1025-
1026-
1027-
1028-
1029-
1030-
1031-
1032-
1033-
1034-
1035-
1036-
1037-
1038-
1039-
1040-
1041-
1042-
1043-
1044-
1045-
1046-
1047-
1048-
1049-
1050-
1051-
1052-
1053-
1054-
1055-
1056-
1057-
1058-
1059-
1060-
1061-
1062-
1063-
1064-
1065-
1066-
1067-
1068-
1069-
1070-
1071-
1072-
1073-
1074-
1075-
1076-
1077-
1078-
1079-
1080-
1081-
1082-
1083-
1084-
1085-
1086-
1087-
1088-
1089-
1090-
1091-
1092-
1093-
1094-
1095-
1096-
1097-
1098-
1099-
1100-
1101-
1102-
1103-
1104-
1105-
1106-
1107-
1108-
1109-
1110-
1111-
1112-
1113-
1114-
1115-
1116-
1117-
1118-
1119-
1120-
1121-
1122-
1123-
1124-
1125-
1126-
1127-
1128-
1129-
1130-
1131-
1132-
1133-
1134-
1135-
1136-
1137-
1138-
1139-
1140-
1141-
1142-
1143-
1144-
1145-
1146-
1147-
1148-
1149-
1150-
1151-
1152-
1153-
1154-
1155-
1156-
1157-
1158-
1159-
1160-
1161-
1162-
1163-
1164-
1165-
1166-
1167-
1168-
1169-
1170-
1171-
1172-
1173-
1174-
1175-
1176-
1177-
1178-
1179-
1180-
1181-
1182-
1183-
1184-
1185-
1186-
1187-
1188-
1189-
1190-
1191-
1192-
1193-
1194-
1195-
1196-
1197-
1198-
1199-
1200-
1201-
1202-
1203-
1204-
1205-
1206-
1207-
1208-
1209-
1210-
1211-
1212-
1213-
1214-
1215-
1216-
1217-
1218-
1219-
1220-
1221-
1222-
1223-
1224-
1225-
1226-
1227-
1228-
1229-
1230-
1231-
1232-
1233-
1234-
1235-
1236-
1237-
1238-
1239-
1240-
1241-
1242-
1243-
1244-
1245-
1246-
1247-
1248-
1249-
1250-
1251-
1252-
1253-
1254-
1255-
1256-
1257-
1258-
1259-
1260-
1261-
1262-
1263-
1264-
1265-
1266-
1267-
1268-
1269-
1270-
1271-
1272-
1273-
1274-
1275-
1276-
1277-
1278-
1279-
1280-
1281-
1282-
1283-
1284-
1285-
1286-
1287-
1288-
1289-
1290-
1291-
1292-
1293-
1294-
1295-
1296-
1297-
1298-
1299-
1300-
1301-
1302-
1303-
1304-
1305-
1306-
1307-
1308-
1309-
1310-
1311-
1312-
1313-
1314-
1315-
1316-
1317-
1318-
1319-
1320-
1321-
1322-
1323-
1324-
1325-
1326-
1327-
1328-
1329-
1330-
1331-
1332-
1333-
1334-
1335-
1336-
1337-
1338-
1339-
1340-
1341-
1342-
1343-
1344-
1345-
1346-
1347-
1348-
1349-
1350-
1351-
1352-
1353-
1354-
1355-
1356-
1357-
1358-
1359-
1360-
1361-
1362-
1363-
1364-
1365-
1366-
1367-
1368-
1369-
1370-
1371-
1372-
1373-
1374-
1375-
1376-
1377-
1378-
1379-
1380-
1381-
1382-
1383-
1384-
1385-
1386-
1387-
1388-
1389-
1390-
1391-
1392-
1393-
1394-
1395-
1396-
1397-
1398-
1399-
1400-
1401-
1402-
1403-
1404-
1405-
1406-
1407-
1408-
1409-
1410-
1411-
1412-
1413-
1414-
1415-
1416-
1417-
1418-
1419-
1420-
1421-
1422-
1423-
1424-
1425-
1426-
1427-
1428-
1429-
1430-
1431-
1432-
1433-
1434-
1435-
1436-
1437-
1438-
1439-
1440-
1441-
1442-
1443-
1444-
1445-
1446-
1447-
1448-
1449-
1450-
1451-
1452-
1453-
1454-
1455-
1456-
1457-
1458-
1459-
1460-
1461-
1462-
1463-
1464-
1465-
1466-
1467-
1468-
1469-
1470-
1471-
1472-
1473-
1474-
1475-
1476-
1477-
1478-
1479-
1480-
1481-
1482-
1483-
1484-
1485-
1486-
1487-
1488-
1489-
1490-
1491-
1492-
1493-
1494-
1495-
1496-
1497-
1498-
1499-
1500-
1501-
1502-
1503-
1504-
1505-
1506-
1507-
1508-
1509-
1510-
1511-
1512-
1513-
1514-
1515-
1516-
1517-
1518-
1519-
1520-
1521-
1522-
1523-
1524-
1525-
1526-
1527-
1528-
1529-
1530-
1531-
1532-
1533-
1534-
1535-
1536-
1537-
1538-
1539-
1540-
1541-
1542-
1543-
1544-
1545-
1546-
1547-
1548-
1549-
1550-
1551-
1552-
1553-
1554-
1555-
1556-
1557-
1558-
1559-
1560-
1561-
1562-
1563-
1564-
1565-
1566-
1567-
1568-
1569-
1570-
1571-
1572-
1573-
1574-
1575-
1576-
1577-
1578-
1579-
1580-
1581-
1582-
1583-
1584-
1585-
1586-
1587-
1588-
1589-
1590-
1591-
1592-
1593-
1594-
1595-
1596-
1597-
1598-
1599-
1600-
1601-
1602-
1603-
1604-
1605-
1606-
1607-
1608-
1609-
1610-
1611-
1612-
1613-
1614-
1615-
1616-
1617-
1618-
1619-
1620-
1621-
1622-
1623-
1624-
1625-
1626-
1627-
1628-
1629-
1630-
1631-
1632-
1633-
1634-
1635-
1636-
1637-
1638-
1639-
1640-
1641-
1642-
1643-
1644-
1645-
1646-
1647-
1648-
1649-
1650-
1651-
1652-
1653-
1654-
1655-
1656-
1657-
1658-
1659-
1660-
1661-
1662-
1663-
1664-
1665-
1666-
1667-
1668-
1669-
1670-
1671-
1672-
1673-
1674-
1675-
1676-
1677-
1678-
1679-
1680-
1681-
1682-
1683-
1684-
1685-
1686-
1687-
1688-
1689-
1690-
1691-
1692-
1693-
1694-
1695-
1696-
1697-
1698-
1699-
1700-
1701-
1702-
1703-
1704-
1705-
1706-
1707-
1708-
1709-
1710-
1711-
1712-
1713-
1714-
1715-
1716-
1717-
1718-
1719-
1720-
1721-
1722-
1723-
1724-
1725-
1726-
1727-
1728-
1729-
1730-
1731-
1732-
1733-
1734-
1735-
1736-
1737-
1738-
1739-
1740-
1741-
1742-
1743-
1744-
1745-
1746-
1747-
1748-
1749-
1750-
1751-
1752-
1753-
1754-
1755-
1756-
1757-
1758-
1759-
1760-
1761-
1762-
1763-
1764-
1765-
1766-
1767-
1768-
1769-
1770-
1771-
1772-
1773-
1774-
1775-
1776-
1777-
1778-
1779-
1780-
1781-
1782-
1783-
1784-
1785-
1786-
1787-
1788-
1789-
1790-
1791-
1792-
1793-
1794-
1795-
1796-
1797-
1798-
1799-
1800-
1801-
1802-
1803-
1804-
1805-
1806-
1807-
1808-
1809-
1810-
1811-
1812-
1813-
1814-
1815-
1816-
1817-
1818-
1819-
1820-
1821-
1822-
1823-
1824-
1825-
1826-
1827-
1828-
1829-
1830-
1831-
1832-
1833-
1834-
1835-
1836-
1837-
1838-
1839-
1840-
1841-
1842-
1843-
1844-
1845-
1846-
1847-
1848-
1849-
1850-
1851-
1852-
1853-
1854-
1855-
1856-
1857-
1858-
1859-
1860-
1861-
1862-
1863-
1864-
1865-
1866-
1867-
1868-
1869-
1870-
1871-
1872-
1873-
1874-
1875-
1876-
1877-
1878-
1879-
1880-
1881-
1882-
1883-
1884-
1885-
1886-
1887-
1888-
1889-
1890-
1891-
1892-
1893-
1894-
1895-
1896-
1897-
1898-
1899-
1900-
1901-
1902-
1903-
1904-
1905-
1906-
1907-
1908-
1909-
1910-
1911-
1912-
1913-
1914-
1915-
1916-
1917-
1918-
1919-
1920-
1921-
1922-
1923-
1924-
1925-
1926-
1927-
1928-
1929-
1930-
1931-
1932-
1933-
1934-
1935-
1936-
1937-
1938-
1939-
1940-
1941-
1942-
1943-
1944-
1945-
1946-
1947-
1948-
1949-
1950-
1951-
1952-
1953-
1954-
1955-
1956-
1957-
1958-
1959-
1960-
1961-
1962-
1963-
1964-
1965-
1966-
1967-
1968-
1969-
1970-
1971-
1972-
1973-
1974-
1975-
1976-
1977-
1978-
1979-
1980-
1981-
1982-
1983-
1984-
1985-
1986-
1987-
1988-
1989-
1990-
1991-
1992-
1993-
1994-
1995-
1996-
1997-
1998-
1999-
2000-
2001-
2002-
2003-
2004-
2005-
2006-
2007-
2008-
2009-
2010-
2011-
2012-
2013-
2014-
2015-
2016-
2017-
2018-
2019-
2020-
2021-
2022-
2023-
2024-
2025-
2026-
2027-
2028-
2029-
2030-
2031-
2032-
2033-
2034-
2035-
2036-
2037-
2038-
2039-
2040-
2041-
2042-
2043-
2044-
2045-
2046-
2047-
2048-
2049-
2050-
2051-
2052-
2053-
2054-
2055-
2056-
2057-
2058-
2059-
2060-
2061-
2062-
2063-
2064-
2065-
2066-
2067-
2068-
2069-
2070-
2071-
2072-
2073-
2074-
2075-
2076-
2077-
2078-
2079-
2080-
2081-
2082-
2083-
2084-
2085-
2086-
2087-
2088-
2089-
2090-
2091-
2092-
2093-
2094-
2095-
2096-
2097-
2098-
2099-
2100-
2101-
2102-
2103-
2104-
2105-
2106-
2107-
2108-
2109-
2110-
2111-
2112-
2113-
2114-
2115-
2116-
2117-
2118-
2119-
2120-
2121-
2122-
2123-
2124-
2125-
2126-
2127-
2128-
2129-
2130-
2131-
2132-
2133-
2134-
2135-
2136-
2137-
2138-
2139-
2140-
2141-
2142-
2143-
2144-
2145-
2146-
2147-
2148-
2149-
2150-
2151-
2152-
2153-
2154-
2155-
2156-
2157-
2158-
2159-
2160-
2161-
2162-
2163-
2164-
2165-

```